Multi-Label Image Classification On Remote Sensing Imagery Using Deep Learning

Isaac Attard

Supervisor: Mr. Tony Spiteri Staines

June 2025

Submitted in partial fulfilment of the requirements
for the degree of Bachelor of Science (Honours) (Software Development).



L-Università ta' Malta
Faculty of Information &
Communication Technology



University of Malta Library - Electronic Thesis & Dissertations (ETD) Repository

The copyright of this thesis/dissertation belongs to the author. The author's rights in respect of this work are as defined by the Copyright Act (Chapter 415) of the Laws of Malta or as modified by any successive legislation.

Users may access this full-text thesis/dissertation and can make use of the information contained in accordance with the Copyright Act provided that the author must be properly acknowledged. Further distribution or reproduction in any format is prohibited without the prior permission of the copyright holder.

Abstract

Land cover classification plays a vital role in environmental monitoring, natural resource management, and urban planning, supporting a wide range of applications, from conservation efforts to sustainable land use [1]. Traditional manual interpretation of satellite imagery is often subjective, time-consuming, and unsuitable for large-scale applications. This project addresses these limitations by developing an automated land cover classification system based on Convolutional Neural Networks (CNNs) and multi-label, multi-spectral satellite imagery from the BigEarthNet [21] dataset. The proposed system is designed to classify multiple land cover types simultaneously, improving generalisation and predictive accuracy across diverse environmental conditions.

The resulting prototype demonstrates robust performance and is validated using quantitative metrics and practical evaluation scenarios. These results highlight the feasibility and effectiveness of deep-learning-based approaches for remote sensing applications. This work contributes to ongoing research in automated land cover classification and provides a scalable foundation for future advancements in Earth observation systems.

Acknowledgements

I want to express my gratitude to my supervisor, Mr. Tony Spiteri Staines, for his continuous support, invaluable guidance, and constructive feedback throughout the project development. His encouragement was essential for overcoming the challenges I faced and ensuring the successful completion of the project.

I am also deeply thankful to Prof. John Abela, whose assistance proved essential in conducting the experiments critical to this work. Without his support, these vital experiments would not have been possible.

Lastly, I would like to thank my family and girlfriend for their unwavering support and encouragement throughout this project. Their understanding and patience have been a constant source of motivation.

Contents

Abstract ..... **iii**

Acknowledgements ..... **iv**

Contents ..... **v**

List of Figures ..... **viii**

List of Tables ..... **x**

List of Abbreviations ..... **xi**

1. Introduction **1**
- 1.1. Problem Definition 1
- 1.2. Aims and Objectives 2
- 1.3. Dissertation Structure 3

2. Background & Literature Review **4**
- 2.1. Remote Sensing Foundations 4
  - 2.1.1. Remote Sensing Principles 4
  - 2.1.2. Electromagnetic Spectrum and Reflectance 5
  - 2.1.3. Resolution in Remote Sensing 5
- 2.2. Remote Sensing Datasets for LULC 6
  - 2.2.1. Pixel-Level Datasets 6
  - 2.2.2. Patch-Level Datasets 6
- 2.3. Land Use and Land Cover Classification 7
- 2.4. Deep Learning Techniques 8
  - 2.4.1. Foundations of Deep Learning and CNNs 8
  - 2.4.2. Advanced CNN Architectures 9
  - 2.4.3. Attention Mechanisms 9
- 2.5. Literature Review 10
  - 2.5.1. Introduction 10
  - 2.5.2. Traditional Techniques 11
  - 2.5.3. Rise of Deep Learning 11
  - 2.5.4. Relevant LULC Datasets 12
  - 2.5.5. LULC Deep Learning Techniques 14
  - 2.5.6. Key Findings, Gaps, & Conclusion 18

3. Specification & Design **19**
- 3.1. System Overview 19
- 3.2. Dataset and Preprocessing 19
  - 3.2.1.Dataset Selection 19
  - 3.2.2.Preprocessing Strategy 20
- 3.3. Model and Training 21
  - 3.3.1.Training and Optimisation Strategy 21
  - 3.3.2.Baseline Models 22
  - 3.3.3.Model Architecture 22

3.4. Evaluation and Interpretability 23

4. Implementation 25
4.1. System Setup and Environment 25
4.2. Dataset Preparation Pipeline 25
4.3. Training and Optimisation Pipeline 26
4.3.1. Training and Validation Process 27
4.3.2. Loss Function Implementation 27
4.3.3. Optimisation Techniques 28
4.3.4. Reproducibility and Code Organisation 29
4.4. Testing and Evaluation Pipeline 29
4.4.1. Performance Metrics and Per-Class Analysis 30
4.4.2. Visualisation and Interpretability 30
4.5. Model Development 31
4.6. Custom Model Development 31
4.7. Issues during Implementation 33

5. Evaluation 34
5.1. Evaluation Approach and Metrics 34
5.2. Performance Results 34
5.3. Per-Class Analysis 35
5.4. Model Interpretability using Grad-CAM 37
5.5. Discussion and Analysis 38

6. Future Work 39
6.1. Enhancements and Scalability 39
6.2. Broader Applications 39

6. Conclusions 40
6.1. Summary of Achievements 40
6.2. Final Reflection 40

References 41

Bibliography 48

Appendix A: Remote Sensing Dataset Characteristics 50
A.1 BigEarthNet Spectral Bands 50
A.2 BigEarthNet Dataset Analysis and Usage Strategy 51
A.2.1 Class Distribution Across Subsets 51
A.2.2 Label Co-Occurrence Statistics 53
A.2.2.1 Raw Co-Occurrence Matrix 53
A.2.2.2 Normalised and Log-Scaled Co-Occurrence Matrix 54
A.3 Dataset Comparisons and Specifications 55
A.3.1 Comparison of Patch-Level and Pixel-Level Datasets 55
A.3.2 Pixel-Level Dataset Specifications 56

Appendix B: Model Architectures & Hyper-parameters 57
B.1 Custom CNN Architectures 57
B.1.1 Development of Intermediate Models 57
B.1.2 CustomModelV6: High-Performance Hybrid Attention Network 58

B.1.3 CustomModelV9: Optimised Fusion Network 59
B.1.3.1 CustomModelV9 Spectral Mixer 59
B.1.3.2 CustomModelV9 Block 1: Low Level Feature Extraction 60
B.1.3.3 CustomModelV9 Block 2: Mid-Level Abstraction 60
B.1.3.4 CustomModelV9 Block 3: Deep Feature Learning 61
B.1.3.5 CustomModelV9 Block 4: Multi-Scale Feature Fusion 62
B.1.3.6 CustomModelV9 Classification Head 62
B.1.4 Attention Mechanisms in CustomModelV9 65
B.1.4.1 Spectral Attention (SA) 65
B.1.4.2 Coordinate Attention (CA) 66
B.1.4.3 Efficient Channel Attention (ECA) 67
B.1.4.4 Squeeze and Excitation (SE) 68
B.1.4.5 Convolutional Block Attention Module (CBAM) 69
B.1.5 Enhancement Modules in CustomModelV9 70
B.1.5.1 Wide Bottle Neck 70
B.1.5.2 Depth-wise Separable Convolution (DepthwiseSeparableConv) 71
B.1.5.3 Multi-Scale Block 71
B.1.6 Dynamic Fusion and Skip Connections in CustomModelV9 71
B.1.6.1 Stage 1: Deep-Shallow Fusion 72
B.1.6.2 Stage 2: Multi-Stream Fusion 72
B.2 State-of-the-Art Sample Modifications 72
B.2.1 ResNet18 Modification 72
B.2.2 Swin-Transformer Modification 73
B.3 Hyper-Parameters and Training Settings 73
B.3.1 Combined Focal Loss with Positive Weight Implementation 74
B.4 Experiment Configurations 74

Appendix C: Computational Resources & Training Setup 76
C.1 Hardware Specifications and Performance 76
C.2 Python Environment and Dependencies 77
C.3 Code Availability 77

Appendix D: Web-Based Application 79
D.1 Overview Of Application 79
D.2 Application Interface Screenshots 80
D.3 Backend Architecture 85

Appendix E: Evaluation Metrics & Results 86
E.1 Metrics In Detail 86
E.2 V9 Per-Class Performance Highlights 87
E.3 Further Sample Visualisations 88
E.3.1 Additional Grad-CAM visualisation 88
E.3.2 Batch Prediction Images 90
E.3.3 Per-Label Confusion Matrices Grid 91

Appendix F: System Architecture & Project Timeline 92

List of Figures

Figure 2.1. Spectral Reflectance Curves [15] 5
Figure 2.2. Publication Trends On LULC Classification 7
Figure 2.3. Basic CNN Architecture 8
Figure 2.4. Channel And Spatial Attention Mechanisms [79] 10
Figure 3.1. Three-Stage System Classification Pipeline 19
Figure 3.2. High-Level Data Preparation Pipeline 20
Figure 3.3. Training and Optimisation Workflow 21
Figure 3.4. High-level CNN Workflow for BigEarthNet Classification 22
Figure 3.5. High-Level Evaluation Pipeline 23
Figure 4.1. BigEarthNet Data Preparation Pipeline 26
Figure 4.2. Forward & Backward Propagation Pass 27
Figure 4.3. Combined Focal Loss Computation 28
Figure 4.4. Evaluation Pipeline 29
Figure 4.5. CustomModelV9 Block Overview 32
Figure 5.1. 19x19 Per-Class Multi-Label Confusion Matrix 36
Figure 5.2. CustomModelV9 vs ResNet50 Grad-CAM Patch 1 37
Figure 5.3. CustomModelV9 Grad-CAM Patch 2 37
Figure A.1. Sample BigEarthNet Patch Across 12 Spectral Bands 51
Figure A.2. Bar-Chart Label Distribution: 100%,10%,1% Subsets 52
Figure A.3. Raw Co-Occurrence Matrix 54
Figure A.4. Normalised and Log-Scaled Co-Occurrence Matrix 55
Figure A.5. Patch-level (BigEarthNet) vs Pixel-level (LoveDA) image patch 56
Figure B.1. CustomModelV6 High-Level Block Diagram 58
Figure B.2. Spectral Mixer Block Diagram (CustomModelV9) 60
Figure B.3. Block 1 Diagram (CustomModelV9) 60
Figure B.4. Block 2 Diagram (CustomModelV9) 61
Figure B.5. Block 3 Diagram (CustomModelV9) 61
Figure B.6. Block 4 Diagram (CustomModelV9) 62
Figure B.7. Classifier Block Diagram (CustomModelV9) 62
Figure B.8. CustomModelV9 Detailed Architecture Overview 64

Figure B.9. Spectral Attention (SA) Implementation....66
Figure B.10. Coordinate Attention (CA) Implementation....67
Figure B.11. Efficient Channel Attention (CA) Implementation....68
Figure B.12. Squeeze-and-Excitation (SE) Implementation....69
Figure B.13. Convolutional Block Attention Module (CBAM) Implementation....70
Figure D.1. Home Page (index.html)....80
Figure D.2. Train Page (train.html)....81
Figure D.3. Batch Result Page (batch_result.html)....81
Figure D.4. Single Result Page (result.html)....82
Figure D.5. Interactive Map Page (map_page.html)....83
Figure D.6. Inference Comparison Page (detailed_inference.html)....84
Figure E.1. Patch T33UUP_32_52 CustomModelV9 Grad-CAM....88
Figure E.2. Patch T33UUP_32_52 ResNet50 Grad-CAM....88
Figure E.3. Satellite Finland patch CustomModelV9 Grad-CAM....89
Figure E.4. Satellite Tokyo, Japan Patch CustomModelV9 Grad-CAM....89
Figure E.5. Satellite Lago Di Como, Italy Patch CustomModelV9 Grad-CAM....89
Figure E.6. Sample Batch Predictions....90
Figure E.7. Per-Label Confusion Matrices....91
Figure F.1. System-Wide Architectural Flow Diagram....92
Figure F.2. System Execution Interface....94
Figure F.3. End-to-End Training Procedure for RS Image Classification....94
Figure F.4. End-to-End Evaluation Procedure for RS Image Classification....95
Figure F.5. Gantt Chart Project Timeline Overview....95

List of Tables

Table 2.1. Patch-Level Datasets 7
Table 2.2. EuroSAT model architecture benchmark 13
Table 2.3. Sumbul et al. BigEarthNet benchmark [59] 15
Table 2.4. Subset of Papoutsis et al.'s Summary of Results [65] 16
Table 2.5. Vincenzi et al.'s Summary of Results [68] 18
Table 5.1. Performance Comparison of Custom and SOTA Models 35
Table 5.2. Performance of Baseline and Leading Models on BigEarthNet 38
Table A.1. BigEarthNet Multi-Spectral Bands 50
Table A.2. Label Distribution In BigEarthNet Subsets 52
Table A.3. Pixel-Level Datasets Table 56
Table B.1. V1-V9 Major Architectural Changes 57
Table B.2. Performance Metrics From CustomModel V1 to V9 58
Table B.3. General Training Hyper-Parameters 73
Table C.1. Hardware Specifications 76
Table C.2. Subset Performance Metrics 76
Table C.4. Main Project Dependencies 77
Table D.1. Summary of Web Application Features 79
Table E.1. CustomModelV9 Per-Class Classification performance 87

List of Abbreviations

DL	Deep Learning
ML	Machine Learning
CNN	Convolutional Neural Network
LULC	Land Use Land Cover
RS	Remote Sensing
S2	Sentinel-2
MSI	Multi-Spectral Imagery
HSI	Hyper-Spectral Imagery
SOTA	State-Of-The-Art
SVM	Support Vector Machine
RF	Random Forest
NIR	Near-Infrared
SWIR	Short-Wave-Infrared
mAP	Mean Average Precision
GradCAM	Gradient-weighted Class Activation Mapping
VGG	Visual Geometry Group
ResNet	Residual Network
ViT	Vision Transformer
WRN	Wide Residual Network
ECA	Efficient Channel Attention
CBAM	Convolutional Block Attention Module
SE	Squeeze and Excitation
BCE	Binary Cross Entropy

1. Introduction

Recent advancements in remote sensing (RS) technologies have revolutionized how we observe and analyse our planet, resulting in the generation of vast datasets that offer new possibilities for scientific applications [2]. Among these, Land Use Land Cover (LULC) classification plays a crucial role by categorising RS imagery into distinct classes based on land use and surface types, a process vital for critical real-world applications such as environmental monitoring, natural disaster analysis, geospatial detection, and urban planning [3].

However, the sheer volume and complexity of RS data demand efficient and innovative solutions. Traditional machine learning approaches, such as Support Vector Machines (SVM) [4, 5] and Random Forests (RF) [6], often struggle with scalability and effective feature extraction in real-world scenarios. Such limitations have led to the widespread adoption of deep learning (DL) techniques, notably Convolutional Neural Networks (CNNs), which excel at automatic feature extraction and pattern detection on RS imagery [7]. This project builds on these advancements to address the challenges of modern RS data through improved multi-label classification techniques.

1.1. Problem Definition

LULC classification is complicated by the multi-spectral, high-dimensional nature of RS data. Unlike conventional RGB images, RS imagery captures multiple spectral bands, each reflecting different surface characteristics. While this spectral richness provides valuable insights, it also introduces significant dimensionality that complicates classification tasks [8]. Pre-trained models built on RGB datasets, such as those using standard CNN architectures, often fail to utilise the full potential of multi-spectral properties, reducing their effectiveness for RS applications.

Additionally, RS images are prone to geometric distortions and noise caused by factors like sensor orientation or atmospheric interference [9], requiring robust preprocessing pipelines. A further complication arises from the inherent multi-label nature of RS imagery, where patches often contain mixed land cover types (e.g., forest and urban areas). Traditional single-label classifiers are insufficient, as they cannot

capture the overlapping class structures or the relationships across spectral bands and spatial patterns.

Another key challenge in LULC classification is the severe class imbalance, where some land cover categories appear infrequently, leading to biased models that favour majority classes [10]. Moreover, RS datasets are typically large-scale and consist of high-resolution imagery, which imposes considerable computational demands during training and inference [11]. In response to these challenges, this research aims to develop a scalable, automated LULC classification system that can effectively handle multi-spectral inputs, class imbalance, and multi-label complexity using deep learning. The goal is to support practical, real-world applications by minimising manual intervention while maintaining strong performance and computational efficiency.

1.2. Aims and Objectives

The primary aim is to design a system that can robustly predict all relevant land cover classes for each geographic patch in multi-spectral RS data while ensuring optimal accuracy and efficiency. To support this aim, the project defines several specific objectives:

I. Acquire and preprocess a large-scale multi-spectral RS dataset, applying proper transformations, including band selection, normalisation, augmentation, and noise reduction.
II. Design, implement, and benchmark deep learning models, including a custom CNN architecture and transfer learning approaches.
III. Optimize model performance through hyperparameter tuning, advanced loss functions, and adaptive learning strategies to address class imbalance and spectral variability.
IV. Evaluate models using key metrics such as precision, recall, F1/F2 scores, and mean average precision (mAP).
V. Assess the system's generalizability on unseen data and analyse model decisions using visualisation techniques like Grad-CAM [70].

1.3. Dissertation Structure

The dissertation is structured as follows:

- **Chapter 2** presents an in-depth overview of the key methodologies, technologies, and foundational concepts related to remote sensing, multi-label classification, and deep learning. It also includes a literature review of prior studies and datasets.
- **Chapter 3** outlines the system's specification and design, detailing functional and technical requirements, data preprocessing pipelines, CNN architectures, and strategic decisions to address design challenges.
- **Chapter 4** describes the system's implementation, including environment setup, static and dynamic data preparation, training and testing pipelines, and the development of custom and state-of-the-art (SOTA) models.
- **Chapter 5** evaluates the trained models, using aggregated and per-class metrics, supported by visualisation tools. It compares the performance of the custom architecture against benchmark SOTA models.
- **Chapter 6** discusses the potential future directions for the system, including architectural, functional, and deployment enhancements.
- **Chapter 7** concludes the dissertation by summarising key findings, contributions, and the developed system's practical implications.

2. Background & Literature Review

This chapter introduces key methodologies for this project, including Remote Sensing (RS) imagery, Land Use Land Cover (LULC) classification, Convolutional Neural Networks (CNN), and multi-label datasets. These concepts are critical for addressing multi-label classification challenges within LULC. The literature review then analyses machine learning studies on LULC, highlighting their techniques, performance, and limitations.

2.1. Remote Sensing Foundations

2.1.1. Remote Sensing Principles

Remote Sensing (RS) is the science of gathering information about Earth's surface without direct physical contact, using sensors aboard satellites (e.g., Sentinel-2, Landsat [12, 13]) or aircraft [14, 15]. These sensors detect electromagnetic radiation (EMR), which is processed into imagery for analysis [16, 17]. Researchers generally classify RS sensors as either passive or active. Passive sensors, such as Sentinel-2's multi-spectral instruments [12], capture sunlight reflected from the Earth in visible, infrared, and thermal bands. These are especially suitable for LULC classification due to their broad spatial coverage and frequent revisit times. In contrast, active sensors (e.g., SAR, LiDAR) emit their energy, enabling data collection regardless of lighting conditions or cloud cover [18–20]. Despite their advantages, passive sensors tend to dominate LULC studies because their multi-spectral data supports more effective classification of overlapping land types.

This project builds upon these RS principles by using imagery from the BigEarthNet [21] dataset, which researchers constructed using data from Sentinel-2's passive multi-spectral sensors. These sensors capture reflected sunlight across 12 spectral bands, providing rich information suitable for distinguishing diverse land cover types. This data is processed using tools like Python's *Rasterio* library, which enables efficient reading and manipulation of geospatial raster data. This is essential for handling the volume and complexity of satellite imagery in deep learning workflows.

2.1.2. Electromagnetic Spectrum and Reflectance

The Electromagnetic Spectrum (EMS) covers all EMR wavelengths, ranging from radio to gamma waves, each serving distinct purposes [16, 17]. In LULC classification, RS relies on three key bands: visible (0.4-0.7 micrometres), near-infrared (NIR, 0.7-1.3 micrometres), and short-wave infrared (SWIR, 1.3- 3.0 micrometres). The visible band captures surface colours, like green vegetation or blue water. The NIR band is particularly useful for assessing vegetation health, as healthy plants reflect strongly in this region. In contrast, the SWIR band is sensitive to moisture content and can differentiate between dry and wet surfaces [81].

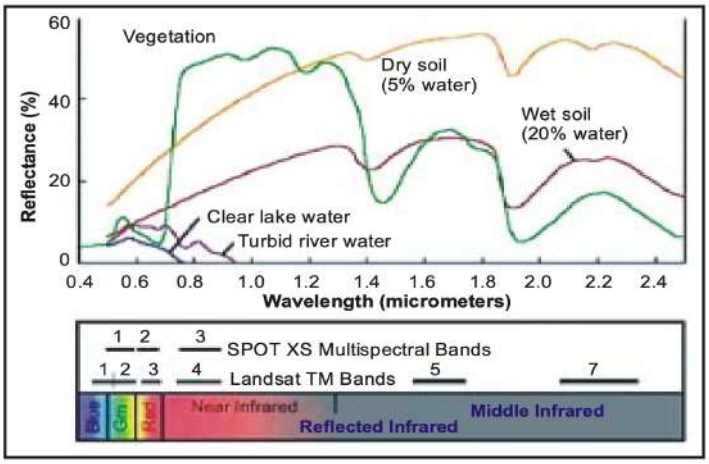


Figure 2.1. Spectral Reflectance Curves [15]

Spectral reflectance is the proportion of incoming radiation that a surface reflects at different wavelengths [82], which varies between materials. As illustrated in Figure 2.1, vegetation shows a high reflectance in the NIR region, while dry soils reflect across the visible and SWIR bands. Water bodies, in contrast, tend to absorb most radiation in the NIR and SWIR regions [15]. Sentinel-2 sensors capture these distinctive patterns, which are included within the BigEarthNet [21] dataset. The availability of such detailed spectral information makes it possible to accurately classify multiple land cover types within a single geographic patch.

2.1.3. Resolution in Remote Sensing

The resolution of RS data, particularly its spatial and spectral resolution, strongly influences its effectiveness and is critical to this project.

I. Spatial resolution refers to the level of detail an image can capture, defined by the size of each pixel. Images with smaller pixel sizes such as one with 10 m resolution offers more detail than an image with 20 m resolution [22].
II. Spectral resolution defines the sensor's ability to differentiate wavelengths. Multi-spectral (MS) sensors capture a limited number of spectral bands (3 to 15), while hyper-spectral (HS) sensors record hundreds of bands, offering greater accuracy but also increased complexity [22].

While other resolution types, such as temporal resolution (revisit frequency) and radiometric resolution (bit-depth sensitivity), also play roles in remote sensing, they are less central to this study. With the fundamental aspects of RS imagery outlined, the focus now shifts to how this data is transformed into structured datasets that enable effective classification.

2.2. Remote Sensing Datasets for LULC

The increasing availability of remote sensing (RS) data has led to the creation of well-labelled benchmark datasets [82], significantly advancing research in Land Use Land Cover (LULC) classification. Many publicly available datasets have emerged in recent years, varying in size, format, and task focus. Broadly, these datasets fall into two main categories: pixel-level and patch-level datasets.

2.2.1. Pixel-Level Datasets

Pixel-level datasets (Table A.3.2) assign a land cover class to each pixel across multiple spectral bands, offering fine-grained detail suited for segmentation tasks. However, they require intensive manual annotation, often using GIS tools such as QGIS, and demand high computational resources. While they can handle multi-label overlaps effectively, their complexity and limited availability make them unsuitable for this project's scalable, CNN-based design, which instead leverages patch-level datasets.

2.2.2. Patch-Level Datasets

Patch-level datasets (Table 2.1) divide RS imagery into localised patches and assign each patch to a scene category rather than per-pixel labels. This structure cuts annotation effort and computational cost, enabling scalable CNN experiments crucial

for multi-label LULC. For instance, BigEarthNet [21], a patch-level dataset utilised within the project, supports overlapping class analysis with its extensive, pre-structured data. The contrast between BigEarthNet's patch-level structure and pixel-level datasets, such as LoveDA [28], is illustrated in Appendix A.3.1.

Table 2.1. Patch-Level Datasets

Name	Image Type	#Images	Image Size	Image Format	Spatial Resolution	#Labels	#Labels per Image	Predefined Splits	Year
UC Merced [29]	RGB	2100	256x256	TIFF	0.3m	21	1	No	2010
RESISC45 [30]	RGB	31500	256x256	JPEG	Varies	45	1	Yes	2017
AID [31]	RGB	10000	600x600	JPEG	0.5-8m	30	1	Yes	2017
EuroSAT [32]	MS	27000	64x64	JPEG/ TIFF	10m	10	1	Yes	2019
BigEarthNet [21]	MS	590236	120x120	TIFF	10m, 20m, 60m	19	Multi-Label	Yes	2020
Optimal-31 [33]	MS	93000	256x256	PNG	2m	31	Multi-Label	Yes	2023
MLRSNet [34]	MS	102500	224x224	JPEG	Varies	17	Multi-Label	Yes	2022

2.3. Land Use and Land Cover Classification

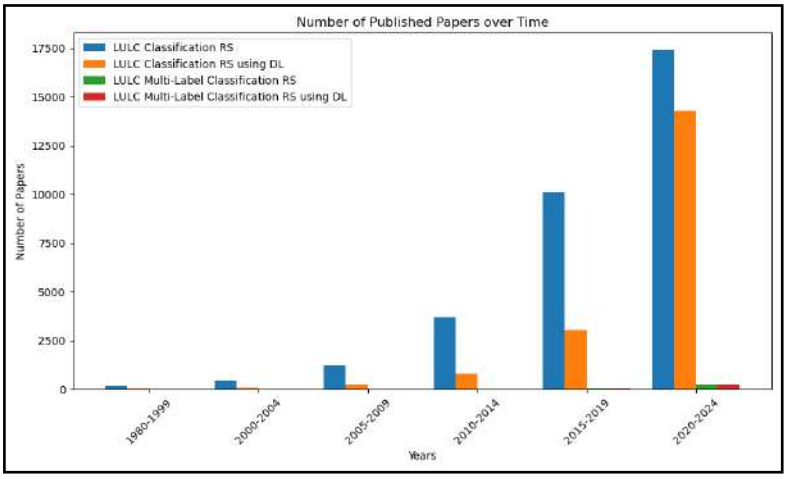


Figure 2.2. Publication Trends On LULC Classification

The Food and Agriculture Organisation (FAO) [3] defines "land cover" as the observable biophysical surfaces of the Earth and "land use" as human activities that shape it. This definition implies a strong interdependency between land cover and land use,

reinforcing that LULC classification must consider both elements together. The study of LULC classification and its challenges has rapidly expanded due to the advancements in Artificial Intelligence (AI), computational power, and high-resolution imagery [35].

Unlike traditional single-label classification, where each pixel or patch is assigned to one category, real-world landscapes blend multiple features, like water bodies, vegetation, and urban areas, within the same patch. As a result, multi-label LULC classification has emerged as the more realistic approach. Figure 2.2, sourced from Google Scholar, tracks the growing number of publications on the subject, highlighting the shift from general LULC classification (blue) to deep learning-based approaches (orange) and from single-label to multi-label classification (green and red). This trend demonstrates the growing importance of more advanced techniques in handling complex, overlapping land classes.

2.4. Deep Learning Techniques

2.4.1. Foundations of Deep Learning and CNNs

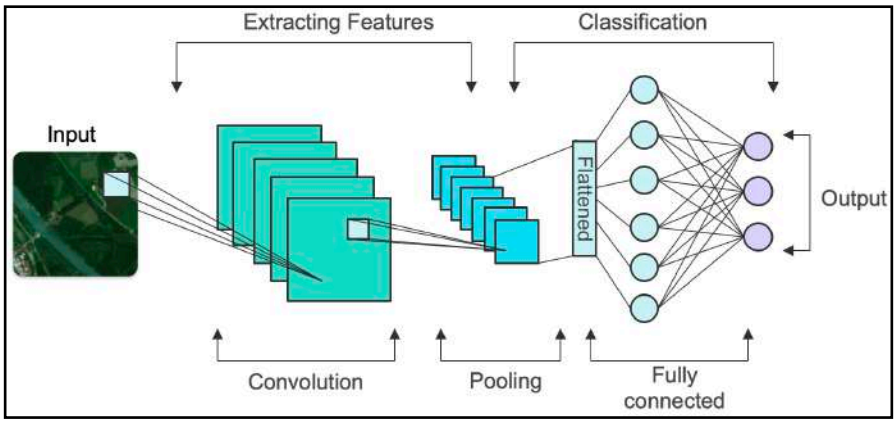


Figure 2.3. Basic CNN Architecture

Deep learning (DL), a subset of Machine Learning (ML), enables the automatic learning of hierarchical features, such as edges and textures, directly from remote sensing (RS) imagery, eliminating the need for manual feature engineering [7]. Convolutional Neural Networks (CNNs), in particular, are well-suited for processing spatially structured data and have proven effective in handling the complexity of multi-label LULC classification

[8]. Given these strengths, CNNs serve as the foundational architecture for this project.

A typical CNN consists of an input layer, a series of hidden layers, and an output layer. The hidden layers primarily include convolutional layers, which apply learnable filters to extract features like edges, textures, and patterns, resulting in feature maps that preserve spatial relationships. Pooling layers then reduce the spatial dimensions of these feature maps through operations like max pooling, lowering computational cost while retaining key information. The extracted features are ultimately passed to fully connected layers, which integrate them to produce the final output predictions. Activation functions like ReLU are applied throughout the network to introduce non-linearity, allowing the model to learn complex relationships between inputs and outputs [83].

2.4.2. Advanced CNN Architectures

Advanced CNN architectures have significantly enhanced image classification by capturing detailed hierarchical patterns from multi-spectral imagery. These capabilities are essential for remote sensing (RS) applications. State-of-the-art (SOTA) models used in RS imagery include ResNet (Residual Networks) [38], which introduces residual connections to support deeper networks and improve gradient flow, VGG (Visual Geometry Group) [39] which utilises uniform layers for consistent learning, and EfficientNet [40] which balances depth, width and resolution for high accuracy with lower computational costs.

Although these models were initially designed for single-label RGB image classification, they required adaptation to suit the multi-label, multi-spectral characteristics of datasets like BigEarthNet [21]. In this project, a custom CNN is developed that integrates ResNet's depth for detailed feature extraction, EfficientNet's efficiency for computational optimization, and advanced techniques like attention mechanisms to address the complex multi-label requirements of Sentinel-2 imagery.

2.4.3. Attention Mechanisms

Attention mechanisms enhance CNNs by enabling them to focus on the most relevant features within complex inputs. Unlike traditional CNNs that process all features

uniformly, attention mechanisms assign dynamic weights to emphasise critical bands or spectral regions, allowing for improved classification accuracy [79]. In LULC classification, where multiple spectral bands and mixed land cover types exist, attention mechanisms are valuable in prioritising key bands (e.g., near-infrared for vegetation) or patch areas (e.g., urban and forest regions) [80] within complex RS images.

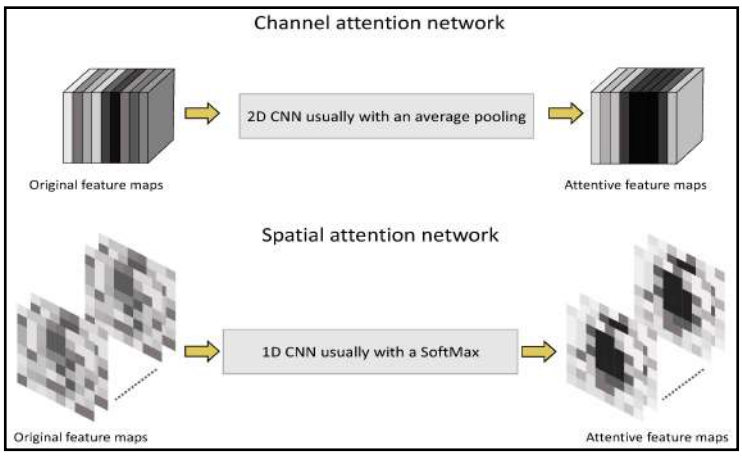


Figure 2.4. Channel And Spatial Attention Mechanisms [79]

Several types of attention mechanisms have been adapted for RS. Figure 2.4 visually illustrates the difference between channel and spatial attention, which play important roles in refining CNN outputs. Channel Attention mechanisms, such as Squeeze-and-Excitation (SE) [77] and Efficient Channel Attention (ECA) [76], recalibrate the relative importance of spectral bands. While Spatial Attention mechanisms, like Coordinate Attention (CA) [74], highlight key patch regions for improved focus in multi-label scenarios. Additionally, Spectral Attention mechanisms capture inter-band relationships, improving feature extraction for multi-spectral data.

2.5. Literature Review

2.5.1. Introduction

This study analyses peer-reviewed studies from IEEE JSTARS, Remote Sensing, and IGARSS to explore the evolution from traditional methods to deep learning, focusing on multi-label LULC classification. The analysis evaluates techniques, results, and research gaps to highlight trends and issues that shape the development of models designed for LULC classification.

2.5.2. Traditional Techniques

Early LULC classification relied heavily on hand-crafted features such as Global Image Scene Texture (GIST) and Scale-Invariant Feature Transformation (SIFT) [41], which were designed to capture basic image elements like edges or textures through manual interpretation by image analysts [8]. Developed in the 2000s, these approaches, alongside classifiers like Support Vector Machines (SVMs), Random Forests, and K-Nearest Neighbours (KNN) [4, 5, 6, 36], achieved around 80% accuracy for simple single-label tasks such as distinguishing vegetation from water. However, their dependence on human analysts introduced significant drawbacks, such as slow processing times, limited adaptability across different datasets, and inconsistent results due to subjectivity [37, 42].

Wolpert's "No Free Lunch" theorem [43] highlights the limitation of traditional approaches, as no single algorithm is universally optimal for all problems. In the context of RS imagery, conventional methods struggled with the complexity and variability of real-world LULC data, as their pre-defined feature sets failed to capture the subtle interactions between overlapping land cover types, making them ineffective for multi-label classification. These issues emphasized the need for more adaptable and data-driven solutions, leading to the shift towards deep learning techniques, as explored in the next section.

2.5.3. Rise of Deep Learning

Over the last decade, deep learning (DL) has significantly outperformed traditional machine learning methods in various RS applications, including semantic segmentation, scene understanding, and geospatial object detection [8]. Unlike traditional approaches, which depend on hand-crafted feature extraction, DL models learn hierarchical representations from raw RS data, making them more adaptable and scalable.

A critical moment in deep learning occurred in 2012 with the development of AlexNet [44], which marked the birth of an era fuelled by CNNs. The success of AlexNet fuelled the development of more advanced architectures such as Network in Network (NiN) [45], VGGNet [39], Inception [46], ResNet [38], EfficientNet [40], and

Vision/Swin Transformers [47, 48]. These architectures introduced key innovations, such as residual learning, multi-scale feature extraction, and self-attention mechanisms, significantly improving classification accuracy across all domains.

Numerous studies [49-55, 56-60, 61, 62, 79] have attempted to apply these architectures to LULC classification, consistently observing performance enhancements over traditional ML methods. Authors in [11] argue that while deep learning has shown promising performance gains in conventional datasets, RS remains inherently complex due to the inherent challenges presented in the data, requiring architectures that adapt to varying spatial resolutions and spectral dependencies.

Despite such inherent challenges, [49] and [50] leveraged the potential of utilising CNNs for LULC classification by actively selecting training samples at each iteration, increasing model performance. Data augmentation techniques such as flipping, cropping, and colour transformations [51, 52] have proven effective in mitigating overfitting and enhancing model robustness [53]. These techniques highlight the importance of optimised training strategies and dataset-specific adaptations when applying deep learning to RS imagery.

2.5.4. Relevant LULC Datasets

The quality and quantity of the dataset used heavily influence LULC classification performance. Historically, the lack of large, well-labelled datasets has limited progress in LULC classification, consequently restricting the model's capacity to generalise across different geographic areas. Recently, the availability of publicly accessible RS datasets has significantly increased, enabling researchers to develop and evaluate deep learning models on larger and more representative data sources.

EuroSAT, introduced by Helber et al. [32], is a widely used publicly available dataset comprising 27,000 Sentinel-2 [61] image patches categorised into 10 different land cover classes across 13 spectral bands. EuroSAT was created to assist in multi-class (single label) classification, providing a structured dataset widely used in RS applications. The work in [32] evaluates several deep learning models restrictively using RGB bands (bands 2, 3, and 4). GoogleNet obtained a 96.69% accuracy, while ResNet-50 reached the highest reported accuracy of 98.57%. Since EuroSAT's

introduction, multiple researchers have explored alternative architectures and techniques to improve the high performance on EuroSAT [32], each refining model efficiency, feature extraction, and spectral band utilisation.

Table 2.2. EuroSAT model architecture benchmark

Authors	Model	Bands	Accuracy
Chen et al. [54]	Knowledge distillation	RGB	94.74%
Chong [55]	Custom CNN	13 Spectral Bands	94.90%
Sonune [56]	VGG19	RGB	98.58%
Helber et al. [32]	ResNet-50	RGB	98.57%
Li et al. [57]	DDRL-AM	RGB	98.74%
Naushadet al. [9]	ResNet-50	RGB	99.17%
Yassine et al. [58]	Custom CNN	13 Spectral Bands + VIRE + NNIR + BR	99.58%

While EuroSAT remains a valuable benchmark for multi-class classification, it was not chosen for this study due to the saturation of classification performance observed by prior studies. The consistently high accuracy is likely due to the clarity of dominant land cover types within the small image patches. The simplified single-label format reduces task complexity and opportunities for further improvements. This study instead focuses on multi-label classification, which presents a more realistic and challenging problem due to overlapping land cover types.

As noted by [21] and [55], one of the most significant drawbacks of RS datasets like EuroSAT [32] is their relatively small sample size, which increases the risk of overfitting unless substantial data augmentation and normalisation techniques are applied. Such an issue restricts the model's ability to generalise across different landscapes, leading to performance degradation. Another concern is that datasets following a multi-class setup assign a single label to each image patch based on the dominant land cover type. However, in real-world scenarios, satellite imagery often contains multiple overlapping land cover types within a single patch, making single-label classification insufficient.

To address these shortcomings, researchers developed BigEarthNet [21] as a large-scale multi-label Sentinel-1 and Sentinel-2 benchmark archive [62, 63], consisting

of 590,326 image patches with 12 spectral bands. While initially designed for 43 land cover classes, classification challenges led to a reduction to 19 categories, which are more balanced and better suited for accurate classification. Its scale and multi-label nature allow models to better capture real-world label overlaps and spectral variations, making it highly suitable for advanced LULC classification tasks.

A variety of other datasets, including UC Merced [29], AID [31], NWPU_RESISC45 [30], Optimal-31 [33], MLRSNet [34], and LoveDa [28], were explored to assess the applicability of different classification techniques. However, BigEarthNet [21] was ultimately selected as the best fit for the project due to I) its multi-label classification setup, II) its diverse land-cover categories, and III) its inherent complexity. These qualities make it an ideal benchmark for developing and evaluating state-of-the-art (SOTA) deep learning models in multi-label RS classification.

2.5.5. LULC Deep Learning Techniques

Sumbul et al. [59] demonstrate BigEarthNet's potential by stacking Sentinel-1 VV and VH bands with Sentinel-2 optical bands, focusing on 10m and 20m spatial resolutions, while excluding 60m bands, as they are primarily used for cloud screening and atmospheric correction. Additionally, patches affected by snow, cloud cover, or missing labels were removed to ensure higher dataset quality. Various state-of-the-art (SOTA) CNN architectures, including ResNet [38] and VGG [39], were evaluated both from scratch and through transfer learning over 100 training epochs. The Adam Optimiser [64] was used with an initial learning rate of 10^-3 and a batch size of 256-500 images.

Experimental results highlighted that models trained directly on BigEarthNet [21] significantly outperformed those using transfer learning on ImageNet. The average F2-score for the 19 classes was a measly 39.81%, while when directly trained on BigEarthNet [21], the average rises to 67.23%, highlighting the potential benefits of dataset-specific training. Additionally, ResNet [38] architectures consistently outperformed their VGG [39] counterparts, suggesting that deeper networks are better suited for capturing the complexity of remote sensing imagery. Table 2.3 summarises the performance of these models across several key metrics, including

Hamming Loss, Overall Error (OE), Recall, and F2-score (Appendix E.1), with ResNet50 achieving the strongest overall performance.

Table 2.3. Sumbul et al. BigEarthNet benchmark [59]

Model	Hamming Loss	One Error (%)	Recall (%)	F2-Score (%)
ResNet50	0.074	5.93	80.05	78.73
ResNet101	0.074	6.46	78.85	77.88
ResNet152	0.073	6.42	78.13	77.46
VGG16	0.078	7.35	76.97	76.18
VGG19	0.080	8.12	76.17	75.35

Within BigEarthNet [21], there is a severe class imbalance, leading to models favouring majority classes while underrepresenting minority ones. Koßmann et al. [10] propose the Multi-label Random Over-Sampling (MROS) strategy, designed explicitly for multi-label problems. Unlike conventional oversampling methods, which risk inflating majority classes, MROS ensures that minority classes receive more attention during training without distorting the overall class distribution. The strategy achieves this by assigning higher importance to underrepresented classes during training while normalising the weights to preserve model stability.

Koßmann et al. [10] introduce the Dynamic Class-Weighted Loss (DCW) to further tackle class imbalance, designed to complement the MROS strategy. DCW dynamically adjusts class weights during training, ensuring the model does not overfit to majority classes and promotes balanced learning across all classes. Koßmann et al. [10] evaluate the MROS and DCW strategy on BigEarthNet [21] on the official split as presented by Sumbul et al. [59] using 269,695, 123,723, and 125,666 image patches for training, testing, and validation. The experiments utilised modified versions of DenseNet121 [69] and ResNet50 [38], adapted to accept all 12 spectral bands and output predictions across 19 land cover categories. Both architectures were initialised with ImageNet pre-trained weights and trained for 200 epochs with a batch size of 256 images, using the Adam optimiser (learning rate 10^-3). Results reveal a significant 20% improvement in predictive accuracy for minority classes, with minimal trade-offs for majority classes. The study also reinforced the value of transfer learning, as ImageNet pre-training contributed notably to overall performance gains.

While BigEarthNet [21] provides a robust benchmark for multi-label LULC classification, its scale introduces considerable computational challenges. Papoutsis et al. [65] highlight that the dataset's large size leads to prolonged training times, making it difficult to fully exploit its capability. To address this, the authors benchmark a range of state-of-the-art (SOTA) models, including CNNs, Multi-Layer Perceptrons (MLPs), and Vision Transformers (ViTs), and propose a training framework based on EfficientNet's [40] compound scaling. This approach optimises model performance by tuning the depth, width, and input resolution to balance classification accuracy and computational costs. Their findings, summarised in Table 2.4, provide a comparative overview of model performance across several key metrics.

Table 2.4. Subset of Papoutsis et al.'s Summary of Results [65]

Model	Accuracy (%)	Precision (%)	Recall (%)	F-Score (%)	Training Time	Inference Rate	Model Size
VGG16	63	81.4	73.6	77.3	0.49	242	14,728,467
VGG19	63.6	81.7	74.2	77.7	0.57	218	20,038,163
ResNet50	62.4	78.8	75.0	76.8	0.59	351	23,648,595
DenseNet121	62.3	79.3	74.5	76.8	0.50	407	7,078,931
WRNB0-ECA	61.7	81.5	71.8	76.3	0.11	942	306,817
VIT/12	61.3	80.7	71.9	76	0.36	668	15,984,915
WRNB4-ECA	/	82.4	75.5	78.8	0.34	/	985,961

Among the evaluated models, WRN-B4-ECA (Wide Residual Network with Efficient Channel Attention) emerged as the best-performing model. The model achieves similar results to Sumbul et al.'s [59] ResNet-50, but with significantly reduced training time, from 59 minutes to 11 minutes. Beyond speed, WRN models are also more lightweight and computationally efficient due to their significantly lower parameter count, making them ideal for handling large-scale datasets like BigEarthNet [21] without sacrificing performance.

Building upon the advances highlighted by Papoutsis et al. [65] and Sumbul et al. [59], authors in [66] introduce a novel deep learning framework that groups Sentinel-2 band properties based on their spatial resolution. This approach allows all spectral bands to contribute to classification tasks, enhancing the model's ability to capture detailed patterns. The authors [66] propose a cross-triplet loss metric to further refine

feature representation, improving inter-class separability and enhancing classification precision. Their custom CNN obtained slightly better performance than the benchmark set by Sumbul et al. [59], showcasing the potential of band grouping and advanced loss functions in improving model performance.

Neumann et al. [60] highlight the importance of domain-specific knowledge in improving the performance of RS models, particularly for LULC classification. Their study evaluates the impact of domain-specific pre-training by experimenting with five well-known RS datasets, including BigEarthNet [21], EuroSAT [32], RESISC-45 [30], So2SAT [67], and UC-Merced [29]. The authors compare three training strategies: I) models trained from scratch, II) models pre-trained on ImageNet, and III) those pre-trained on RS-specific datasets across varying sample sizes (100, 1000, and full datasets). The results demonstrate that models pre-trained with RS-domain data consistently outperform ImageNet-based counterparts, with an increased accuracy range of 11% to 40%. This study highlights the effectiveness of domain-specific pre-training to enhance model generalisation and accuracy.

In contrast, Vincenzi et al. [68] present a novel approach that reduces reliance on labelled data through a self-supervised learning framework built on the ResNet-18 [38] architecture. Instead of requiring large labelled datasets, their method employs a colourisation-based pretext task where the model learns to reconstruct RGB channels using spectral inputs such as infrared and short-wave infrared. This approach enables the network to extract meaningful spatial and spectral features from RS imagery without ground-truth labels, making it useful in limited data scenarios. Once the self-supervised pre-training is complete, the model is fine-tuned on labelled data for downstream tasks such as land cover classification or disease detection.

To further improve performance, Vincenzi et al. [68] implement a dual-branch ensemble model that combines outputs from both the self-supervised model and a traditional ImageNet-pretrained model. This dual strategy combines the strengths of both spectral-based and RGB-based representations. As shown in Table 2.5, this model outperforms the baselines set by Sumbul et al. [59], achieving higher precision and recall. These findings highlight the potential of self-supervised learning and multi-

branch ensemble models in advancing RS classification without heavy dependence on large labelled datasets.

Table 2.5. Vincenzi et al.'s Summary of Results [68]

Method	Precision (%)	Recall (%)	F1-Score (%)
K-Branch CNN	71.6	78.9	72.7
VGG19	79.8	76.7	75.9
ResNet-50	81.3	77.4	77.1
ResNet-101	80.1	76.2	76.5
Ensemble	84.3	78.1	81.1

2.5.6. Key Findings, Gaps, & Conclusion

This review establishes a strong hypothesis for this work, offering key insights into LULC classification's evolution and deep learning's role in its progress. It is evident that DL models consistently outperform traditional ML, particularly in multi-label RS scenarios. Architectures like VGG and ResNet excel in single-label datasets like EuroSAT [32], achieving optimal accuracy. However, their success in single-label contexts does not fully extend to multi-label scenarios, where generalisation remains challenging due to overlapping classes.

BigEarthNet [21] has advanced multi-label LULC classification, motivating strategies like domain-specific pre-training, optimised spectral band usage, deeper CNNs, fine-tuning, and augmentation. Nevertheless, severe class imbalance, computational costs, and inefficient training persist. While dynamic loss weighting and self-supervised learning show promise, their optimisation requires further study. This work tackles these gaps with a custom CNN for BigEarthNet [21], drawing inspiration from Papoutsis et al.'s [65] WRN-B4-ECA. The model uses a parameter-efficient design and integrates attention mechanisms for improved accuracy and efficiency, supported by advanced feature extraction and tailored augmentation.

3. Specification & Design

This chapter outlines the key design decisions behind the deep learning-based multi-label classification system on RS imagery. It covers the functional and technical requirements, dataset selection, CNN architecture choices, and the system's evolution.

3.1. System Overview

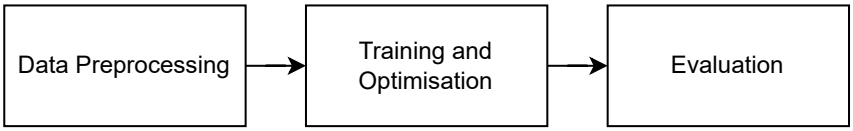


Figure 3.1. Three-Stage System Classification Pipeline

The proposed classification system performs multi-label classification on BigEarthNet's multi-spectral patches to predict overlapping LULC labels. It is structured as a three-stage pipeline (illustrated in Figure 3.1), consisting of I) Data Preprocessing, II) Training and Optimisation, and III) Evaluation stages. This modular structure enables flexible experimentation, allowing adjustments to dataset subsets, model architectures, and evaluation metrics based on specific experimental goals.

A staged pipeline was chosen over an early monolithic design, which risked entangling preprocessing errors with model flaws, which would have complicated debugging and scaling across BigEarthNet's large volume. In contrast, the modular approach isolated tasks for systematic refinement, which proved essential given the complexity of RS data and the iterative nature of deep learning workflows.

3.2. Dataset and Preprocessing

3.2.1. Dataset Selection

The BigEarthNet [21] dataset was selected based on the discussion outlined in Chapter 2. It comprises 590,326 image patches from Sentinel-2, covering 12 spectral bands at varying spatial resolutions (10 m, 20 m, and 60 m), and supports classification across 19 land cover categories (Appendix A.1). Unlike EuroSAT [32], which is limited to single-

label classification and only 27,000 samples, or MLRSNet [34], which lacks full spectral support, BigEarthNet [21] takes full advantage of Sentinel-2's spectral range. The dataset includes critical bands such as Near-Infrared (NIR) and Short-Wave Infrared (SWIR), which are particularly valuable for improving model performance in complex classification scenarios. The dataset's large scale and spectral diversity make it well-suited for deep learning applications, providing a strong foundation for this project's preprocessing and design stages.

3.2.2. Preprocessing Strategy

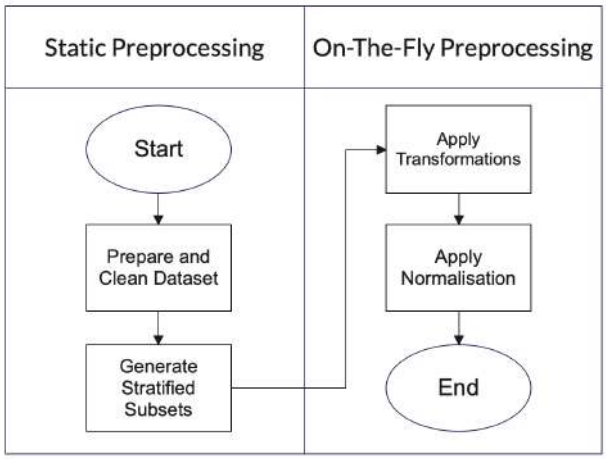


Figure 3.2. High-Level Data Preparation Pipeline

The preprocessing strategy, illustrated in Figure 3.2, prepares BigEarthNet for training by addressing challenges such as noisy data and inconsistent spatial resolutions. It adopts a hybrid approach, combining a static preprocessing phase with on-the-fly augmentations. In the static phase, noisy image patches, such as those containing clouds or snow, are filtered out to ensure data quality. The spectral bands are then resampled to a uniform resolution and reformatted into a consistent input structure suitable for deep-learning models. Stratified subsets of varying sizes are also generated, preserving the original class distribution to enable controlled experimentation (Appendix A.2).

During training, the on-the-fly phase applies real-time augmentations, including rotations, flips, and normalisation. These transformations introduce variability during training, helping to prevent overfitting and improve generalisation. This hybrid design was selected over fully static or fully dynamic alternatives to balance performance and

efficiency. By handling computationally intensive operations before training and applying lightweight augmentations during training, the system remains efficient and adaptable to the demands of multi-spectral RS data.

3.3. Model and Training

3.3.1. Training and Optimisation Strategy

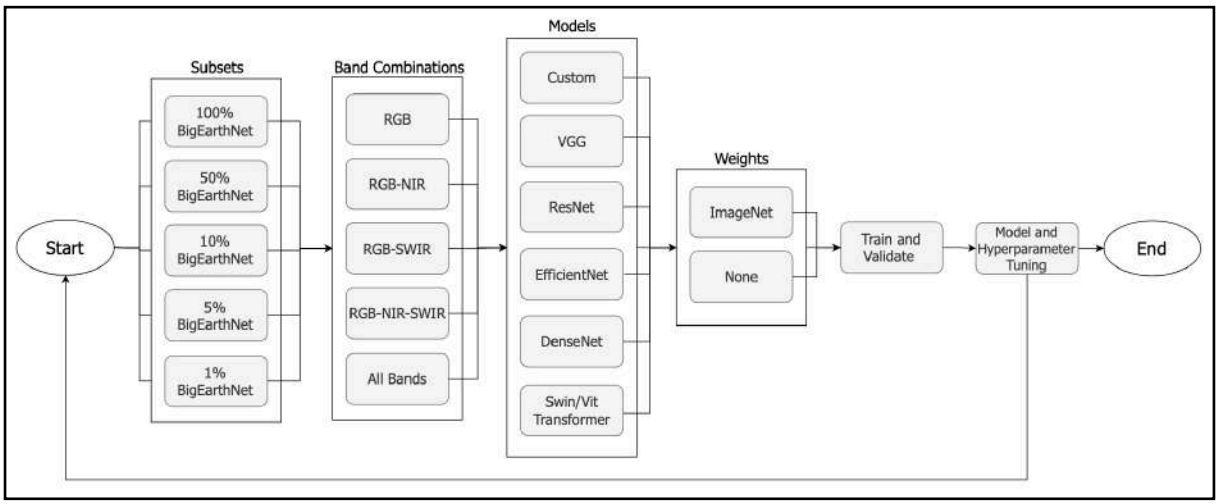


Figure 3.3. Training and Optimisation Workflow

The training pipeline, shown in Figure 3.3, was designed for modular experimentation, allowing flexible configuration of key components such as dataset subsets, spectral band combinations, model architectures, and the inclusion of pre-trained weights. This design ensures adaptability, making it suitable for various experiment setups on the BigEarthNet [21] dataset. The training process followed an iterative approach, continuously refining model performance through systematic adjustments to hyperparameters and architecture configurations.

To address the significant class imbalance in BigEarthNet (Appendix A.2), Combined Focal Loss was selected over standard loss functions such as cross-entropy loss, as it prioritises harder-to-classify and underrepresented image classes. This choice supported better generalisation across the full label distribution. In addition, the training strategy was designed with a strong emphasis on system efficiency, ensuring that performance gains were balanced against the constraints of available hardware and computational resources.

3.3.2. Baseline Models

To inform the design of the Custom CNN, a range of well-established architectures, including CNNs like ResNet [38], VGG [39], DenseNet [69], and EfficientNet [40], and transformer-based models like Vision Transformer [47] and Swin Transformer [48], were chosen. These models were selected for their architectural diversity, offering features like residual connections, dense connectivity, and attention mechanisms. They are also widely adopted in RS multi-label classification tasks, making them highly suitable for benchmarking on BigEarthNet [21].

CNNs were selected for their computational efficiency and proven effectiveness in spatial feature extraction, as outlined in Chapter 2. In contrast, transformer-based models were explored for their potential to capture long-range dependencies across spectral bands. However, they required greater computational resources. Together, these baselines enabled a comprehensive evaluation of design trade-offs, such as the balance between model complexity, performance, and efficiency, and provided a robust foundation for assessing generalisation across BigEarthNet's 19 diverse land cover categories. These ultimately guided the development of a custom model tailored for the specific use of BigEarthNet [21].

3.3.3. Model Architecture

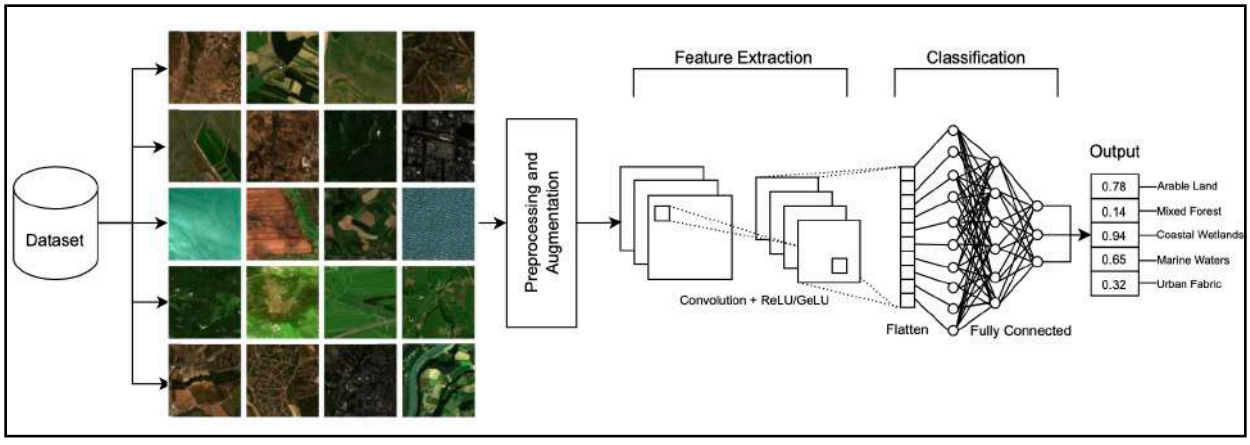


Figure 3.4. High-level CNN Workflow for BigEarthNet Classification

Figure 3.4 presents a high-level workflow of the custom CNN developed for BigEarthNet [21]. The architecture was specifically designed to accommodate the dataset's unique challenges, particularly its 12 spectral bands and 19 land cover

categories. The model emphasises spectral sensitivity, spatial interpretability, and computational efficiency.

The architecture consists of three key stages: I) preprocessing and augmentation, II) feature extraction via convolutional layers, and III) classification through fully connected layers. Lightweight convolutional blocks were used to reduce computational overhead while maintaining the ability to extract spatial features. This choice was made based on the observation that traditional deep CNNs introduce unnecessary complexity without significant gains [71] through standard convolutions. To facilitate deeper feature propagation, residual connections [38] were integrated throughout the network. These connections support spectral diversity across the network by preserving features from earlier layers. This approach directly addresses limitations observed in networks like VGG, where critical information can be lost due to overfitting in deep layers.

Additionally, attention mechanisms [79] were included to enable the model to focus on relevant spectral and spatial regions within the images. Incorporating attention improved the model's ability to distinguish between overlapping land cover regions and to manage the complexity of RS data more effectively. Informed by insights gathered from baseline model evaluations, the custom CNN was iteratively refined to optimise the trade-off between model performance and computational resource usage. Further details on the specific arrangement of the model's architecture layers, dimensions, and mechanisms are provided within Section 4.5.

3.4. Evaluation and Interpretability

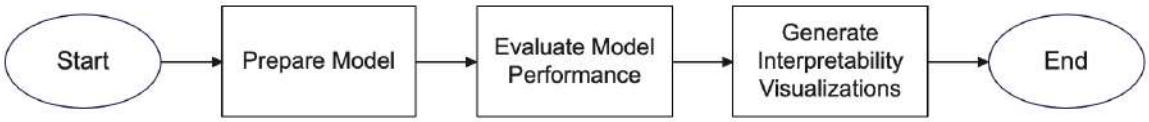


Figure 3.5. High-Level Evaluation Pipeline

The evaluation pipeline, illustrated in Figure 3.5, was designed to assess the effectiveness of custom CNNs on BigEarthNet by integrating quantitative evaluation and interpretability analysis. The pipeline consists of three high-level steps: I)

preparing the model for evaluation, II) evaluating performance using multi-label metrics, and III) generating visualisations for interpretability. Basic performance metrics were initially considered but found insufficient due to BigEarthNet's class imbalance (Appendix A.2), as such metrics tend to overemphasise majority classes and fail to capture performance on underrepresented ones. Instead, the pipeline was designed to integrate metrics that ensure a balanced assessment across all classes and provide a fair evaluation for multi-label classification.

The pipeline integrates visualisation tools designed to enhance interpretability by providing insights into the model's decision-making process. This approach addresses the limitations of purely quantitative evaluations, which may lack the contextual understanding needed for practical deployment, such as identifying which features influence specific predictions. By combining robust metrics with interpretable visualisations, the pipeline supports the model's adaptability and justifies its deployment in real-world applications, where transparency and reliability are critical. Detailed explanations of the metrics and visualisation methods used are provided in Section 4.3.

4. Implementation

This chapter outlines the technical implementation of deep learning models for multi-label classification, detailing the system setup, core pipelines, model development, and challenges encountered.

4.1. System Setup and Environment

The project utilised an isolated Anaconda environment to maintain consistency across development, supporting clean version control and dependency management. PyTorch Lightning served as the core framework, chosen for its modular design, built-in checkpointing, and TensorBoard logging, which streamlined training and evaluation, and improved code maintainability.

Key libraries included Rasterio for processing multi-spectral TIFF files, Pandas for metadata management, NumPy for numerical operations, and Matplotlib for visualising outputs and metrics. Experiments were conducted on a machine with an Intel Core i5-12400F CPU, 32GB RAM, and an RTX 3050 GPU with CUDA support. Although adequate for general tasks, this hardware setup struggled with BigEarthNet's computational demands during preprocessing and training, with bottlenecks addressed in Section 4.7 and detailed further in Appendix C.

4.2. Dataset Preparation Pipeline

This section describes the BigEarthNet [21] dataset preparation pipeline, combining static preprocessing and on-the-fly augmentation, as outlined in Section 3.2.3. As illustrated in Figure 4.1, the static phase began with the dataset being downloaded as a *.tar.zst* archive, then decompressed using *zstandard's* streaming method, and extracted with *tarfile*. A script filtered noisy patches (e.g., those with clouds, snow, or shadows) by cross-referencing provided metadata, which reduced the dataset by 19% (590,326 patches to 480,038 patches).

Each patch's 12 spectral bands were resampled to 120x120 resolution using Rasterio's *nearest neighbour resampling* for standardisation and then stacked into multi-

band TIFFs. Stratified subsets (50%, 10%, 5%, 1%) were created using *MultilabelStratifiedShuffleSplit* from *iterstrat* to preserve class distribution and support scalable experimentation.

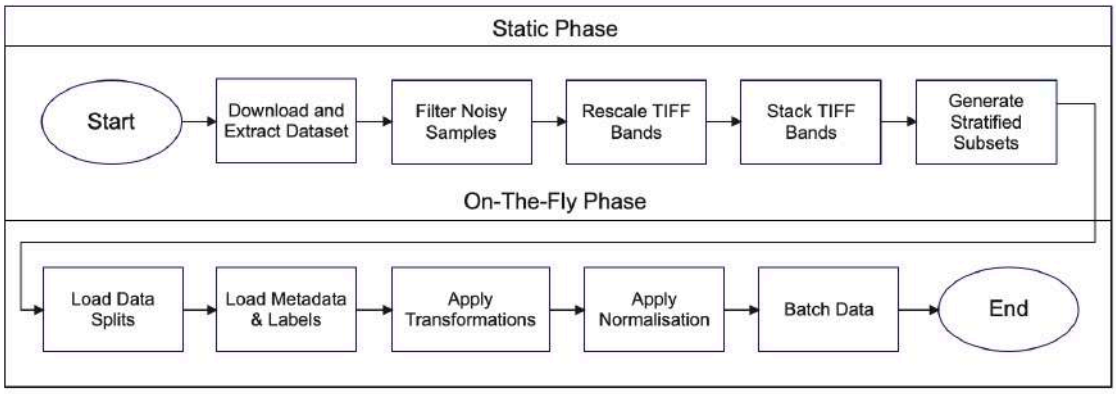


Figure 4.1. BigEarthNet Data Preparation Pipeline

In the on-the-fly phase, a custom dataset class loaded patches, selected specified bands, and converted TIFFs to *float32* tensors. Training augmentations included random horizontal and vertical flips, rotations (0-180 degrees), and erasing (p=0.2). Channel-wise z-score normalisation was applied using precomputed means and standard deviations from the training set:

$$x_{\text{norm},i} = \frac{x_i - \mu_i}{\sigma_i}$$

where x_i is the pixel value and μ_i, σ_i are the mean and standard deviation for each band. Normalisation ensured zero-centred, scaled inputs for improved training stability. Finally, a custom data module splits the dataset into train, validation, and test sets using PyTorch's DataLoader (batch size 256, 8 workers, pin_memory=True) for efficient GPU loading.

4.3. Training and Optimisation Pipeline

Based on the modular design outlined in Section 3.3.2, the training pipeline supports experimentation with various architectures, input band combinations, dataset subsets, and pre-trained weights. Its workflow, illustrated in Figure 4.2, outlines data augmentation, a forward pass to produce logits, loss computation using Combined Focal Loss, prediction thresholding, metric calculation, and back-propagation using AdamW.

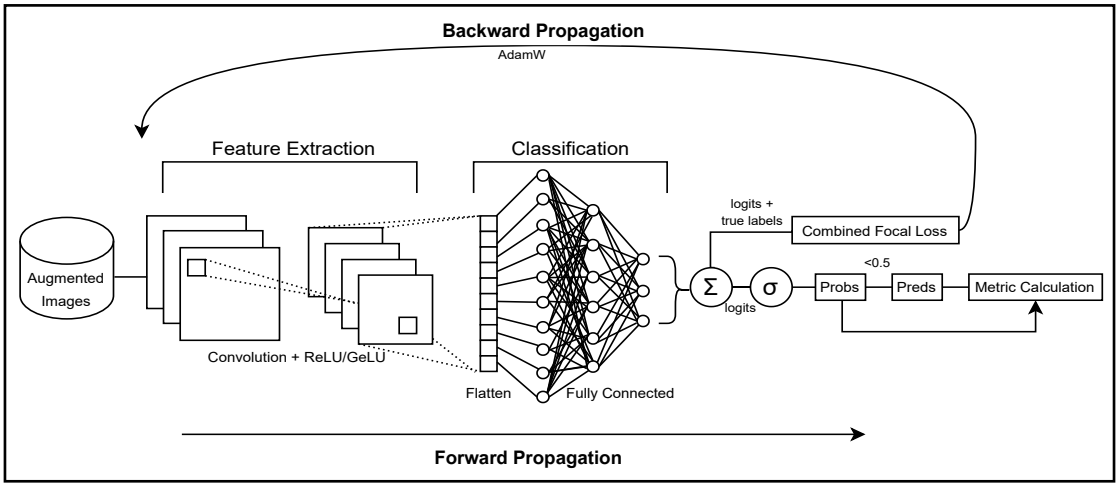


Figure 4.2. Forward & Backward Propagation Pass

4.3.1. Training and Validation Process

Training follows a supervised learning approach, where preprocessed images are passed through the model to produce raw multi-label predictions (logits) using a forward pass. These logits are then processed along two parallel paths, as illustrated in Figure 4.2.

The logits and ground-truth labels are used to compute the loss with Combined Focal Loss. This custom loss function addresses class imbalance. Gradients are then back-propagated using the AdamW optimiser (learning rate = 0.001, weight decay = 0.01), with gradient clipping at 10.0 to prevent instability. Simultaneously, the logits are passed through a sigmoid (σ) activation function to produce probabilities between 0 and 1 for each of the 19 cover classes. These probabilities are thresholded at 0.5, and then converted to binary predictions (preds). Multi-label metrics, including F1/F2 scores, precision, recall, Hamming Loss, and One Error (Appendix E.1), are computed using *torchmetrics*. After completing forward propagation on the training batches, the model is validated on a separate set. Per-class metrics are logged via *TensorBoard*, revealing persistent challenges in minority class performance.

4.3.2. Loss Function Implementation

The *Combined Focal Loss with Pos Weight* function is a critical component of the training pipeline as it attempts to address the class imbalance within BigEarthNet. Its computation process, illustrated in Figure 4.3, combines binary cross-entropy (BCE)

with frequency-based weighting, focal modulation, and alpha balancing. The process begins with logits and ground-truth labels as inputs and proceeds as follows:

I. BCE With Logits is computed using a *pos_weight* tensor derived from inverse label frequencies, penalising errors on rare classes more heavily.

II. The Alpha factor (α = 0.5) is calculated, balancing the importance of positive and negative labels.

III. Probabilities are generated by passing logits through sigmoid (σ) and calculating p_t as the probability of the correct label.

IV. Focal Modulation $(1 - p_t)^{\gamma}$, γ = 3.0, reduces the contribution of easy examples and focuses on difficult, misclassified ones.

V. The final loss is calculated by combining these components element-wise, reducing them to a scalar loss, as given by the equation:

$$\mathscr{L}_{\text{CFocal}} = -\alpha_t(1 - p_t)^{\gamma} w_t \left[y \log(\sigma(x)) + (1 - y)\log(1 - \sigma(x)) \right]$$

This approach indicated a significant improvement in minority class performance, though tuning alpha (α) and gamma (γ) involved trade-offs.

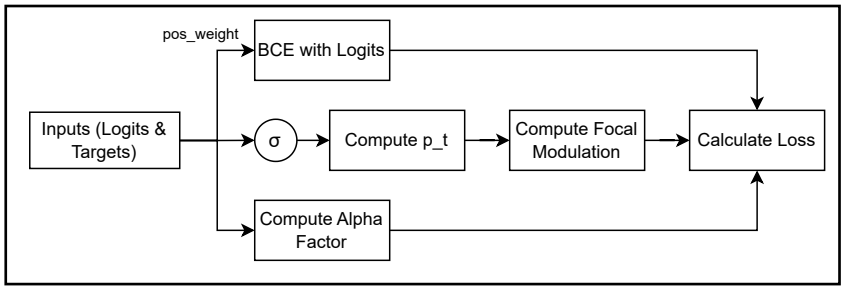


Figure 4.3. Combined Focal Loss Computation

4.3.3. Optimisation Techniques

Several optimisation strategies were integrated to enhance training efficiency and reduce overfitting:

I. *ReduceLROnPlateau* reduced the learning rate by a factor of 0.1 if validation loss plateaued for five epochs, addressing early convergence.

II. Gradient accumulation enabled simulation of larger batch sizes on the RTX 3050 GPU, though tuning was required due to slower convergence with larger batches.

III. Early stopping halted training if validation loss did not improve after ten epochs, conserving resources and preventing overfitting.

IV. Model checkpointing saved models based on best validation performance, enabling robust recovery.

V. Custom callbacks tracked learning rate changes, early stopping events, training time, model size (MB), and inference rate (images/second), all saved with evaluation metrics.

4.3.4. Reproducibility and Code Organisation

Reproducibility was ensured by centralising settings in a *config.py* file, organised through three data classes: *DataConfig* (dataset paths, augmentations, metadata), *ModelConfig* (architecture, bands, hyperparameters), and *ModuleConfig* (custom module settings). Utility functions in the *utils* directory handled core tasks such as evaluation, data handling, visualisation, and overall experiment setup. This structure enhanced code clarity and maintainability. Each training run generated structured outputs including model checkpoints, logs, and visual plots for traceability.

4.4. Testing and Evaluation Pipeline

The testing pipeline assesses model performance on unseen data. It restores model weights using PyTorch Lightning's *load_from_checkpoint* method. The model is set to evaluation mode (*eval()*) to turn off training-specific behaviours like dropout and batch normalisation. The same DataModule used for training is utilised for the test set, ensuring consistency in preprocessing.

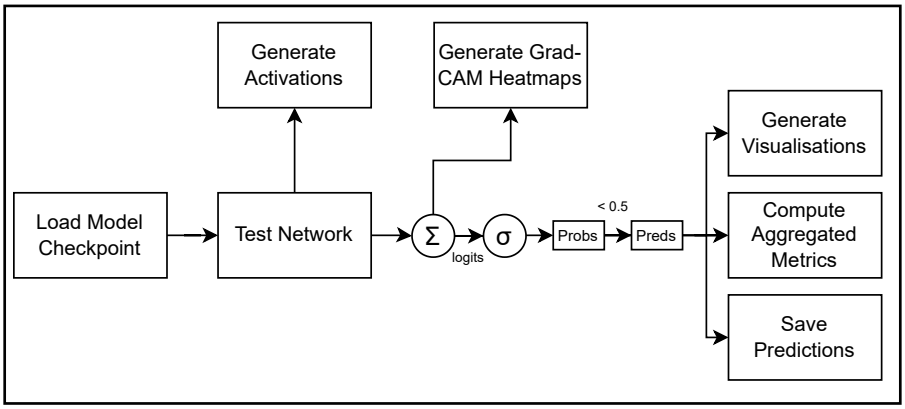


Figure 4.4. Evaluation Pipeline

During inference, the model outputs raw logits, which are then passed through a sigmoid activation (σ) to produce class-wise probabilities. These are thresholded at 0.5 to obtain multi-label predictions. The logits and predictions are logged and stored to compute aggregated performance metrics and to generate visual outputs. During the testing of the network, intermediate activations are extracted using forward hooks, providing insight into the model's internal representation. Grad-CAM [70] heatmaps are also generated to highlight which regions contribute most to each prediction. This process, illustrated in Figure 4.4, shows how each component contributes to the generation of predictions, interpretability tools, and performance metrics.

4.4.1. Performance Metrics and Per-Class Analysis

Performance metrics are computed from the predictions and include F1/F2 scores, precision, recall, hamming loss, and one-error using *torchmetrics* (Appendix E.1). A custom callback tracks and stores the best metric values across test runs, saving results in structured files within the *results/* directory. Per-class metrics are preserved, enabling detailed analysis of model behaviour on minority and majority classes. Predictions and ground-truth labels are saved in .npz format within the *results/* directory for reproducibility and to support future work, such as ensemble methods.

4.4.2. Visualisation and Interpretability

To enhance interpretability, several visual tools are used:
I. **Intermediate activations** offer a window into the network's internal representations, helping to reveal what features are learned.
II. **Grad-CAM** [70] heatmaps show which input regions contribute most to each label prediction, enhancing transparency of the model's decisions.
III. **Per-Label Confusion** and **Co-occurrence matrices** help identify misclassifications and label correlations by visualising the relationships between actual and predicted labels.

Furthermore, the pipeline supports real-time predictions through a lightweight web interface developed using *Flask*. This inference enables users to upload images or select satellite patches from an interactive map and receive predicted and actual labels, along with Grad-CAM heatmaps. Additionally, within the web interface, a bubble chart

visualises trade-offs between the F2-score, training time, and complexity, allowing for direct model comparisons (Appendix D).

4.5. Model Development

SOTA architectures from *torchvision* and *timm*, including ResNet (18,50,101) [38], VGG (16,19) [39], EfficientNet (B0, V2) [40], DenseNet121 [69], Vision Transformer [47], and Swin Transformer [48], were adapted for multi-spectral, multi-label classification, supporting 19 output LULC classes and up to 12 input channels.

For CNNs, the initial convolutional layer was modified to accept the configured number of spectral bands while preserving original parameters (output channels, kernel size, stride, padding, and bias) to maintain architecture consistency. *Kaiming normal initialisation* with *fan_out* and *ReLU* non-linearity was applied to ensure stable convergence. The final classification layer was replaced with a fully connected layer projecting to 19 output classes. For transformer models, *timm's create_model* utility enabled direct configuration of input channels and output classes, with internal adjustments to the patch embedding layer to accommodate 120×120 satellite images.

Each modified model was wrapped in a custom class that inherits from a shared *BaseModel*. This base class handles shared logic such as loss weighting, experiment directory setup, forward pass behaviour, and metric logging. If specified, pre-trained weights were loaded using *torchvision* for CNNs and *timm* for Transformers, enabling experimentation with pre-trained and randomly initialised versions across all architectures.

4.6. Custom Model Development

CustomModelV9 was developed as a lightweight yet effective alternative for multi-label classification on RS data, aimed to address the computational inefficiencies observed in SOTA models. Its development involved multiple prototypes, with nine formally documented as V1-V9. Figure 4.5 provides a high-level overview of the model, which begins with a **Spectral Mixer** block composed of a 1×1 convolution, a 3×3 grouped convolution (groups=4) for inter-band mixing, followed by Spectral Attention [73] and max pooling that reduces the spatial resolution from 120×120 to 60×60.

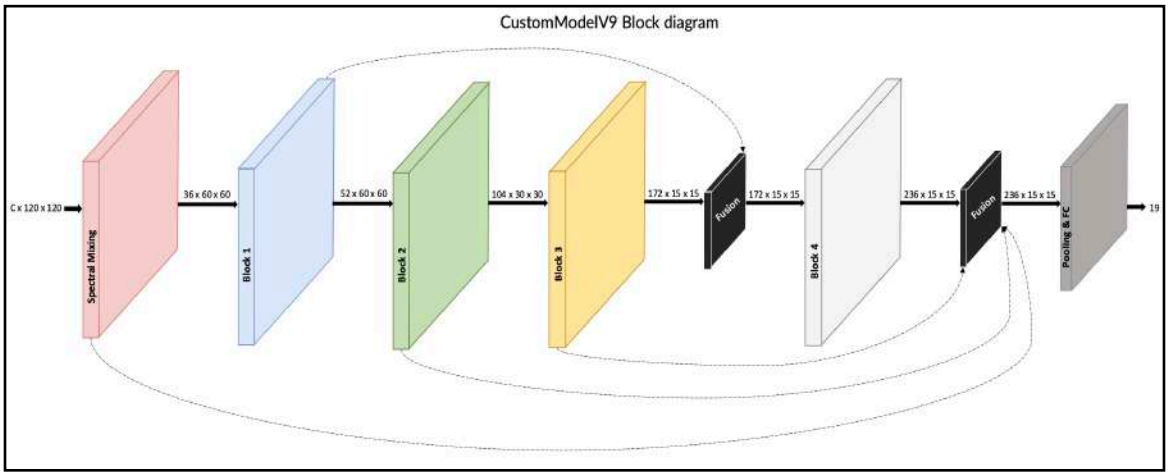


Figure 4.5. CustomModelV9 Block Overview

Feature extraction is organised into four hierarchical blocks:

I. **Block 1** (36->52 channels): Integrates depth-wise separable convolutions [71], a WideBottleneck module inspired by ResNet [38, 72], and Coordinate Attention [74] with a reduction ratio of 16.

II. **Block 2** (52->104 channels): Introduces downsampling and integrates Efficient Channel Attention (ECA) [76] to recalibrate feature channels.

III. **Block 3** (104->172 channels): Further reduces spatial resolution and applies Squeeze-and-Excitation (SE) blocks [77], along with increased dropout (1.5×).

IV. **Block 4** (172->236 channels): Uses a MultiScaleBlock [78] with dilated convolutions at rates 1, 2, and 3, followed by CBAM [75] with a 7×7 spatial kernel.

Dropout regularisation is applied throughout, with progressively increasing rates, peaking at 2× in the classifier block to mitigate overfitting observed in earlier model iterations. Skip adapter connections forward features from the Spectral Mixer and Blocks 1–3 into two fusion stages. Fusion 1 merges early (Block 1) and deep (Block 3) features using attention-weighted summation, while Fusion Stage 2 combines four semantically diverse attention maps from the Spectral Mixer, Blocks 2 and 3, and Block 4 using dynamic soft attention. Before fusion, adapter layers align spatial and channel dimensions to ensure compatibility.

The **Classifier Block** then applies ECA [76] attention, followed by a 1×1 convolution that reduces channels from (236, 15, 15) to (128, 15, 15). Adaptive average pooling collapses the spatial dimensions, and a final fully connected layer produces 19 logits. This configuration results in a total parameter count of 939,604, making the

model significantly more memory-efficient than many benchmark models. Training used the *AdamW* optimiser and a *ReduceLROnPlateau* scheduler, consistent with the earlier described setup. For a detailed breakdown of the network structure, including per-block architectural diagrams and formal definitions of each attention and enhancement module, refer to Appendix B.1.3-B.1.6.

4.7. Issues during Implementation

During implementation, several issues were encountered. Limited hardware (RTX 3050 GPU, i5-12000F CPU) caused delays, out-of-memory errors, and crashes during training, which necessitated working with smaller BigEarthNet subsets. While this strategy sped up iteration cycles, it also introduced generalisability concerns and reduced model performance compared to full-dataset benchmarks.

Class Imbalance posed another challenge, requiring tuning the loss function to ensure fair learning across all 19 LULC classes. Additionally, training instability was a recurring issue, with models suffering from overfitting, underfitting, and gradient-related problems like exploding or vanishing gradients. These were addressed through regularisation techniques such as dropout and gradient clipping. Model convergence was inconsistent. Some architectures plateaued too early or took significantly longer to train, requiring custom learning rate schedules and stopping logic.

Version conflicts across libraries such as PyTorch, NumPy, and Rasterio caused frequent compatibility issues, disrupting the development of the workflow and leading to delays. Handling and preprocessing multi-spectral data introduced bottlenecks in the data pipeline, with the large number of images straining memory and I/O. Moreover, interpretability tools such as Grad-CAM [70] occasionally failed to highlight meaningful regions, limiting their usefulness in explaining predictions. Finally, time constraints and limited computing power restricted the depth of hyperparameter exploration, meaning that some potentially better-performing configurations may have been overlooked. Despite these challenges, iterative debugging, optimisation, and testing ultimately led to a more robust final system.

5. Evaluation

5.1. Evaluation Approach and Metrics

This chapter evaluates the top-performing custom CNNs, V9 and V6, selected from nine formal iterations for their superior performance in multi-label LULC classification against SOTA models. Overlapping class predictions are assessed using quantitative metrics, per-class analysis, and interpretability tools such as Grad-CAM [70], which provide a thorough understanding of model capabilities. Multi-label metrics include Precision, Recall, F1-Score, F2-score, mAP, Hamming Loss, and One Error (Appendix E.1), which quantify performance and align with RS literature standards.

Due to computational constraints (RTX 3050; Appendix C.1), a 10% BigEarthNet subset (47,948 patches) was split into 70% train (33,563 patches), 15% validation (7,192 patches), and 15% test (7,193 patches). This distribution allowed for meaningful evaluation by allocating the majority of the data to training. Models were trained for 50 epochs, with an initial learning rate of 0.001, adjusted by a *ReduceLROnPlateau* scheduler and *early stopping*, with patience values of four and ten epochs, respectively. Consistent preprocessing (Chapter 4) isolates architectural effects and ensures fair comparisons. The evaluation focuses on accuracy-efficiency trade-offs alongside performance disparities across BigEarthNet's 19 categories.

5.2. Performance Results

Table 5.1 compares CustomModelV9, V6, and SOTA architectures on the 10% BigEarthNet test set, evaluating multi-label metrics and efficiency (parameter count and model size). V9 leads with an F2-score of 82.1%, recall of 87.8%, and the lowest One Error (4.7%). With only 939,604 parameters, it outperforms heavier SOTA models like ResNet50, Vision-Transformer, and VGG19 in efficiency and performance. Meanwhile, CustomModelV6 complements V9, excelling in precision (70.2%), F1-score (76.6%), and mAP (90.0%) with the lowest Hamming Loss (8.08%), indicating strong label consistency. However, its higher parameter count (2.05M) increases computational demand. Among SOTA models, DenseNet121 offers solid performance

but lags behind the custom models in key metrics, like Recall and F2-score, with its 7M parameters and 26.7MB size, reducing efficiency. EfficientNetB0 also stands out with an F2-score of 80.5% and moderate efficiency (4M parameters); however, it still lags behind the custom models. Larger architectures like Swin-Transformer (F2: 44.1%, 86.8M parameters) exhibit poor performance, likely due to overfitting on the limited 10% subset.

Table 5.1. Performance Comparison of Custom and SOTA Models

Model	Precision (%)	Recall (%)	F1-Score (%)	F2-Score (%)	mAP (%)	One Error (%)	Hamming Loss (%)	Parameters	Size (MB)
CustomModelV9	65.1	**87.8**	74.8	**82.1**	89.7	**4.7**	9.3	**939,604**	**3.58**
CustomModelV6	**70.2**	84.1	**76.6**	80.9	**90.0**	4.49	**8.08**	2,048,528	7.8
WRN-B4-ECA	62.7	82.4	71.2	77.6	84.6	11.1	10.4	985, 837	3.76
ResNet18	65.7	81.4	72.7	77.7	83.0	12.8	9.6	11,214,483	42.8
ResNet50	64.5	80.6	71.7	76.8	82.4	13.4	9.9	23,575,187	89.9
ResNet101	57.3	80.1	66.8	74.2	81.9	14.5	12.4	42,567,315	162.4
VGG16	59.5	79.8	68.3	72.1	81.5	13.9	12.3	134,343,571	512.5
VGG19	57.8	80.4	67.2	74.6	82.1	13.2	12.8	139,653,267	532.7
EfficientNetB0	66.1	84.0	75.6	80.5	89.5	4.81	8.45	4,034,479	15.4
EfficientNet_V2	54.8	83.3	73.7	79.2	87.6	7.22	9.30	52,884,639	201.7
DenseNet121	68.3	83.2	75.0	70.9	88.8	5.66	8.68	7, 001,555	26.7
ViT-Transformer	54.9	79.2	64.9	72.8	80.0	17.4	13.4	87,469,843	333.7
Swin-Transformer	37.0	46.3	41.1	44.1	54.8	54.9	20.8	86,771,915	331

Lightweight CNNs perform better under computational constraints, as demonstrated by V9's strong recall and streamlined architecture. Its lower precision (65.1%) reflects a focus on rare-class detection, aligning with F2-score's recall bias. V6 offers more balanced metrics, but sacrifices efficiency. Both custom models surpass SOTA in key metrics, confirming their suitability for multi-label classification, with V9 preferred for future optimisation.

5.3. Per-Class Analysis

CustomModelV9's performance across the 19 BigEarthNet classes (Appendix E.2; Table E.1) reflect its design to address class imbalance. It achieves high F2-scores and

recall for frequent classes like “Arable Land” (F2: 0.864, Recall: 0.865) and “Coniferous Forest” (F2: 0.908, Recall: 0.942), aided by *CombinedFocalLoss*. The model also performs well in distinct classes like “Marine Waters” (F2: 0.986, Recall: 0.991), which have clear spectral signatures. However, minority classes like “Coastal Wetlands” (F2: 0.339, Recall: 0.667) and “Beaches, Dunes, Sands” (F2: 0.593, Recall: 0.700) struggle due to subset imbalance, despite weighted loss adjustments. Precision varies widely across classes, ranging from 0.114 for “Coastal Wetlands” to 0.965 for “Marine Waters,” reflecting the model’s recall bias. Despite these challenges, V9 shows versatility, achieving strong results on less common classes like “Agro-Forestry Areas” (F2: 0.850, Recall: 0.955).

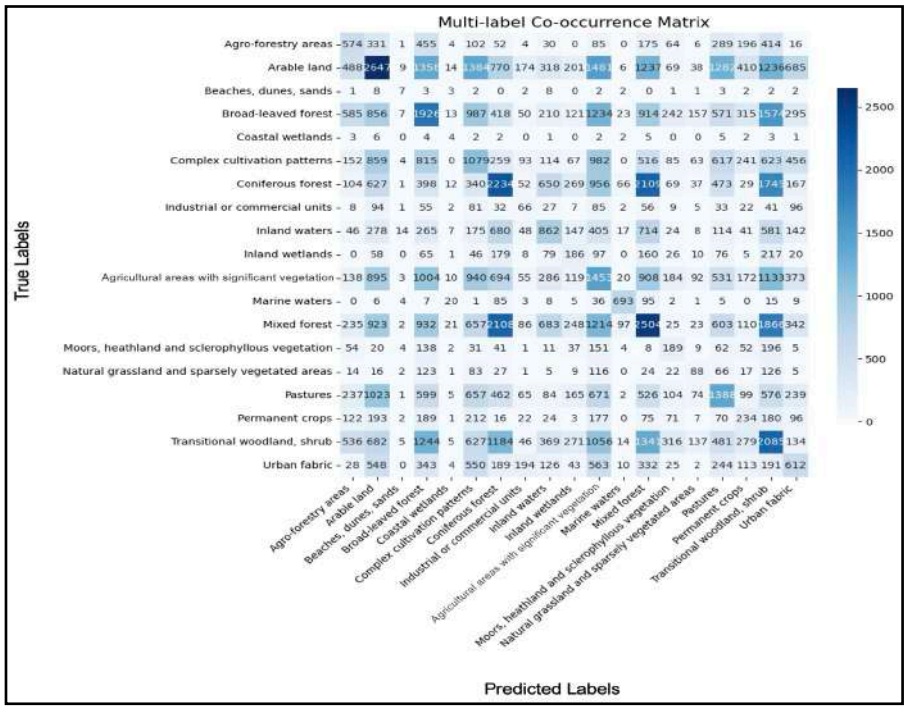


Figure 5.1. 19x19 Per-Class Multi-Label Confusion Matrix

Figure 5.1 supports Appendix E.2-E.3, highlighting high true positive rates for frequent classes like “Arable Land”, “Broad-Leaved Forest”, and “Transitional Woodland, Shrub”. It also shows co-occurrence patterns among semantically related classes (e.g., forest types), indicating the model’s ability to learn inter-class dependencies. Rare classes like “Coastal Wetlands” and “Beaches, Dunes, Sands” show sparse activations due to their scarcity in the 10% subset. Despite this, occasional correct predictions for rare classes, supported by Grad-CAM evidence (Section 5.4), indicate potential for

identification, though performance remains low. The matrix confirms V9's reliability on common classes and inconsistent performance on rare ones due to imbalance.

5.4. Model Interpretability using Grad-CAM

Grad-CAM [70] visualisations demonstrate how CustomModelV9's lightweight design outperforms SOTA models by highlighting focused spectral and spatial activations in satellite patches. Figure 5.2 compares the attention maps of V9 and ResNet50 on a test patch, while Figure 5.3 presents V9's output on a different patch. Further examples in Appendix E.3.1 confirm V9's consistent performance across diverse landscapes.

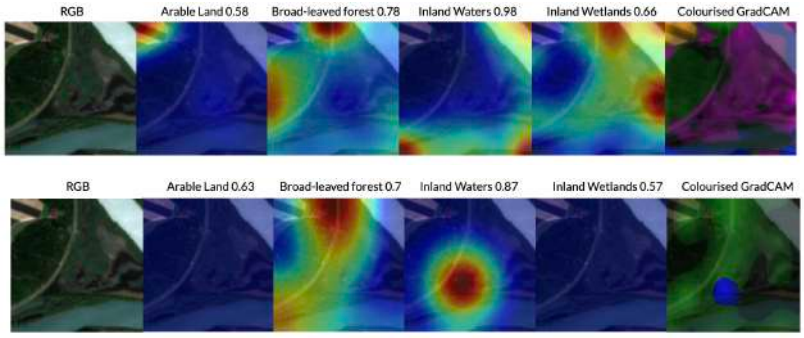


Figure 5.2. CustomModelV9 vs ResNet50 Grad-CAM Patch 1

In Figure 5.2, V9's heatmap (top) shows a concentrated focus on relevant regions associated with classes like "Arable Land" and "Inland Waters", aligning well with ground truth and RGB composites. Colour-coded overlays (pink for wetlands, blue for waters, green for forests, and orange for agriculture) further reinforce this spatial alignment and confirm the model's attention to semantically meaningful areas. In contrast, ResNet's activation (bottom) displays a broader, less focused activation pattern, indicating weaker multi-label sensitivity. This lack of spatial precision aligns with its poorer performance metrics on the 10% dataset subset (see Table 5.1).

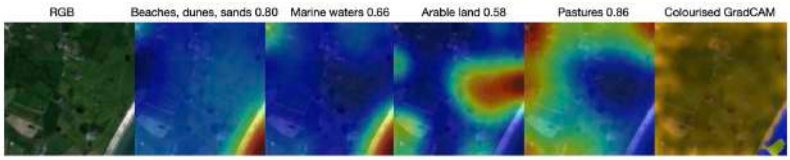


Figure 5.3. CustomModelV9 Grad-CAM Patch 2

Figure 5.3 shows V9's attention map for a second patch, effectively detecting rare classes like "Beaches, Dunes, Sands." The model's attention mechanisms enhance

sensitivity to NIR and Red Edge bands, boosting rare-class recall, as supported by confusion matrices and batch predictions (Appendix E.3). However, some class activations remain spatially diffuse, with borderline probabilities, suggesting room for model and training pipeline tuning, especially for rare or visually similar classes.

5.5. Discussion and Analysis

Table 5.2. Performance of Baseline and Leading Models on BigEarthNet

Authors	Model	Precision(%)	Recall (%)	F1-Score (%)	F2-Score (%)	Parameters
Papoutsis et al. [65]	WRN-B4-ECA	82.4	75.5	78.8	/	985,961
Sumbul et al. [59]	ResNet50	80.05	75.93	/	78.73	23,648,595
This Study	ResNet50	64.5	80.6	71.7	76.8	23,575,187
	WRN-B4-ECA	62.7	82.4	71.2	77.6	985, 837
	CustomModelV9	65.1	87.8	74.8	82.1	939,604

Table 5.2 compares CustomModelV9 to baseline and leading models on BigEarthNet. V9 outperforms this study's ResNet50 (F2: 76.8%, 23,575,187 parameters) and WRN-B4-ECA (F2: 77.6%, 985,837 parameters) across all metrics, despite having only 939,604 parameters. These results highlight the efficiency of V9's lightweight design. Nevertheless, V9's precision lags behind Sumbul et al. [59] and Papoutsis et al. [65], whose models achieve precision above 80%. In contrast, V9's recall (87.8%) exceeds their recall, ranging from 71–76%. These gaps reflect setup differences, as Sumbul used pre-trained weights, the Adam optimiser (0.001 learning rate), and 4 V100 GPUs on the full BigEarthNet dataset, while Papoutsis et al. utilised 10 K-40 GPUs for over 30 epochs with the full dataset. This study's 10% subset (Appendix A.2), single RTX 3050 (Appendix C.1), and 50 epochs prioritise efficiency.

The 10% subset limits rare-class diversity (Appendix A.2.2), causing precision to range between 60–70%, compared to the 75%+ in full-dataset benchmarks. *CombinedFocalLoss's* high gamma boosts recall (87.8%) by focusing on complex labels but trades off precision (65.1%). Scaling to a full dataset with Sumbul's multi-GPU setup could increase recall above 87% and precision past 75%. Additionally, fine-tuning the loss weights and better handling rare classes would position V9 to effectively challenge SOTA models, further showcasing the potential of its lightweight design.

6. Future Work

6.1. Enhancements and Scalability

While the system achieves promising results, several improvements could extend its efficiency and performance. Integrating lightweight transformer-based attention mechanisms, such as mobile variants, could enhance the model's ability to capture long-range spectral dependencies without additional complexity. Advanced loss functions tailored to extreme class imbalance, alongside targeted augmentation for minority classes, could enhance minority class prediction performance. The preprocessing pipeline could benefit from automated cloud and snow detection to improve data quality. Additionally, supporting formats like GeoTIFF and NetCDF would enable in-domain pre-training, as shown in Neumann et al. [60], boosting performance by leveraging large-scale geospatial archives.

From a deployment perspective, the Flask-based web interface demonstrates usability but struggles under high request loads. Transitioning to a faster backend and deploying on scalable cloud platforms like Google Cloud would improve responsiveness and accessibility. Additionally, incorporating an active learning loop, where uncertain predictions are refined through user feedback, could help the model evolve dynamically over time.

6.2. Broader Applications

Beyond LULC classification, the system's techniques could be adapted using multi-temporal data for real-time scenarios such as disaster monitoring, urban change detection, or deforestation tracking. Lightweight deployment on edge devices (e.g., drones) could be achieved through quantisation or pruning, enabling real-time inference under severe resource constraints. Another promising direction is multi-modal data fusion. Combining Sentinel-2 optical data with complementary sources such as Sentinel-1 radar could improve classification under cloud cover and enhance spatial-spectral understanding, allowing the model to generalise better across varied landscapes and environmental conditions.

6. Conclusions

6.1. Summary of Achievements

This project successfully advances multi-label classification on BigEarth's multi-spectral RS data, meeting its key objectives and offering valuable insights. A primary goal was to preprocess a large-scale dataset, which was achieved through BigEarthNet [21] with a pipeline that included band selection, normalisation, and augmentation. Deep learning models, both custom-designed and SOTA, were implemented and benchmarked on a range of multi-label metrics with optimisation efforts involving hyper-parameter tuning, advanced loss functions, and layer modifications. Though minority classes posed persistent challenges, generalisability was validated on unseen data, with Grad-CAM [70] visualisations clarifying model decisions. These findings align with prior research, such as Sumbul et al. [59] and Vincenzi et al. [68], confirming deep learning's superiority over traditional techniques and the importance of large, diverse datasets for robust performance. Spectral band combinations proved influential, supporting calls for an automated selection strategy as proposed by [10], while computational challenges, as noted by Papoutsis et al. [65], underscore scalability limitations.

6.2. Final Reflection

This deep learning project significantly enhanced expertise in remote sensing and multi-label classification, mainly due to the complexity of BigEarthNet. A key outcome was the realisation that data quality and model architecture are equally crucial in achieving desirable results. Although well-optimised SOTA models such as ResNet50 initially outperformed the custom architecture, consistent tuning and experimentation led to competitive performance. The development of a Flask-based training and visualisation interface further highlighted the importance of accessible tools in environments with limited computational resources. Overall, the project contributed to a deeper proficiency in Python, PyTorch, and geospatial AI while reinforcing the relevance of robust, user-friendly systems for satellite imagery applications.

References

[1] S. Mohan, N. Munoth, and N. Sharma, Smart Buildings and Cities with Remote Sensing and GIS. Boca Raton: Chapman and Hall/CRC, 2025. doi: https://doi.org/10.1201/9781003453321.

[2] "Towards a European AI4EO R&I Agenda," 2018. [Online]. Available: https://eo4society.esa.int/wp-content/uploads/2018/09/ai4eo_v1.0.pdf.

[3] Nations, U. Food and Agriculture Organisation. 2019. Available online: http://www.fao.org/faostat.

[4] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, Sept. 1995, doi: 10.1007/BF00994018.

[5] D. A. Pisner and D. M. Schnyer, "Support vector machine," in Machine Learning, A. Mechelli and S. Vieira, Eds. Academic Press, 2020, ch. 6, pp. 101–121. doi: 10.1016/B978-0-12-815739-8.00006-7.

[6] B. de Ville, "Decision trees," Wiley Interdisciplinary Reviews: Computational Statistics, vol. 5, no. 6, pp. 448–455, Oct. 2013, doi: https://doi.org/10.1002/wics.1278.

[7] S. Talukdar et al., "Land-Use Land-Cover Classification by Machine Learning Classifiers for Satellite Observations—A Review," Remote Sensing, vol. 12, no. 7, p. 1135, Apr. 2020, doi: https://doi.org/10.3390/rs12071135.

[8] B. Ekim and E. Sertel, "Deep neural network ensembles for remote sensing land cover and land use classification," International Journal of Digital Earth, pp. 1–14, Sep. 2021, doi: https://doi.org/10.1080/17538947.2021.1980125.

[9] R. Naushad, T. Kaur, and E. Ghaderpour, "Deep transfer learning for land use and land cover classification: A comparative study," Sensors, vol. 21, no. 23, p. 8083, Dec. 2021.

[10] D. Koßmann, T. Wilhelm, and G. A. Fink, "Towards Tackling Multi-Label Imbalances in Remote Sensing Imagery," 2020 25th International Conference on Pattern Recognition (ICPR), Milan, Italy, 2021, pp. 5782-5789, doi: 10.1109/ICPR48806.2021.9412588.

[11] A. Vali, S. Comai, and M. Matteucci, "Deep Learning for Land Use and Land Cover Classification Based on Hyperspectral and Multi-spectral Earth Observation Data: A Review," Remote Sensing, vol. 12, no. 15, p. 2495, Aug. 2020, doi: https://doi.org/10.3390/rs12152495.

[12] "Landsat Science," Landsat Science | A joint NASA/USGS Earth observation program., 2021. https://landsat.gsfc.nasa.gov.

[13] U.S. Geological Survey, "Landsat Missions," USGS, 2023. [Online]. Available: https://www.usgs.gov/landsat-missions.

[14] Schowengerdt, R. A., *Remote Sensing: Models and Methods for Image Processing*, 3rd ed. Amsterdam, Netherlands: Academic Press, 2007. [Online]. Available: https://books.google.com.mt/books?hl=en&lr=&id=KQXNaDH0X-IC&oi=fnd&pg=PP1

[15] J. B. Campbell and R. H. Wynne, *Introduction to Remote Sensing*, 5th ed. New York, NY, USA: Guilford Press, 2011. [Online]. Available: https://books.google.com.mt/books?hl=en&lr=&id=NkLmDjSS8TsC&oi=fnd&pg=PP1

[16] G. Butcher and United States. National Aeronautics And Space Administration, Tour of the electromagnetic spectrum. Washington, Dc: National Aeronautics And Space Administration, 2016.

[17] J. Zwinkels, "Light, electromagnetic spectrum," Encyclopedia of Color Science and Technology, vol. 8071, pp. 1-8, 2015, Springer Berlin, Heidelberg.

[18] P. Kogut, "Types Of Remote Sensing: Technology Changing The World," EOS Data Analytics, Nov. 18, 2020. https://eos.com/blog/types-of-remote-sensing.

[19] "Active and Passive Remote Sensing," Grind GIS. [Online]. Available: https://grindgis.com/remote-sensing/active-and-passive-remote-sensing.

[20] "Remote Sensing - NASA," NASA, Aug. 02, 2023. https://www.nasa.gov/directorates/somd/space-communications-navigation-program/remote-sensing.

[21] G. Sumbul, M. Charfuelan, B. Demir, and V. Markl, "Bigearthnet: A Large-Scale Benchmark Archive for Remote Sensing Image Understanding," IEEE Xplore, Jul. 01, 2019. https://ieeexplore.ieee.org/abstract/document/8900532/

[22] "Remote Sensing," NASA Earthdata. [Online]. Available: https://www.earthdata.nasa.gov/learn/backgrounders/remote-sensing.

[23] M. F. Baumgardner, L. L. Biehl, and D. A. Landgrebe, "220 Band AVIRIS Hyperspectral Image Data Set: June 12, 1992 Indian Pine Test Site 3," doi: https://doi.org/10.4231/R7RX991C.

[24] "Hyperspectral Remote Sensing Scenes - Grupo de Inteligencia Computacional (GIC)," Ehu.eus, 2021. http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes.

[25] X.-Y. Tong, G.-S. Xia, Q. Lu, H. Shen, S. Li, S. You, and L. Zhang, "Land-cover classification with high-resolution remote sensing images using transferable deep models," Remote Sensing of Environment, vol. 237, pp. 111322, 2020

[26] M. J. Hughes and R. Kennedy, "High-Quality Cloud Masking of Landsat 8 Imagery Using Convolutional Neural Networks," Remote Sensing, vol. 11, no. 21, p. 2591, Nov. 2019, doi: https://doi.org/10.3390/rs11212591.

[27] H. Alemohammad and K. Booth, "LandCoverNet: A global benchmark land cover classification training dataset," arXiv preprint arXiv:2012.03111, 2020.

[28] J. Wang, Z. Zheng, A. Ma, X. Lu, and Y. Zhong, "LoveDA: A Remote Sensing Land-Cover Dataset for Domain Adaptive Semantic Segmentation," arXiv (Cornell University), Oct. 2021, doi: https://doi.org/10.48550/arxiv.2110.08733.

[29] Y. Yang and S. Newsam, “Bag-of-visual-words and spatial extensions for land-use classification,” Proceedings of the 18th SIGSPATIAL International Conference on Advances in Geographic Information Systems - GIS ’10, 2010, doi: https://doi.org/10.1145/1869790.1869829.

[30] G. Cheng, J. Han, and X. Lu, “Remote Sensing Image Scene Classification: Benchmark and State of the Art,” Proceedings of the IEEE, vol. 105, no. 10, pp. 1865–1883, Oct. 2017, doi: https://doi.org/10.1109/jproc.2017.2675998.

[31] G.-S. Xia et al., “AID: A Benchmark Data Set for Performance Evaluation of Aerial Scene Classification,” IEEE Transactions on Geoscience and Remote Sensing, vol. 55, no. 7, pp. 3965–3981, Jul. 2017, doi: https://doi.org/10.1109/tgrs.2017.2685945.

[32] P. Helber, B. Bischke, A. Dengel, and D. Borth, “EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification,” IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 12, no. 7, pp. 2217–2226, Jul. 2019, doi: https://doi.org/10.1109/JSTARS.2019.2918242.

[33] Q. Wang, S. Liu, J. Chanussot, and X. Li, "Scene classification with recurrent attention of VHR remote sensing images," IEEE Transactions on Geoscience and Remote Sensing, vol. 57, no. 2, pp. 1155-1167, Feb. 2018, doi: 10.1109/TGRS.2018.2858821.

[34] X. Qi et al., "MLRSNet: A multi-label high spatial resolution remote sensing dataset for semantic scene understanding," ISPRS Journal of Photogrammetry and Remote Sensing, vol. 169, pp. 337-350, Nov. 2020, doi: 10.1016/j.isprsjprs.2020.09.010.

[35] S. Li, X. Kang, L. Fang, J. Hu, and H. Yin, “Pixel-level image fusion: A survey of the state of the art,” *Information Fusion*, vol. 33, pp. 100–112, 2017.

[36] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proc. 5th Berkeley Symp. Math. Statist. Probab., Berkeley, CA, USA: Univ. of California Press, 1967, pp. 281–297.

[37] D.G. Lowe, "Distinctive Image Features from Scale-Invariant Key-points," International Journal of Computer Vision, vol. 60, no. 2, pp. 91-110, Nov. 2004. DOI: 10.1023/B:VISI.0000029664.99615.94.

[38] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” arXiv:1512.03385, 2015. [Online]. Available: https://arxiv.org/abs/1512.03385

[39] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” arXiv:1409.1556, 2015. [Online]. Available: https://arxiv.org/abs/1409.1556

[40] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, PMLR, vol. 97, 2019, pp. 6105–6114. [Online]. Available: https://proceedings.mlr.press/v97/tan19a/tan19a.pdf

[41] A. Oliva and A. Torralba, “Chapter 2 Building the gist of a scene: the role of global image features in recognition,” Progress in Brain Research, pp. 23–36, 2006, doi: https://doi.org/10.1016/s0079-6123(06)55002-2.

[42] S. Zhao, K. Tu, S. Ye, H. Tang, Y. Hu, and C. Xie, “Land Use and Land Cover Classification Meets Deep Learning: A Review,” Sensors, vol. 23, no. 21, pp. 8966–8966, Nov. 2023, doi: https://doi.org/10.3390/s23218966.

[43] D. H. Wolpert and W. G. Macready, “Coevolutionary Free Lunches,” IEEE Transactions on Evolutionary Computation, vol. 9, no. 6, pp. 721–735, Dec. 2005, doi: https://doi.org/10.1109/tevc.2005.856205.

[44] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” Communications of the ACM, vol. 60, no. 6, pp. 84–90, May 2012

[45] M. Lin, Q. Chen, and S. Yan, "Network in network," arXiv preprint arXiv:1312.4400, 2014. [Online]. Available: https://arxiv.org/abs/1312.4400

[46] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” arXiv:1409.4842, 2014. [Online]. Available: https://arxiv.org/abs/1409.4842

[47] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” arXiv:2010.11929, 2021. [Online]. Available: https://arxiv.org/abs/2010.11929

[48] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” arXiv:2103.14030, 2021. [Online]. Available: https://arxiv.org/abs/2103.14030

[49] P. Liu, H. Zhang, and K. B. Eom, “Active Deep Learning for Classification of Hyperspectral Images,” IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 10, no. 2, pp. 712–724, Feb. 2017, doi: https://doi.org/10.1109/jstars.2016.2598859.

[50] Sankaranarayanan Piramanayagam, W. Schwartzkopf, F. W. Koehler, and E. Saber, “Classification of remote sensed images using random forests and deep learning framework,” Proceedings of SPIE, Oct. 2016, doi: https://doi.org/10.1117/12.2243169.

[51] X. Yu, X. Wu, C. Luo, and P. Ren, “Deep learning in remote sensing scene classification: A data augmentation enhanced convolutional neural network framework,” *GIScience Remote Sens.*, vol. 54, pp. 741–758, 2017. doi: 10.1080/15481603.2017.1320591.

[52] V. Lalitha and B. Latha, “A review on remote sensing imagery augmentation using deep learning,” Materials Today: Proceedings, vol. 62, pp. 4772–4778, 2022, doi: https://doi.org/10.1016/j.matpr.2022.03.341.

[53] R. Roelofs, V. Shankar, B. Recht, S. Fridovich-Keil, M. Hardt, J. Miller, and L. Schmidt, "A Meta-Analysis of Overfitting in Machine Learning," in *Advances in

Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alch\'{e}-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, vol. 32. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/ee39e503b6bedf0c98c388b7e8589aca-Paper.pdf.

[54] G. Chen, X. Zhang, X. Tan, Y. Cheng, F. Dai, K. Zhu, Y. Gong, and Q. Wang, "Training small networks for scene classification of remote sensing images via knowledge distillation," *Remote Sens.*, vol. 10, p. 719, 2018.

[55] E. Chong, "EuroSAT land use and land cover classification using deep learning," 2020. [Online]. Available: https://github.com/echong/Remote-Sensing.

[56]: N. Sonune, "Land cover classification with EuroSAT dataset," 2020. [Online]. Available: https://www.kaggle.com/nilesh789/landcover-classification-with-eurosat-dataset.

[57]: J. Li, D. Lin, Y. Wang, G. Xu, Y. Zhang, C. Ding, and Y. Zhou, "Deep discriminative representation learning with attention map for scene classification," *Remote Sens.*, vol. 12, p. 1366, 2020.

[58]: H. Yassine, K. Tout, and M. Jaber, "Improving LULC classification from satellite imagery using deep learning—EuroSAT dataset," Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci., vol. XLIII-B3-2021, pp. 369–376, 2021.

[59] G. Sumbul, A. de Wall, T. Kreuziger, F. Marcelino, H. Costa, P. Benevides, M. Caetano, B. Demir, and V. Markl, "BigEarthNet-MM: A large-scale, multimodal, multilabel benchmark archive for remote sensing image classification and retrieval [Software and Data Sets]," *IEEE Geoscience and Remote Sensing Magazine*, vol. 9, no. 3, pp. 174–180, Sep. 2021. [Online]. Available: http://dx.doi.org/10.1109/MGRS.2021.3089174

[60] M. Neumann, A. S. Pinto, X. Zhai, and N. Houlsby, "Training General Representations for Remote Sensing Using in-Domain Knowledge," in *2020 IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, Waikoloa, HI, USA, 2020, pp. 6730-6733, doi: 10.1109/IGARSS39084.2020.9324501.

[61] D. Phiri, M. Simwanda, S. Salekin, V. R. Nyirenda, Y. Murayama, and M. Ranagalage, "Sentinel-2 Data for Land Cover/Use Mapping: A Review," *Remote Sensing*, vol. 12, no. 14, p. 2291, 2020, doi: 10.3390/rs12142291.

[62] DIMA Group, "Database Systems and Information Management (DIMA)," Technische Universität Berlin, [Online]. Available: https://www.tu.berlin/dima.

[63] Technische Universität Berlin, "Technische Universität Berlin," [Online]. Available: https://www.tu.berlin.

[64] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv preprint arXiv:1412.6980, 2017. [Online]. Available: https://arxiv.org/abs/1412.6980.

[65] I. Papoutsis, N. I. Bountos, A. Zavras, D. Michail, and C. Tryfonopoulos, "Efficient deep learning models for land cover image classification," arXiv preprint arXiv:2111.09451, vol. 5, 2021.

[66] U. Chaudhuri, S. Dey, M. Datcu, B. Banerjee, and A. Bhattacharya, "Interband retrieval and classification using the multilabeled Sentinel-2 BigEarthNet archive," *IEEE J. Sel. Topics Appl. Earth Observations Remote Sens.*, vol. 14, pp. 9884-9898, 2021, doi: 10.1109/JSTARS.2021.3112209.

[67] X. X. Zhu, J. Hu, C. Qiu, Y. Shi, J. Kang, L. Mou, H. Bagheri, M. Häberle, Y. Hua, R. Huang, L. Hughes, H. Li, Y. Sun, G. Zhang, S. Han, M. Schmitt, and Y. Wang, "So2Sat LCZ42: A Benchmark Dataset for Global Local Climate Zones Classification," arXiv preprint arXiv:1912.12171, 2019. [Online]. Available: https://arxiv.org/abs/1912.12171

[68] S. Vincenzi, A. Porrello, P. Buzzega, M. Cipriano, P. Fronte, R. Cuccu, C. Ippoliti, A. Conte, and S. Calderara, "The color out of space: learning self-supervised representations for earth observation imagery," in *2020 25th International Conference on Pattern Recognition (ICPR)*, 2021, pp. 3034–3041.

[69] G. Huang, Z. Liu, and Weinberger, Kilian Q, "Densely Connected Convolutional Networks," *arXiv.org*, 2016. https://arxiv.org/abs/1608.06993

[70] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," *International Journal of Computer Vision*, vol. 128, no. 2, pp. 336–359, Feb. 2020, doi: https://doi.org/10.1007/s11263-019-01228-7.

[71] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions." Available: https://arxiv.org/pdf/1610.02357

[72] S. Zagoruyko and N. Komodakis, "Wide Residual Networks." Available: https://arxiv.org/pdf/1605.07146

[73] L. Dang, L. Weng, W. Dong, S. Li, and Y. Hou, "Spectral-Spatial Attention Transformer with Dense Connection for Hyperspectral Image Classification," Computational Intelligence and Neuroscience, vol. 2022, pp. 1–17, May 2022, doi: https://doi.org/10.1155/2022/7071485

[74] Q. Hou, D. Zhou, and J. Feng, "Coordinate Attention for Efficient Mobile Network Design," *arXiv:2103.02907 [cs]*, Mar. 2021, Available: https://arxiv.org/abs/2103.02907

[75] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," *arXiv:1807.06521 [cs]*, Jul. 2018, Available: https://arxiv.org/abs/1807.06521

[76] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks," *arXiv:1910.03151 [cs]*, Apr. 2020, Available: https://arxiv.org/abs/1910.03151

[77] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-Excitation Networks," arXiv:1709.01507 [cs], May 2019, Available: https://arxiv.org/abs/1709.01507

[78] F. Yu and V. Koltun, "MULTI-SCALE CONTEXT AGGREGATION BY DILATED CONVOLUTIONS." Available: https://arxiv.org/pdf/1511.07122

[79] S. Ghaffarian, J. Valente, M. Voort, and B. Tekinerdogan, "Effect of Attention Mechanism in Deep Learning-Based Remote Sensing Image Processing: A Systematic Literature Review," Remote Sensing, vol. 13, no. 15, p. 2965, 2021, doi: 10.3390/rs13152965.

[80] R. Xu, Y. Tao, Z.-Y. Lu, and Y. Zhong, "Attention-Mechanism-Containing Neural Networks for High-Resolution Remote Sensing Image Classification," Remote Sensing, vol. 10, no. 10, pp. 1602–1602, Oct. 2018, doi: https://doi.org/10.3390/rs10101602.

[81] D. Pouliot, Development of a Remote Sensing Based Coniferous Regeneration Monitoring System, Ph.D. dissertation, Dept. of Geography and Environmental Studies, Carleton Univ., Ottawa, ON, Canada, 2006. [Online]. Available: https://carleton.scholaris.ca/server/api/core/bitstreams bc5ff950-95d0-4c83-9b0f-98ece65a0dbd/content

[82] N. S. Rengma and M. Yadav, "Generation and classification of patch-based land use and land cover dataset in diverse Indian landscapes: a comparative study of machine learning and deep learning models," Environmental Monitoring and Assessment, vol. 196, no. 6, May 2024, doi: https://doi.org/10.1007/s10661-024-12719-7.

[83] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org

Bibliography

Python Software Foundation, “3.7.3 Documentation,” Python.org, 2019. https://docs.python.org/3/

PyTorch, "PyTorch Documentation," 2025. [Online]. Available: https://pytorch.org/docs stable/index.html.

PyTorch Lightning, "PyTorch Lightning Documentation," 2025. [Online]. Available: https://lightning.ai/docs/pytorch/stable/.

Torchvision, "Torchvision Documentation," 2025. [Online]. Available: https://pytorch.org/vision/stable/index.html.

Torchmetrics, "Torchmetrics Documentation," 2025. [Online]. Available: https://torchmetrics.readthedocs.io/en/stable/.

Torchviz, "Torchviz Documentation," 2025. [Online]. Available: https://github.com/szagoruyko/pytorchviz.

TorchCAM, "TorchCAM: Class Activation Maps for PyTorch," 2025. [Online]. Available: https://frgfm.github.io/torch-cam/.

Rasterio, "Rasterio Documentation," 2025. [Online]. Available: https: rasterio.readthedocs.io/en/latest/.

Pandas, "Pandas Documentation," 2025. [Online]. Available: https://pandas.pydata.org/docs/.

NumPy, "NumPy Documentation," 2025. [Online]. Available: https://numpy.org/doc/stable/.

Scikit-learn, "Scikit-learn Documentation," 2025. [Online]. Available: https://scikit-learn.org/stable/.

Matplotlib, "Matplotlib Documentation," 2025. [Online]. Available: https://matplotlib.org/stable/users/index.html.

Seaborn, "Seaborn Documentation," 2025. [Online]. Available: https://seaborn.pydata.org/.

tqdm, "tqdm Documentation," 2025. [Online]. Available: https://tqdm.github.io/docs/tqdm/.

Python Imaging Library (Pillow), "Pillow Documentation," 2025. [Online]. Available: https://pillow.readthedocs.io/en/stable/.

K. Reitz, “Requests: HTTP for Humans™ — Requests 2.27.1 Documentation,” requests.readthedocs.io, 2024. https://requests.readthedocs.io/en/latest/

J. Fu et al., “Dual Attention Network for Scene Segmentation,” 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Jun. 2019, doi: https://doi.org/10.1109/cvpr.2019.00326

A. Vaswani et al., “Attention Is All You Need,” arXiv, Jun. 12, 2017. https://arxiv.org/abs/1706.03762

S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, “Linformer: Self-Attention with Linear Complexity,” arXiv:2006.04768 [cs, stat], Jun. 2020, Available: https://arxiv.org/abs/2006.04768

G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Weinberger, “Deep Networks with Stochastic Depth,” arXiv:1603.09382 [cs], Jul. 2016, Available: https://arxiv.org/abs/1603.09382

M. Tan and Q. V. Le, “MixConv: Mixed Depthwise Convolutional Kernels,” arXiv.org, 2019. https://arxiv.org/abs/1907.09595 (accessed Mar. 06, 2025).

Long Qing Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,” arXiv (Cornell University), Jun. 2016, doi: https://doi.org/10.48550/arxiv.1606.00915

Plotlygraphs, “Bubble Charts,” plotly.com, Jul. 03, 2019. https://plotly.com/python/bubble-charts/

S.-H. by Sinergise, “Sentinel-2 Bands,” Sentinel-Hub custom scripts. https://custom-scripts.sentinel-hub.com/custom-scripts/sentinel-2/bands/

“Using Imagery Layers | ArcGIS API for Python,” Arcgis.com, 2022. https://developers.arcgis.com/python/latest/guide/using-imagery-layers/.

“Quick Start Guide - Leaflet - a JavaScript library for interactive maps,” leafletjs.com. https://leafletjs.com/examples/quick-start/

“Sentinel Hub,” www.sentinel-hub.com. https://www.sentinel-hub.com/

T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” arXiv:1708.02002 [cs], Feb. 2018, Available: https://arxiv.org/abs/1708.02002

E. Ben-Baruch et al., “Asymmetric Loss For Multi-Label Classification,” arXiv.org, 2020. https://arxiv.org/abs/2009.14119

Y. Dai, F. Gieseke, S. Oehmcke, Y. Wu, and K. Barnard, “Attentional Feature Fusion,

arXiv.org, 2020. https://arxiv.org/abs/2009.14082.

T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” arXiv:1612.03144 [cs], Apr. 2017, Available: https: arxiv.org/abs/1612.03144

Appendix A: Remote Sensing Dataset Characteristics

A.1 BigEarthNet Spectral Bands

As introduced in Chapter 2, BigEarthNet [21] uses Sentinel-2 multi-spectral imagery. Each image patch contains 12 spectral bands spanning visible, near-infrared (NIR), and short-wave infrared (SWIR) regions captured at 10m, 20m, or 60m spatial resolutions. Table A.1 outlines these bands, their wavelengths, resolutions, and typical applications in RS analysis. Figure A.1 complements this by visualising a sample patch across all bands, demonstrating the spectral diversity that allows for more accurate LULC classification.

Table A.1. BigEarthNet Multi-Spectral Bands

Band	Name	Wavelength (nm)	Resolution (m)	Image Resolution	Description
B1	Coastal Aerosol	443	60	20x20	Used for aerosol detection and atmospheric correction in coastal waters.
B2	Blue	492	10	120x120	Useful for water body, forest-type mapping, and identifying man-made features.
B3	Green	560	10	120x120	Highlights vegetation and provides clear contrast between different water types.
B4	Red	665	10	120x120	Useful for identifying vegetation types, soils, and urban areas.
B5	Red Edge 1	704	20	60x60	Detects vegetation stress and helps with crop and forest monitoring.
B6	Red Edge 2	740	20	60x60	Improves vegetation and biomass estimation accuracy.
B7	Red Edge 3	782	20	60x60	Enhances vegetation classification and chlorophyll content detection.
B8	Near-Infrared (NIR)	833	10	120x120	Vital for assessing vegetation health, biomass, and water stress.
B8A	Narrow NIR (Red Edge 4)	865	20	60x60	Provides finer distinction in vegetation classification and canopy structure.
B9	Water Vapour	945	60	20x20	Measures atmospheric water vapour and aids in cloud masking.

Band	Name	Wavelength (nm)	Resolution (m)	Image Resolution	Description
B11	SWIR 1	1610	20	60x60	Useful for detecting moisture in vegetation and soils, snow, and minerals
B12	SWIR 2	2190	20	60x60	Helps identify differences between clouds, ice, and built-up surfaces.

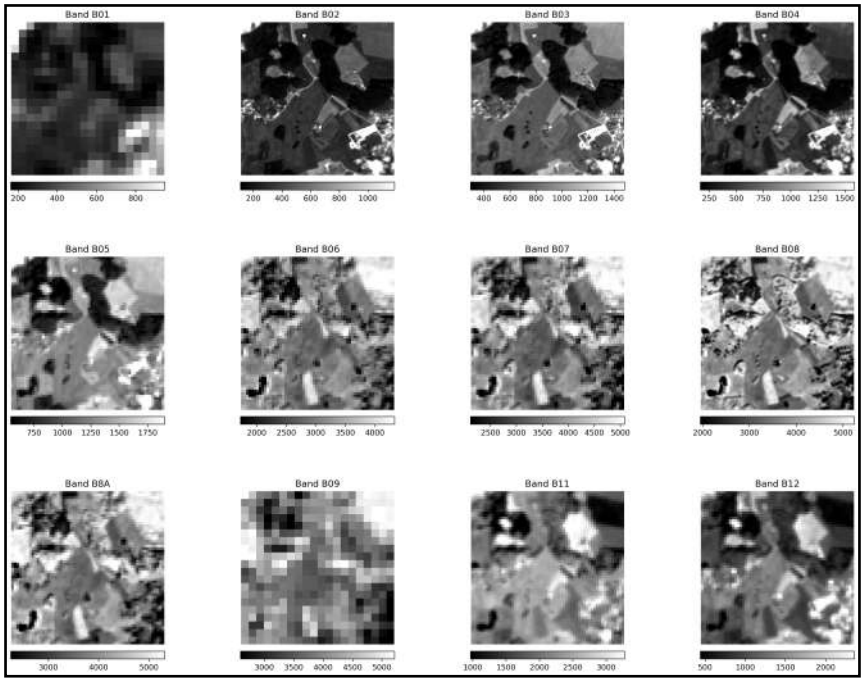


Figure A.1. Sample BigEarthNet Patch Across 12 Spectral Bands

A.2 BigEarthNet Dataset Analysis and Usage Strategy

A.2.1 Class Distribution Across Subsets

To better illustrate the severity of class imbalance across dataset scales, Figure A.2 presents bar charts showing the number of image patches per land cover class for the 1%, 10%, and full BigEarthNet [21] subsets. The distribution clearly reveals a pattern, with a few dominant classes significantly outnumbering rare ones.

I. The most common land types (“Arable Land”, “Mixed Forest”, and “Coniferous Forest”) appear consistently in large quantities across all subsets.

II. The least common land types (“Coastal Wetlands” and “Beaches, dunes, sands”) consistently have the lowest number of patches across the subsets.

III. As the dataset size decreases, particularly in the smaller subsets (1%, 5%), some rare classes are represented by only a handful of samples, further complicating balanced training.

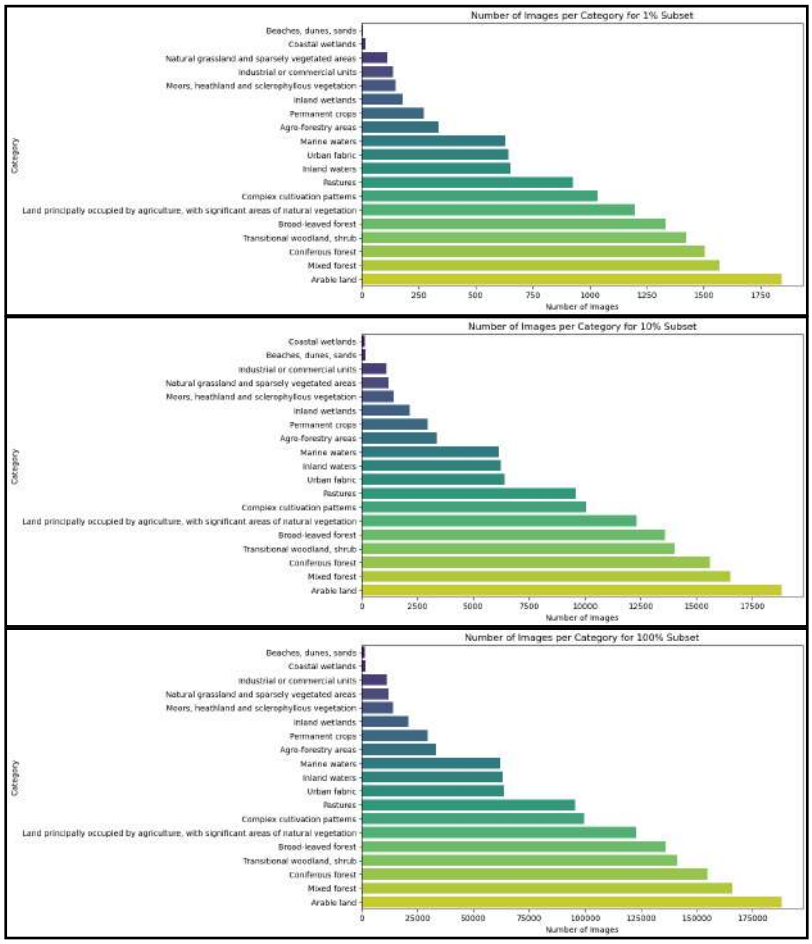


Figure A.2. Bar-Chart Label Distribution: 100%,10%,1% Subsets

These observations informed the use of stratified subset sampling, class-weighted loss functions, and evaluation strategies focused on per-class performance. Table A.2 quantifies the distribution of these three subsets, providing an exact breakdown of sample count per label.

Table A.2. Label Distribution In BigEarthNet Subsets

Category	**100%**	**50%**	**10%**
Beaches, dunes, sands	1,316	641	140
Coastal wetlands	1,397	715	121
Industrial or commercial units	11,142	5,596	1,089

Natural grassland and sparsely vegetated areas	11,882	5,929	1,194
Moors, heathland and sclerophyllous vegetation	13,894	6,866	1,418
Inland wetlands	20,919	10,456	2,139
Permanent crops	29,588	14,781	2,960
Agro-forestry areas	33,181	16,585	3,359
Marine waters	61,832	30,834	6,131
Inland waters	63,212	31,532	6,227
Urban fabric	63,758	31,906	6,392
Pastures	95,605	47,839	9,586
Complex cultivation patterns	99,598	49,718	10,033
Land principally occupied by agriculture, with significant areas of vegetation	122,709	61,076	12,326
Broad-leaved forest	135,928	67,970	13,582
Transitional woodland, shrub	141,150	70,632	14,033
Coniferous forest	154,941	77,551	15,596
Mixed forest	165,780	82,788	16,523
Arable land	188,025	93,958	18,818
Total Images	**480,038**	**239,988**	**47,948**

A.2.2 Label Co-Occurrence Statistics

This section presents two visualisations that show the co-occurrence patterns between the 19 land cover classes within BigEarthNet [21]. These matrices reveal common relationships and dependencies between land cover types.

A.2.2.1 Raw Co-Occurrence Matrix

Figure A.3 displays the raw co-occurrence matrix, where each cell represents the absolute number of times a pair of land cover classes co-occur within the same image. Brighter colours (yellow) represent higher frequencies, while darker shades (purple) denote lower frequencies.

The matrix highlights that dominant classes, such as “Arable Land” and “Mixed Forest”, co-occur with a broad range of other classes, contributing to their high label diversity. In contrast, rarer classes like “Coastal Wetlands” and “Beaches, Dunes,

Sands" show sparse interactions and limited overlap with other land types, further reinforcing their underrepresentation within the dataset.

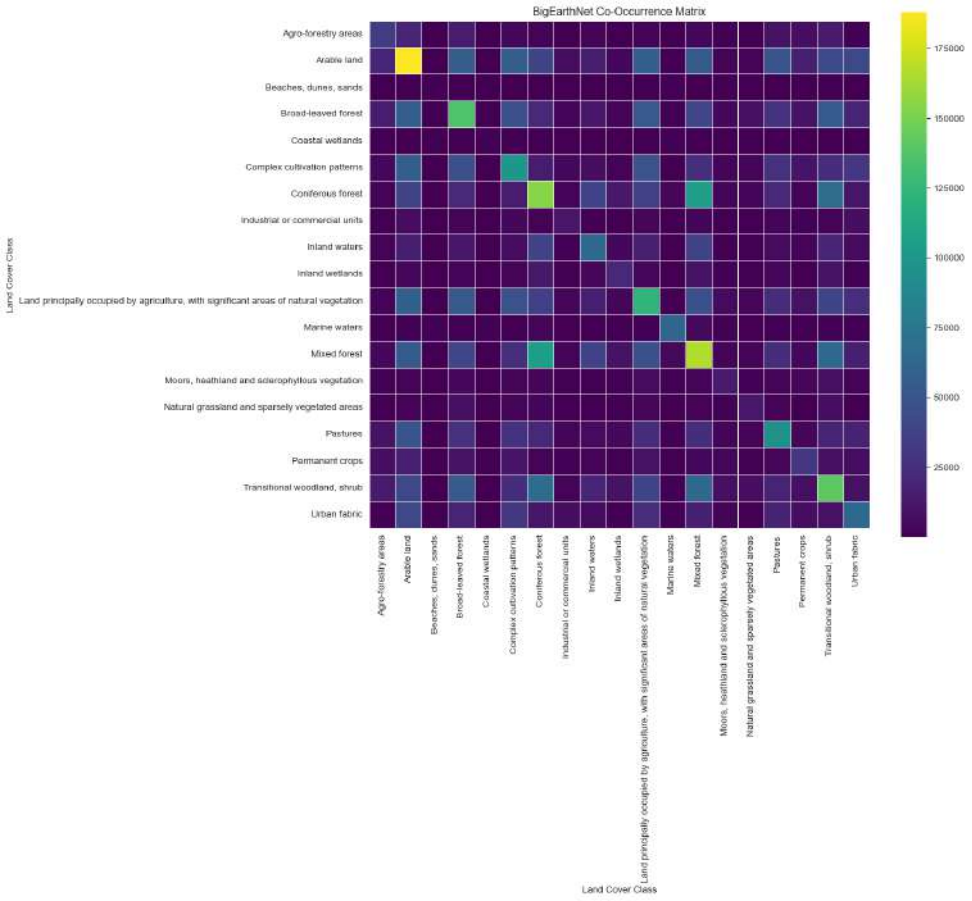


Figure A.3. Raw Co-Occurrence Matrix

A.2.2.2 Normalised and Log-Scaled Co-Occurrence Matrix

To mitigate the issue of dominance of high-frequency classes, Figure A.4 presents a co-occurrence matrix that applies normalisation and logarithmic scaling to the raw co-occurrence counts. This transformation compresses the dynamic range of these values, allowing less frequent but meaningful label pairings to become more visible.

As a result, clearer relationships emerge between land cover types. Notably, "Mixed Forest" and "Coniferous Forest" show strong co-occurrence, which reflects their ecological similarity, and between "Beaches, Dunes, Sands" and "Marine Waters", which suggests real-world geographic and ecological associations. Additionally, class pairs that rarely appear together, such as "Marine Waters" and "Industrial Units," are

distinctly separated, demonstrating the usefulness of the matrix in identifying both strong and weak inter-class dependencies.

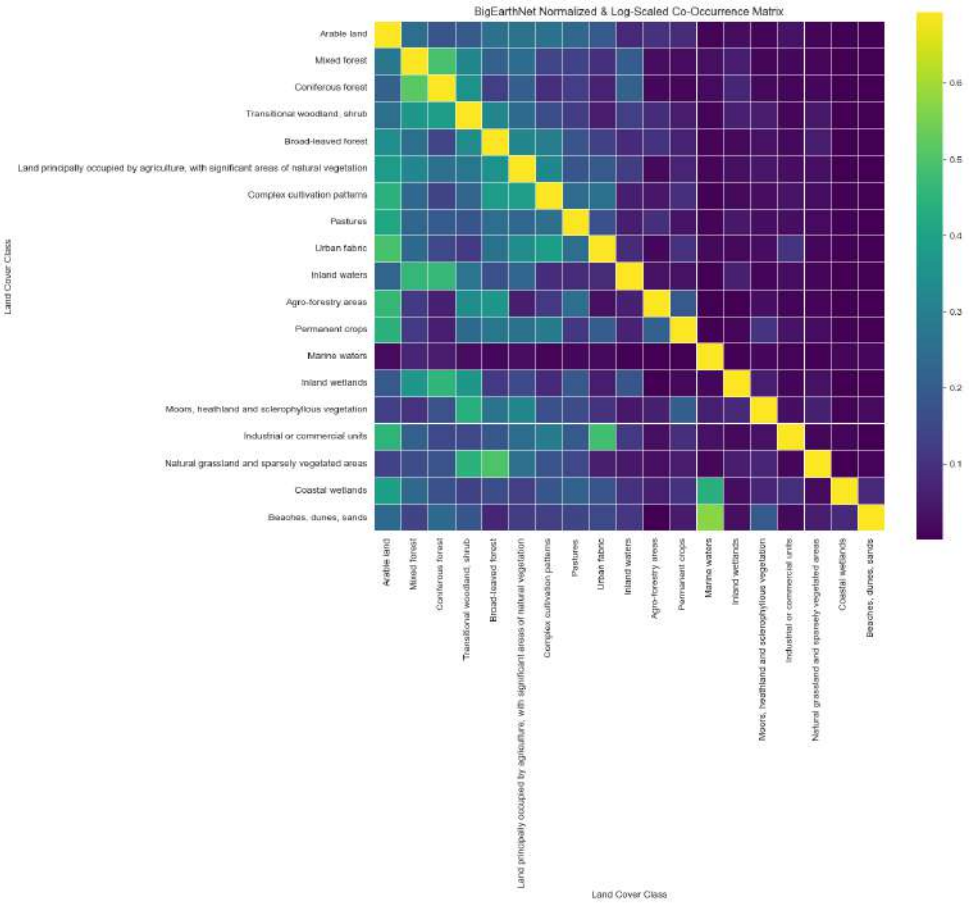


Figure A.4. Normalised and Log-Scaled Co-Occurrence Matrix

A.3 Dataset Comparisons and Specifications

A.3.1 Comparison of Patch-Level and Pixel-Level Datasets

As discussed in Sections 2.2.1 and 2.2.2, RS datasets are categorised into patch-level and pixel-level formats, each suited to different tasks and resource constraints. Patch-level datasets, like BigEarthNet [21], assign labels to entire image patches, enabling efficient multi-label classification over large areas. In contrast, pixel-level datasets, such as LoveDA [28], provide per-pixel annotations, supporting detailed semantic segmentation but requiring significant computational and annotation resources.

Figure A.5 illustrates this contrast; the patch-level sample from BigEarthNet (left) is derived from Sentinel-2 imagery and covers a 1200m x 1200m area across 12

spectral bands at 10m to 60m spatial resolutions. This results in abstract and low-resolution images. On the other hand, the pixel-level image from LoveDA (right), which is captured at 0.3m resolution (1024 x 1024 pixels), reveals sharp boundaries between urban features like roads, buildings, and vegetation. While more informative, this level of detail significantly increases the demand for computational resources and manual labelling effort.



Figure A.5. Patch-level (BigEarthNet) vs Pixel-level (LoveDA) image patch

A.3.2 Pixel-Level Dataset Specifications

Table A.3 lists widely used pixel-level datasets for LULC classification, providing an overview of their characteristics to support the comparison with patch-level datasets like BigEarthNet. These datasets, while powerful for segmentation tasks, were deemed unsuitable for this project's scalable, CNN-based architecture due to their high annotation costs, spatial granularity, and limited multi-spectral coverage. For patch-level dataset specifications, refer to Table 2.1 in Section 2.2.2.

Table A.3. Pixel-Level Datasets Table

Name	Image Type	#Images	Image Size	Image Format	Spatial Resolution	#Labels	#Labels per Image	Predefined Splits	Year
Indian Pines [23]	HS	145	145x145	ENVI	20m	16	1	Yes	1992
Kennedy Center [24]	HS	180	512x614	TIFF	18m	13	1	No	1999
Pavia University [25]	HS	103	610×340	TIFF	1.3m	9	1	Yes	2001
GID [25]	HS	150	6800x7200	TIFF	30m	5	1	Yes	2018
SPARCS [26]	HS	1200	Varies	TIFF	Varies	5	1	Yes	2019
LandCoverNet [27]	HS	19860	256x256	TIFF	10m	7	1	Yes	2021
LoveDA [28]	HS	5987	1024x1024	PNG	0.3m	7	1	Yes	2021

Appendix B: Model Architectures & Hyper-parameters

B.1 Custom CNN Architectures

B.1.1 Development of Intermediate Models

Models V1-V9 represent key steps in an iterative design process involving numerous previous experiments, refining architectures for rare class detection and efficiency. Table B.1 summarises each model's major architectural changes and parameter counts, showing the progression toward V9's lightweight design.

Table B.1. V1-V9 Major Architectural Changes

Model	Major Architectural Changes	Parameters
V1	Sequential 4-block structure with standard Conv2d, BatchNorm2d, ResidualBlocks, and multiple attention modules (Spectral, ECA, SE, Dual)	1,975,770
V2	Added spectral mixing (1×1 Conv), DepthwiseSeparableConv, MultiScaleBlock, and CoordinateAttention	1,804,410
V3	Skip connection (Block 1->4), Transformer module for global context, increased channel widths	8,517,006
V4	Bottlenecks and dilated convolutions for larger receptive fields, expanded Block 4	12,650,898
V5	WideBottleneck blocks for efficiency, CBAM for refined attention, dropout for regularisation	1,716,240
V6	MultiScale block in Block 4 for diverse feature scales, single skip connection (Block 1->3)	2,048,528
V7	Reduced channel counts, multiple skip connections (Blocks 1->3, 2->4, 3->4) with learned fusion weights	950,761
V8	Spectral skip connection from the spectral mixer, dual fusion modules (Fusion 1: Block 3+Block 1, Fusion 2: Block 4+Blocks 2,3)	962,328
V9	Grouped convolutions in spectral mixer and Block 4, normalised fusion weights, BatchNorm2d in skip connections for stability	939,604

Table B.2 provides a comparison of evaluation metrics for the selected iterations. Each model was trained and validated under consistent experimental settings using the same dataset splits and evaluation pipeline. The table highlights model performance and efficiency evolution throughout the development process.

While models like V5 and V6 achieve strong overall scores, V9 ultimately delivered the highest Recall (87.8%) and F2-score (82.1%), indicating its strength in detecting rare or underrepresented classes. Combined with its low parameter count (939,604) and model size (3.58MB), the model was selected as the final model due to its striking balance of performance and efficiency.

Table B.2. Performance Metrics From CustomModel V1 to V9

Model Version	Precision (%)	Recall (%)	F1-Score (%)	F2-Score (%)	mAP (%)	One Error (%)	Hamming Loss (%)	Model Size (MB)
V1	67.6	84.6	75.2	80.5	89.0	5.66	8.77	7.54
V3	66.9	84.6	74.7	80.3	89.1	5.46	8.99	32.49
V5	69.8	84.3	76.3	80.9	89.9	4.95	8.20	6.55
V6	70.2	84.2	76.6	80.9	90.0	4.50	8.09	7.82
V7	68.38	83.9	75.3	80.23	89.5	4.78	8.62	3.62
V8	63.9	87.6	74.0	81.8	89.3	5.45	9.68	3.67
V9	65.1	87.8	74.8	82.1	89.7	4.68	9.29	3.58

B.1.2 CustomModelV6: High-Performance Hybrid Attention Network

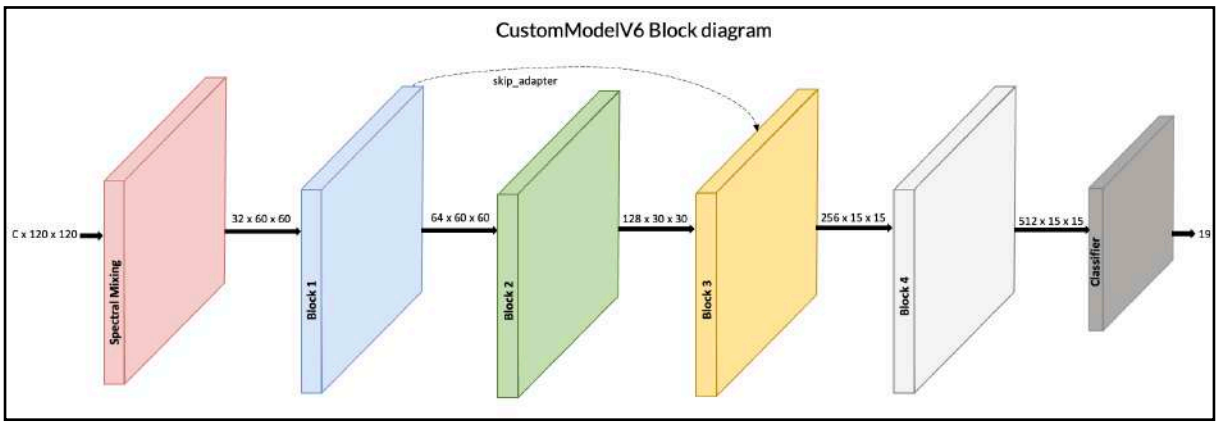


Figure B.1. CustomModelV6 High-Level Block Diagram

As discussed in Chapter 5, CustomModelV6 delivered the best balance of performance across custom variants with the highest precision (70.2%), F1-Score (76.6%), mAP (90%), and lowest hamming loss of (8.09%). Its architecture balanced depth and attention mechanisms in a way that enhanced both precision and recall without overfitting. Its high performance arises from the inclusion of the MultiScaleBlock

within block 4, placed directly after the skip connection. This block is responsible for capturing diverse feature scales using parallel convolutions, enhancing performance.

CustomModelV6 utilises a single skip connection (Block1->Block3), which preserves low-level spatial details and offers stability compared to the more complex multi-skip fusion in models V7-V9. Finally, its WideBottleNecks with a widen_factor of 4, presented a larger widen_factor compared to other models, allowing for enriched feature extraction. This model balances regularisation as it has higher dropout rates than early models but lower than V7-V9. This combination of targeted attention, simple skip connection, efficient blocks, and balanced regularisation helped to outperform simpler (V1–V2), heavier (V3–V4), or overly complex (V7–V9) models. However, its high parameter count compared to later models led to V9's development for better efficiency, as discussed in Section B.1.3.

B.1.3 CustomModelV9: Optimised Fusion Network

CustomModelV9 represents the final and most refined architecture developed in the study. It was designed to maximise rare-class recall while significantly reducing computational cost. Building on insights from earlier iterations, V9 adopts a modular, hierarchical structure that integrates grouped convolutions, spectral and spatial attention, and dynamic fusion with skip connections.

The architecture progresses through a series of carefully designed blocks, starting with the **Spectral Mixer**, followed by four increasingly abstract **feature extraction blocks**, and concluding with an efficient **classification head**. Each block performs distinct operations and attention mechanisms that target spectral redundancy, spatial relationships, and multi-scale fusion.

B.1.3.1 CustomModelV9 Spectral Mixer

The Spectral Mixer acts as the model's initial processing unit, transforming the input tensor **(c, 120, 120)** to **(36, 60, 60)**. It begins with a 1x1 convolution for early channel compression, followed by grouped convolutions to efficiently capture interband relationships. A Spectral Attention module further enhances these channels before the output is downsampled using MaxPool2d. This block performs dimensionality reduction while preserving spectral richness.

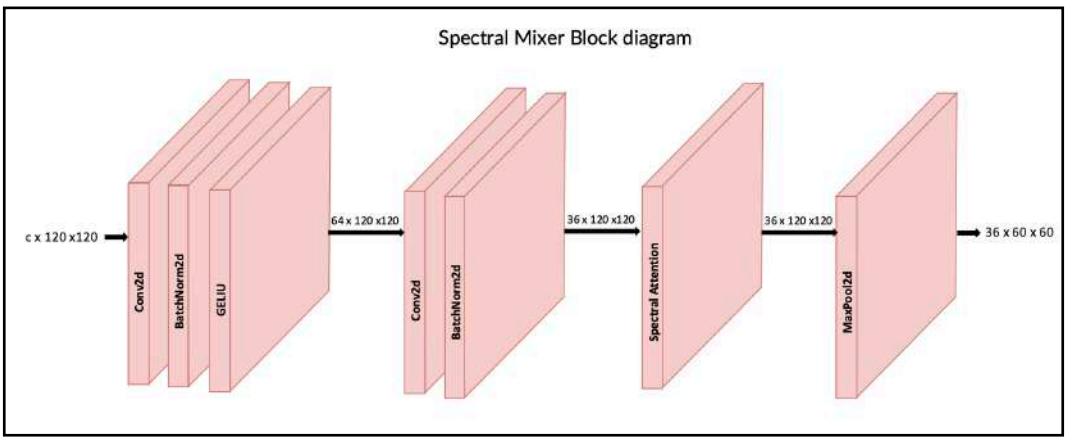


Figure B.2. Spectral Mixer Block Diagram (CustomModelV9)

B.1.3.2 CustomModelV9 Block 1: Low Level Feature Extraction

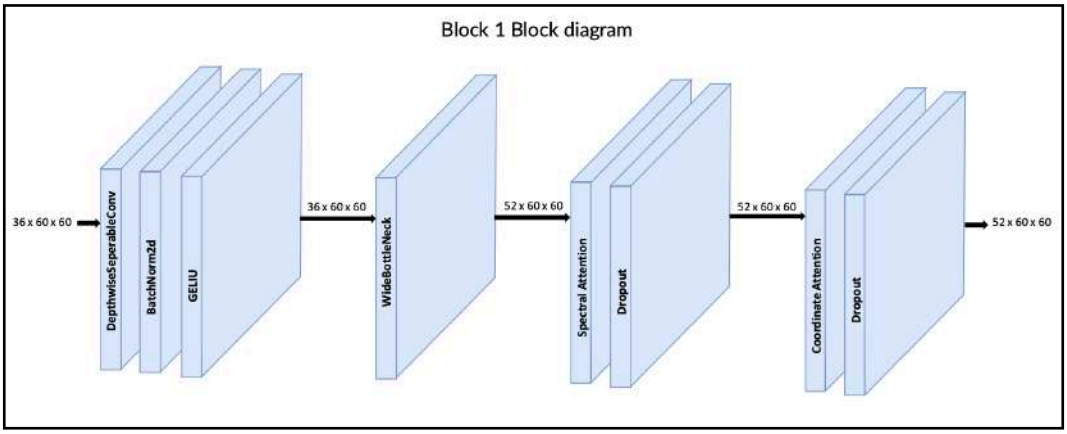


Figure B.3. Block 1 Diagram (CustomModelV9)

Block 1 receives the output of the Spectral Mixer and maintains spatial resolution at **(60, 60)** while expanding depth from **36** to **52** channels. It combines Depthwise Seperable Convolution (3x3) for efficient spatial filtering with a WideBottleneck (widen_factor=2) for enhanced feature capacity. This block also integrated Spectral and Coordinate Attention for channel recalibration and axis-aware context encoding. Dropout (set from *ModuleConfig*) is applied to mitigate overfitting. Skip connections from this block are preserved for later fusion.

B.1.3.3 CustomModelV9 Block 2: Mid-Level Abstraction

Block 2 downsamples spatially from **(60, 60)** to **(30, 30)** and increases the feature depth from **52** to **104**. This block mirrors the structure of Block 1, but replaces Spectral Attention and Coordinate Attention with an Efficient Channel Attention (ECA) module

to recalibrate channel importance with minimal overhead. Block 2 connects low-level features with deeper semantic representations, and its output is routed via skip connections to the final fusion layer.

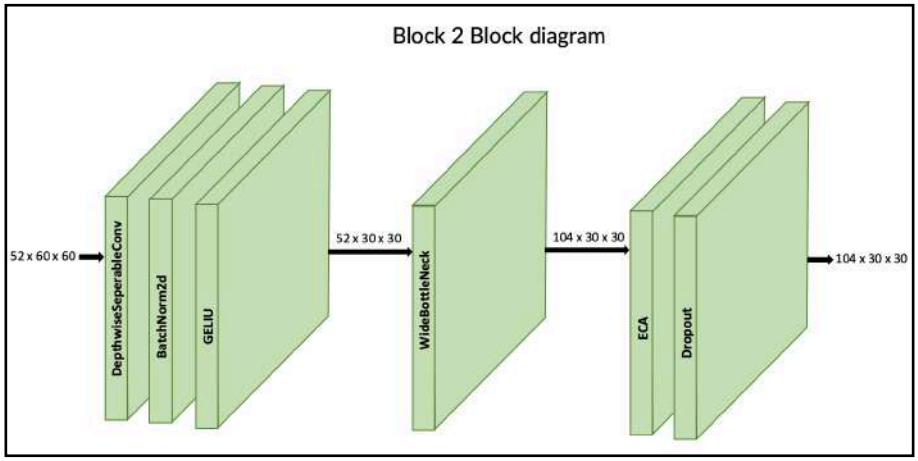


Figure B.4. Block 2 Diagram (CustomModelV9)

B.1.3.4 CustomModelV9 Block 3: Deep Feature Learning

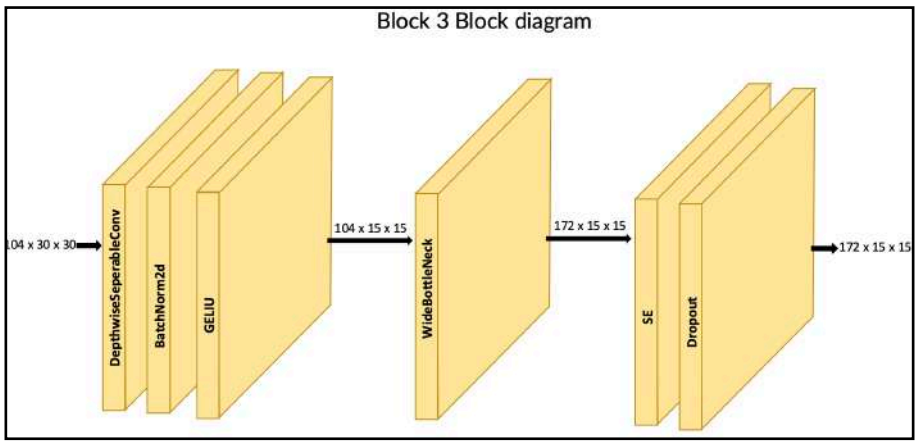


Figure B.5. Block 3 Diagram (CustomModelV9)

Block 3 continues this abstraction, reducing the spatial dimensions from **(30, 30)** to **(15, 15)** and expanding depth to **172 channels**. It keeps the structure of Block 2, retaining Depthwise Seperable Convolution and WideBottleNeck, but replacing ECA with a Squeeze-and-Excitation (SE) module for high-level spectral recalibration. In addition, a 1.5x dropout rate is applied to control overfitting. The extracted features serve as the foundation for high-level semantics and are used in the following fusion steps.

B.1.3.5 CustomModelV9 Block 4: Multi-Scale Feature Fusion

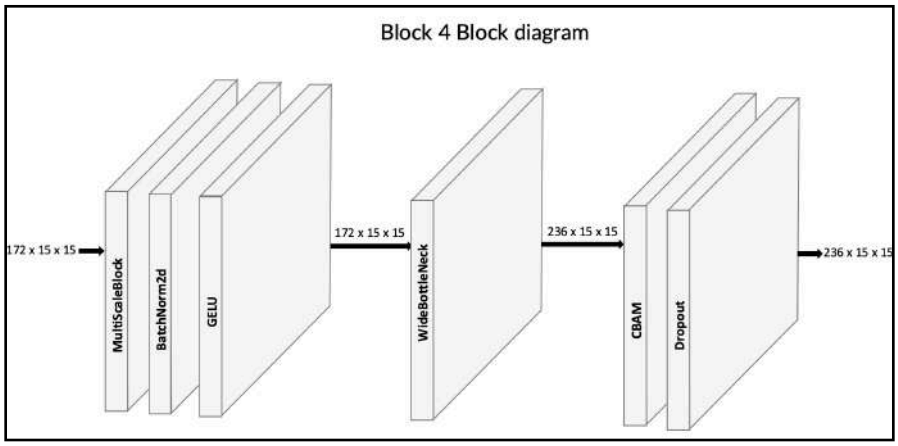


Figure B.6. Block 4 Diagram (CustomModelV9)

Block 4 enhances the high-level representation by applying a MultiScaleBlock (with four groups), combining parallel dilated convolutions at different receptive fields (1, 2, 3). The output depth is increased to **236** channels while keeping spatial dimensions at **(15, 15)**. A final WideBottleneck is applied, followed by a Convolutional Block Attention Module (CBAM) that merges spatial and channel-wise focus. Dropout remains at 1.5x. The block outputs are fed into Fusion 2 before entering the classifier.

B.1.3.6 CustomModelV9 Classification Head

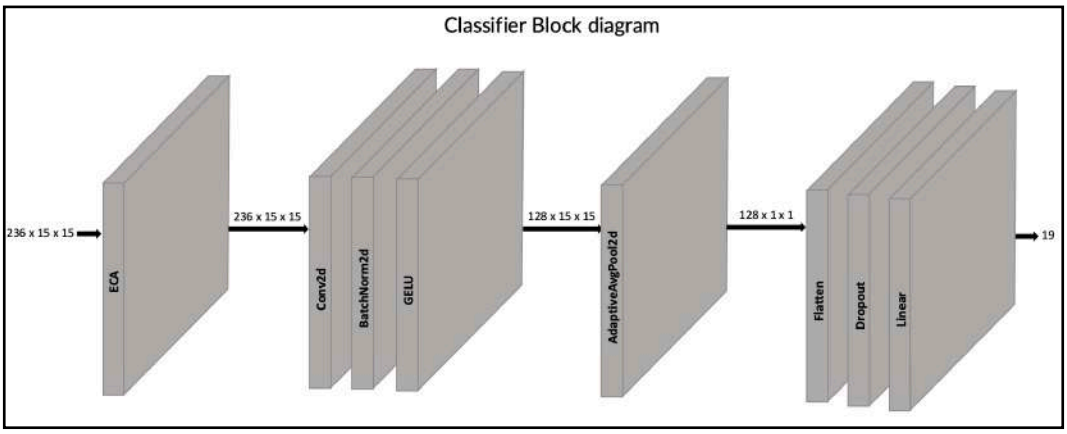


Figure B.7. Classifier Block Diagram (CustomModelV9)

The classification head receives the fused high-level features and converts them into multi-label predictions. This is done by applying ECA to emphasise spectral channels, and then projecting the tensor from **236 channels** to **128** using a 1x1 convolution. This

is followed by BatchNorm2d, GeLU activation, and Global Average Pooling to reduce the spatial dimension. A final Dropout layer and fully connected classifier produce logits for all 19 land cover classes.

Figure B.8 illustrates the complete forward path of CustomModelV9, from initial Spectral Mixing through the four modular blocks, dynamic fusion stages, and the final classifier.

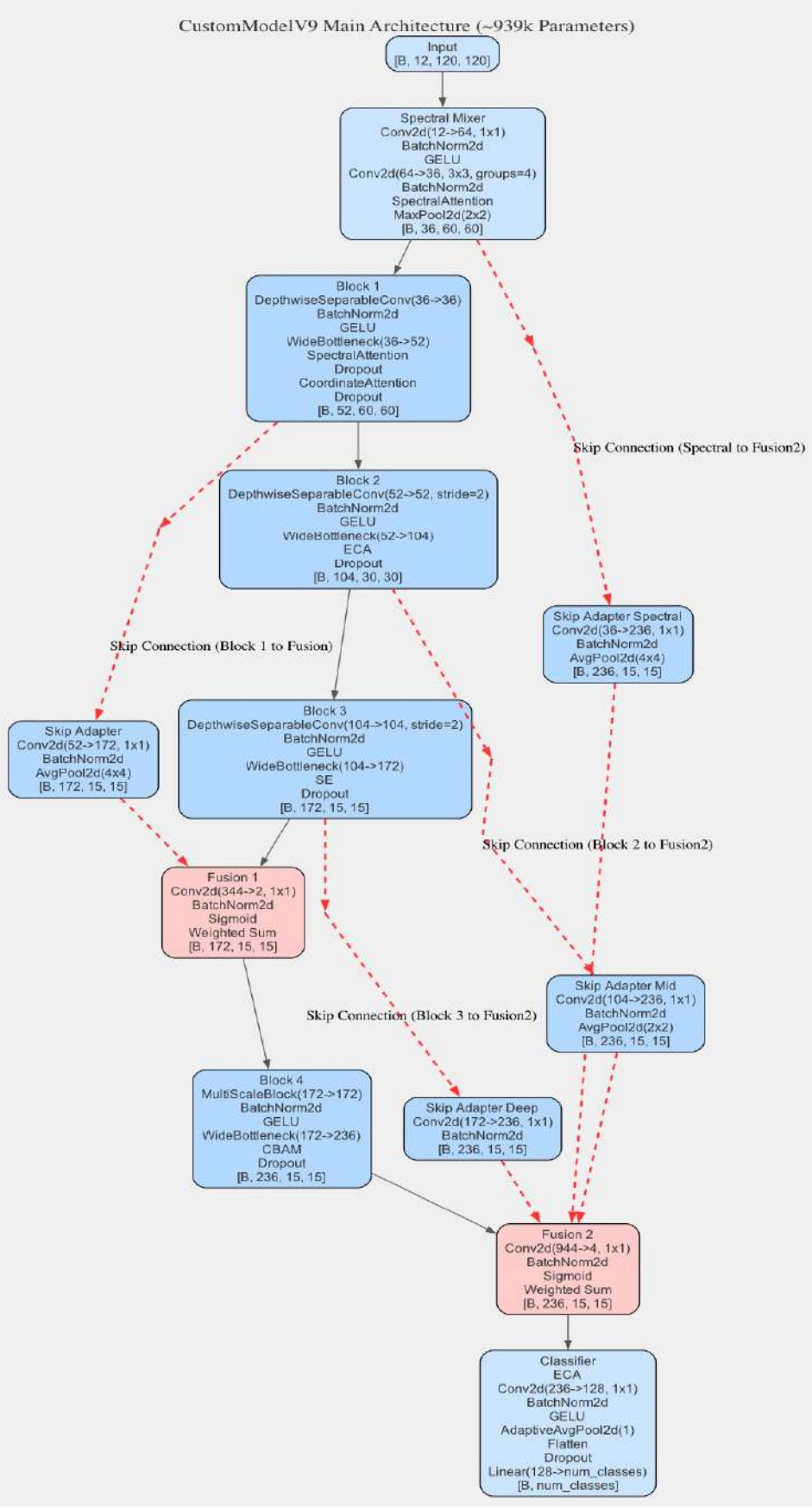


Figure B.8. CustomModelV9 Detailed Architecture Overview

B.1.4 Attention Mechanisms in CustomModelV9

CustomModelV9 integrates five distinct mechanisms to enhance the network's ability to focus on relevant spectral, spatial, and contextual cues throughout the feature hierarchy. These mechanisms help suppress irrelevant noise and reinforce class-specific features [79, 80], and are particularly effective in RS tasks when overlapping LULC types are common. The attention modules used in V9 can be grouped into three primary categories.

I. **Spectral (Channel-wise) Attention:** This attention focuses on the importance of each channel or spectral band, by learning to weigh features from different bands based on their relevance to the target task.

II. **Spatial Attention:** This attention enhances the model's ability to focus on key spatial regions within feature maps, highlighting "where" important features are located in the image.

III. **Coordinate Attention:** This is a hybrid form of spatial attention that encodes positional information along height and width separately, maintaining location-specific semantics.

These mechanisms are applied at different depths in the model, each tailored for the role of different blocks. The following sections detail each module's design, role, and placement within CustomModelV9.

B.1.4.1 Spectral Attention (SA)

Inspired by [73], Spectral Attention (SA) is a lightweight channel-wise mechanism that captures global contextual dependencies across spectral bands by learning channel-wise importance. It applies global average pooling, followed by two linear layers with ReLU activation to generate attention weights. These are added to the original feature map to enhance informative bands without entirely suppressing weaker ones.

In CustomModelV9, SA is used within the Spectral Mixer and Block 1, enabling early-stage feature maps to prioritise informative channels before any spatial attention is introduced. Figure B.9 illustrates the internal process of the SA mechanism, highlighting the sequence of global pooling, transformation, and additive channel-wise enhancement.

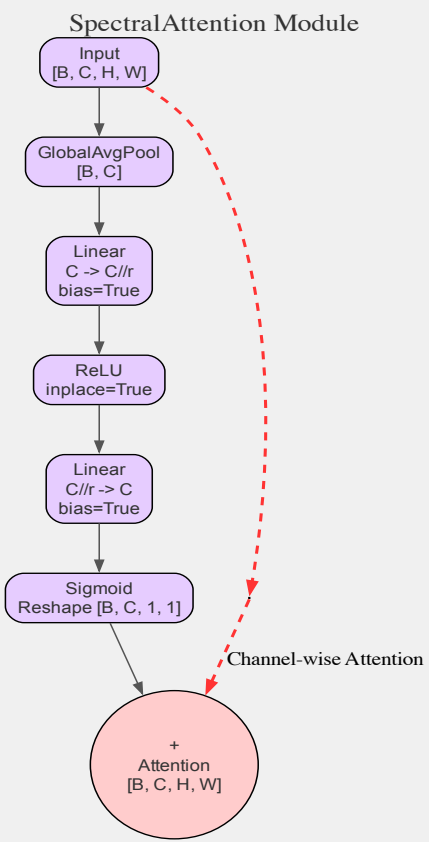


Figure B.9. Spectral Attention (SA) Implementation

B.1.4.2 Coordinate Attention (CA)

Coordinate Attention, based on the work of Hou et al. [74], is a spatial attention mechanism that enhances feature maps by preserving directional position information through separate pooling operations along the height and width. This allows for the capture of long-range dependencies and the learning of axis-specific importance, making it particularly important in RS applications where features like roads or rivers often follow strong spatial orientations.

In CustomModelV9, CA is integrated in Block 1, immediately after SA and WideBottleneck processing. This placement allows the model to encode early spatial cues while retaining spectral focus with minimal computational costs. Figure B.10 visualises the internal flow of the Coordinate Attention mechanism, showing how axis-specific pooling and convolution operations allow the model to learn and apply spatially-aware attention maps.

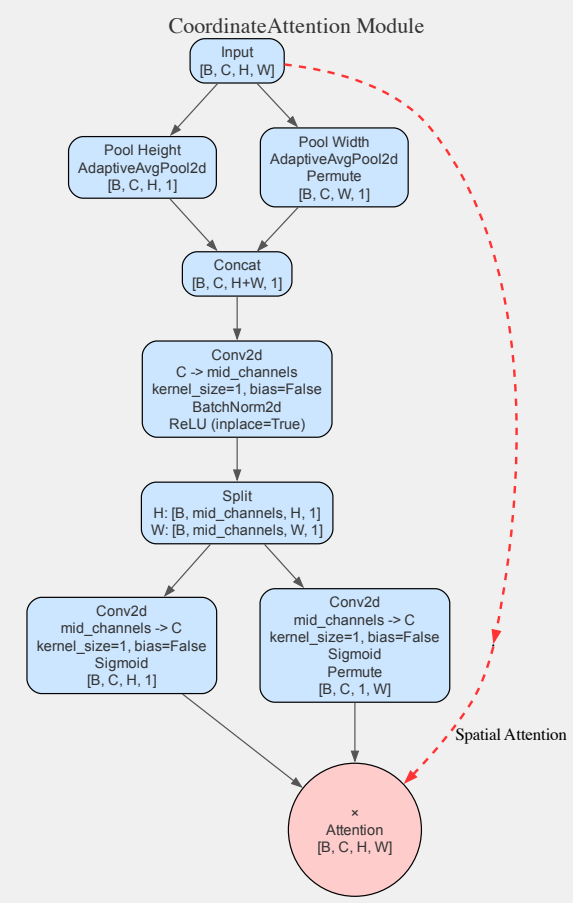


Figure B.10. Coordinate Attention (CA) Implementation

B.1.4.3 Efficient Channel Attention (ECA)

Efficient Channel Attention (ECA), adapted from [76], is a lightweight channel-wise mechanism that enhances feature selectivity without relying on dimensionality reduction. ECA captures local channel interactions by applying a lightweight 1D convolution to globally pooled features. These operations allow the model to refine its focus on relevant channels while preserving the whole feature space.

In CustomModelV9, ECA is utilised in Block 2 for mid-level feature refinement and again within the Classifier Head to emphasise key channels before final prediction. ECA's use of adaptive kernel sizing ensures it is scalable across varying channel depths, providing for subtle but important LULC cues with minimal cost (~7 parameters). The design of ECA is shown in Figure B.11, illustrating the convolutional filtering over pooled channel descriptors.

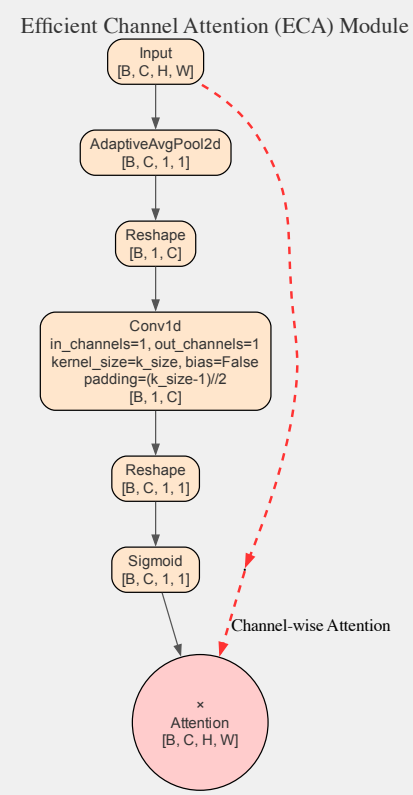


Figure B.11. Efficient Channel Attention (CA) Implementation

B.1.4.4 Squeeze and Excitation (SE)

Squeeze-and-Excitation (SE), originally proposed in [77], is a channel attention mechanism that recalibrates channel-wise features by modelling interdependencies. As shown in Figure B.12, SE compresses spatial information using adaptive average pooling and passes the result to a two-layer bottleneck (1×1 conv -> ReLU -> 1×1 conv) to generate channel-wise attention weights. These are then passed through a sigmoid and multiplied with the original input, allowing for amplified informative bands.

SE is applied within Block 3 of CustomModelV9, where high-level abstract features are extracted. Compared to lighter modules like ECA, SE offers greater expressiveness, making it more suitable for deeper feature recalibration.

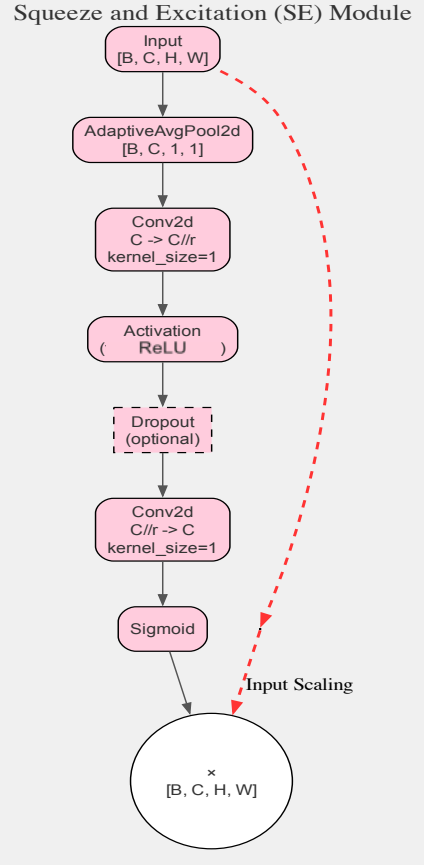


Figure B.12. Squeeze-and-Excitation (SE) Implementation

B.1.4.5 Convolutional Block Attention Module (CBAM)

The Convolutional Block Attention Module (CBAM), introduced in [75], is a two-stage attention mechanism that sequentially applies channel and spatial attention to enhance both "what" and "where" the network focuses. It first generates channel-wise attention weights using Adaptive Pooling and 1x1 convolutions, followed by spatial attention using a 7x7 convolution over average and max-pooled spatial descriptors.

In CustomModelV9, CBAM is applied at the deepest stage (Block 4), where semantic abstraction is high and spatial resolution is low. This allows for emphasis on important regions and spectral bands, improving performance for fine-grained LULC classes. Figure B.13 illustrates the full attention pipeline of CBAM, showcasing its two-stage mechanism.

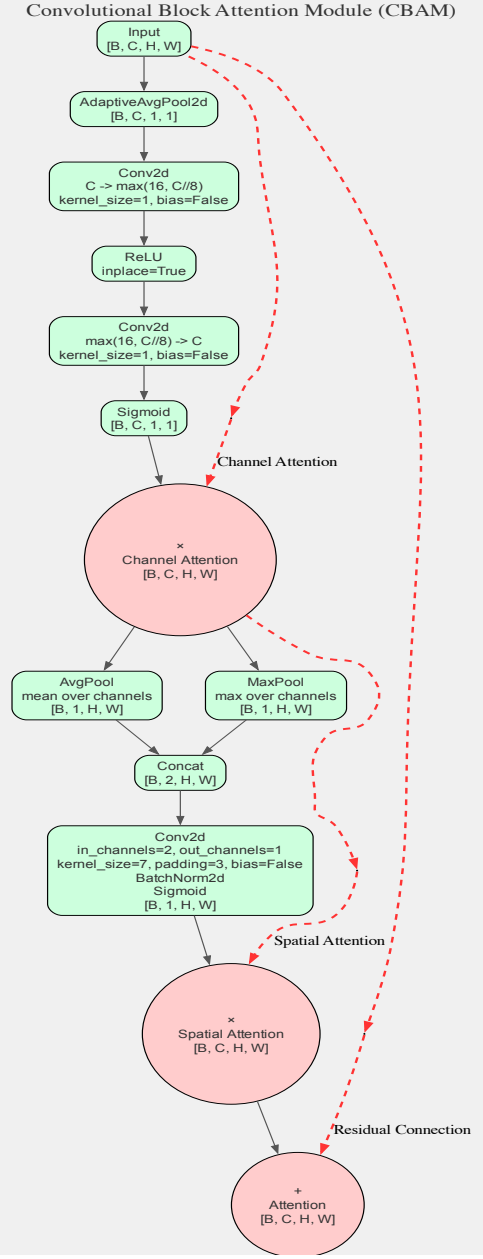


Figure B.13. Convolutional Block Attention Module (CBAM) Implementation

B.1.5 Enhancement Modules in CustomModelV9

In addition to the attention mechanisms outlined above, CustomModelV9 integrates several enhancement modules that improve the network's ability to extract, refine, and represent features across multiple spatial and spectral scales.

B.1.5.1 Wide Bottle Neck

Adapted from Wide Residual Networks (WRNs) [72], Wide Bottlenecks expand the internal channel width while preserving shallow depth. They feature a three-layer

design (1×1 -> 3×3 -> 1×1 convolutions), with widened intermediate channels that improve feature expressiveness without excessive depth or parameters.

In CustomModelV9, Wide Bottlenecks are used in Blocks 1 through 4, processing low-, mid-, deep-, and high-level features, respectively. Their wide and shallow structure complements attention mechanisms and facilitates the extraction of diverse spatial and spectral patterns essential for multi-label LULC classification.

B.1.5.2 Depth-wise Separable Convolution (DepthwiseSeparableConv)

Inspired by the work of Chollet [71], Depthwise Separable Convolutions decompose standard convolutions into two lightweight operations:

I. A depthwise convolution applies one filter per input channel.
II. A pointwise (1x1) convolution combines outputs across channels.

This decomposition significantly reduces computational complexity while retaining spatial feature capability. In CustomModelV9, these operations are applied in Blocks 1 to 3 to extract features efficiently across multiple resolutions.

B.1.5.3 Multi-Scale Block

Inspired by the work in [78], the MultiScaleBlock captures multi-scale contextual information by combining three parallel depth-wise separable convolutions with increasing dilation rates (1, 2, and 3). These parallel convolutions extract local, mid-range, and global features, which are then concatenated and fused using a 1x1 convolution to produce a unified output, leading to enhanced spatial awareness.

In CustomModelV9, this block is placed in Block 4, which refines high-level features, improving the model's ability to distinguish complex land cover patterns.

B.1.6 Dynamic Fusion and Skip Connections in CustomModelV9

As introduced in Chapter 4.6, CustomModelV9 utilises a two-stage fusion strategy combined with dimension-aligned skip connections. This design draws on common principles in multi-scale feature integration and was tailored to maintain and combine features across varying levels of abstraction. By preserving information from shallow, intermediate, and deep stages of the network, the model ensures balanced feature representation before final classification.

B.1.6.1 Stage 1: Deep-Shallow Fusion

In this first stage, outputs from **Block 3 (172, 15, 15)** and a dimension-matched skip connection from **Block 1** (adapted to the exact shape) are fused. A small convolutional block generates two attention masks, one for deep features (w_{deep}) and one for shallow features (w_{low}). These masks are applied as soft weights and used to compute a weighted sum of the two inputs, producing a fused tensor passed into Block 4. The final fused representation of Fusion 1 is computed as:

$$\text{Fusion1} = w_{\text{deep}} \cdot \text{featuresdeep} + w\text{low} \cdot \text{adapted}_{\text{low}}$$

B.1.6.2 Stage 2: Multi-Stream Fusion

After **Block 4**, a second fusion stage combines its output with skip-adapted features from the **Spectral Mixer**, **Block 2**, and **Block 3**. The four tensors are concatenated into a single tensor of shape **(944, 15, 15)**. A secondary fusion block then learns four attention weights (one per stream). To ensure numerical stability, these weights are normalised using the following expression:

$$w_i' = \frac{w_i}{\sum_i w_i + \epsilon}$$

The final fused tensor, Fusion2, is computed as the attention-weighted sum of all four input streams:

$$\text{Fusion2} = w_{\text{high}} \cdot F_{\text{high}} + w_{\text{mid}} \cdot F_{\text{mid}} + w_{\text{deep}} \cdot F_{\text{deep}} + w_{\text{early}} \cdot F_{\text{early}}$$

This final fusion ensures that spatially aligned, semantically diverse information is balanced before entering the classifier. Figure B.8 provides a visual representation of both fusion strategies.

B.2 State-of-the-Art Sample Modifications

B.2.1 ResNet18 Modification

```
class ResNet18(BaseModel):
  def __init__(self, class_weights, num_classes, in_channels, model_weights, main_path):
    resnet_model = resnet18(weights=model_weights)
```

original_conv1 = resnet_model.conv1

resnet_model.conv1 = nn.Conv2d(in_channels, original_conv1.out_channels, original_conv1.kernel_size, original_conv1.stride, original_conv1.padding, original_conv1.bias)

nn.init.kaiming_normal_(resnet_model.conv1.weight, mode='fan_out', non-linearity='relu')

resnet_model.fc = nn.Linear(resnet_model.fc.in_features, num_classes)

super(ResNet18, self).__init__(resnet_model, num_classes, class_weights, in_channels, main_path)

B.2.2 Swin-Transformer Modification

class SwinTransformer(BaseModel):

def __init__(self, class_weights, num_classes, in_channels, model_weights, main_path):

model_weights = bool(model_weights)

swin_model = timm.create_model('swin_base_patch4_window7_224', pre-trained=model_weights, num_classes=num_classes, in_chans=in_channels, img_size=120)

super(SwinTransformer, self).__init__(swin_model, num_classes, class_weights, in_channels, main_path)

B.3 Hyper-Parameters and Training Settings

Table B.3. General Training Hyper-Parameters

Category	Parameter	Value
Training	Epochs	50
	Batch Size	256
	Num Workers	8
	Early Stopping Patience	10
Optimiser	Optimiser	AdamW
	Learning Rate	0.001
	Weight Decay	0.01
Scheduler	Scheduler	ReduceLROnPlateau
	LR Factor	0.5
	LR Patience	4
Loss Function	Loss Function	CombinedFocalLossWithPOS
	Focal Alpha	0.5
	Focal Gamma	3.0

B.3.1 Combined Focal Loss with Positive Weight Implementation

```
import torch

import torch.nn as nn

import torch.nn.functional as F

class CombinedFocalLossWithPosWeight(nn.Module):

  def __init__(self, pos_weight, alpha=ModuleConfig.focal_alpha,
gamma=ModuleConfig.focal_gamma, reduction='mean'):

    super(CombinedFocalLossWithPosWeight, self).__init__()

    self.pos_weight = pos_weight

    self.alpha = alpha

    self.gamma = gamma

    self.reduction = reduction

  def forward(self, inputs, targets):

    bce_loss = F.binary_cross_entropy_with_logits(inputs, targets, pos_weight=self.pos_weight,
reduction='none')

    probas = torch.sigmoid(inputs)

    p_t = probas * targets + (1 - probas) * (1 - targets)

    focal_modulation = (1 - p_t) ** self.gamma

    alpha_factor = self.alpha * targets + (1 - self.alpha) * (1 - targets)

    loss = alpha_factor * focal_modulation * bce_loss

    return loss
```

B.4 Experiment Configurations

The primary experiment configuration used for evaluation involved training each model architecture, including both custom and SOTA models, on the **10% subset of the dataset, using all spectral bands, without any pre-trained weights**. Each model was trained for 50 epochs under this setup to ensure consistency and fairness compared to different architectures.

In addition to the main configuration, several smaller-scale experiments were conducted for quicker evaluations. For instance, ResNet50 and the CustomModel were trained on the **1% subset without pre-trained weights for 20 epochs**. These

experiments were performed across all band combinations to determine the most effective spectral input. A similar approach was followed when evaluating various loss functions, utilising smaller subsets to efficiently identify the best-performing configurations without extensive training time.

Appendix C: Computational Resources & Training Setup

C.1 Hardware Specifications and Performance

Two systems were evaluated to assess computational performance and scalability for training models on BigEarthNet. System 1 serves as the primary setup, with the 10% BigEarth subset (47,948 patches) as the primary training dataset. Other dataset subsets (0.5%, 1%, 5% and 50%) were used for testing and scalability analysis. System 2 was explored for comparison but proved unreliable due to memory constraints.

Table C.1. Hardware Specifications

Component	System 1 (Primary)	System 2 (Exploratory)
CPU	Intel Core i5-1200F (6 cores, 12 logical processors)	Intel Core i7-14700F (20 cores, 28 logical processors)
GPU	NVIDIA GeForce RTX 3050 (8GB VRAM)	NVIDIA GeForce RTX 4060 (8GB VRAM)
RAM	32 GB DDR4	16 GB DDR5
Storage	1TB NVMe SSD	1TB NVMe SSD
Operating System	Windows 11	Windows 11

Table C.2. Subset Performance Metrics

Avg. Time per Epoch	System 1 (Primary)	System 2 (Exploratory)
0.5% Subset	30sec	10sec
1% Subset	1m30sec	20sec
5% Subset	4min	1m45sec
10% Subset	17min30sec	5min30sec
50% Subset	2hrs	37min
100% Subset	4hrs30min	1hr40min

System 1 was chosen as the primary setup due to its stability and sufficient memory. System 2 achieved up to 3x performance improvements over System 1 due to a superior CPU and GPU; however, it was restricted to smaller batch sizes (16-64) compared to System 1's support for a batch size of up to 256. System 2 was susceptible

to frequent out of memory errors and crashes due to lower RAM, which limited its reliability for full-scale training.

The final set of experiments included within the report were conducted on a separate hardware setup provided by Prof. John Abela. While not described in detail here, this system offered the necessary stability and performance needed to complete the final training runs effectively.

C.2 Python Environment and Dependencies

Library versions were structured to ensure compatibility and avoid dependency conflicts, and they were managed using Anaconda to ensure consistency across preprocessing, training, testing, and web deployment workflows. Table C.4 illustrates the main project dependencies:

Table C.4. Main Project Dependencies

Library	Version	Purpose
Python	3.11.10	Core language
PyTorch	2.4.1	Deep learning framework
PyTorch Lightning	2.4.0	Training abstraction and logging
Torchvision	0.19.1	CNN models and image transformations
timm	1.0.12	Image model library (including transformers)
TorchMetrics	1.4.2	Multi-label evaluation metrics
TorchCAM	0.4.0	Grad-CAM visualisation
TorchViz	0.0.2	Model graph visualisation
Scikit-learn	1.6.0	Additional metrics and evaluation utilities
Pillow, OpenCV	11.1.0	Image loading and augmentation
Rasterio	1.3.11	Reading multi-band geospatial TIFFs
Matplotlib, Seaborn	3.1.0, 0.13.2	Plotting and heatmaps
Pandas, Numpy	2.2.3, 1.23.5	Data handling and numerical computation
Flask	3.1.0	Used for web-based deployment interface

C.3 Code Availability

To facilitate reproducibility and enable further research, the complete codebase for this project is publicly available on GitHub. The repository includes all scripts for data

preprocessing, model implementations (CustomModelV1–V9 and SOTA models such as ResNet50 and Swin Transformer), training and evaluation pipelines, and the Flask-based web application described in Appendix D. It also contains a comprehensive README with detailed instructions for setting up the environment, downloading and preprocessing the BigEarthNet dataset, running experiments, and deploying the web application.

GitHub Repository: https://github.com/StaticRevo/FYP-Multi-Label-Classification-using-Deep-Learning

Appendix D: Web-Based Application

This appendix elaborates on the web-based application developed to enhance the practical deployment of the RS image classification system. Built with *Flask*, the interface provides access to trained models for testing and prediction on BigEarthNet imagery, with support for training and advanced visualisations.

D.1 Overview Of Application

The web application acts as a deployment platform for the RS image classification system, enabling users to interact with the custom CNN and state-of-the-art models trained on BigEarthNet data. It facilitates initiating training runs, testing pre-trained models, and performing inference on uploaded RS images or satellite patches retrieved from an interactive map. Features include real-time log monitoring, batch predictions, and Grad-CAM [70] visualisations. Hosted locally at *127.0.0.1:5000* in debug mode, key capabilities include user-driven training with customisable parameters, evaluation of pre-trained checkpoints, and interactive visual analysis of model performance.

Table D.1. Summary of Web Application Features

Feature	Description
Training	Initiate model training with user-selected parameters (e.g., model, bands, dataset subset).
Testing	Evaluate pre-trained models using selected checkpoints (e.g., best accuracy, last).
Prediction	Perform inference on single or batch TIFF uploads, with label predictions and visualisations.
Interactive Map Prediction	Select geographic patches on a map to retrieve Sentinel-2 data, perform predictions, and view heatmaps.
Log Monitoring	Display real-time training and testing logs via /logs and /logs_test routes.
Experiment Overview	Filter and sort trained experiments, displaying metadata and creation dates.
Detailed Inference	Compare performance metrics across multiple experiments with visualisations.
Bubble Chart	Visualise model performance metrics (F2-score vs training time) interactively.
Architecture Display	Show model architecture details from stored files for selected experiments.
Data Exploration	Render a Jupyter notebook (data_exploration.ipynb) for dataset insights.

D.2 Application Interface Screenshots

The application interface includes several pages rendered via *Flask* templates. The home page (*index.html)* serves as the central navigation hub, providing links to training, testing, prediction, experiment management, visualisation and data exploration sections, guiding users to the application's core functionalities.

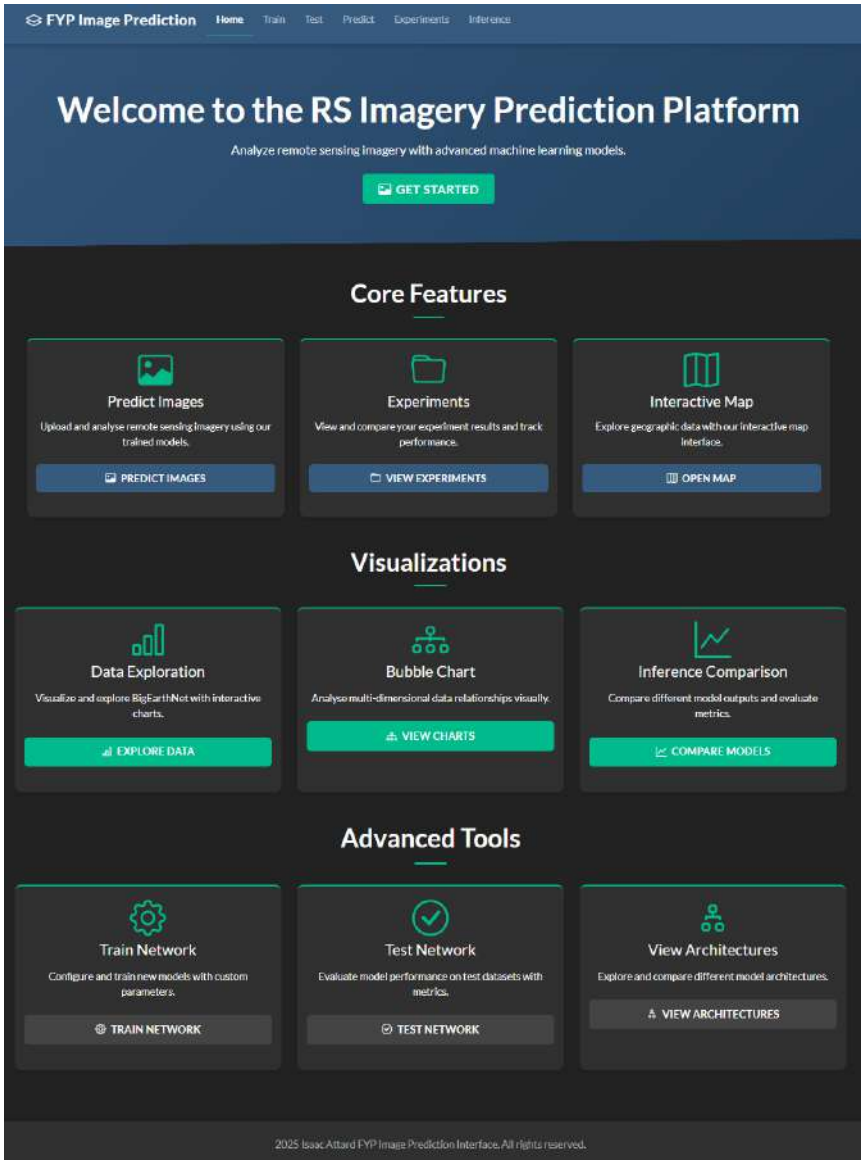


Figure D.1. Home Page (index.html)

The training and testing interfaces streamline model development workflows. The training page (*train.html*) allows users to configure runs by selecting models (e.g., CustomModel, ResNet50), weights (e.g., None, DEFAULT), band combinations (e.g., RGB, all_bands), and dataset subsets (e.g., 10%, 50%) with *train_status.html* displaying

real-time progress and logs accessible via /logs. Similarly, the testing interface (*test.html* and *test_status.html*) lists untested experiments, enabling the evaluation of pre-trained models by selecting checkpoints.

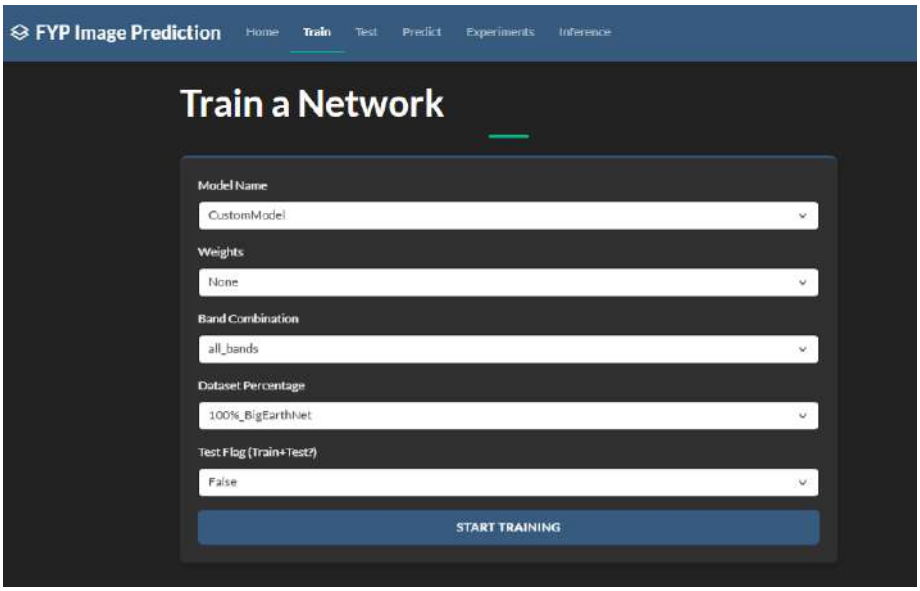


Figure D.2. Train Page (train.html)

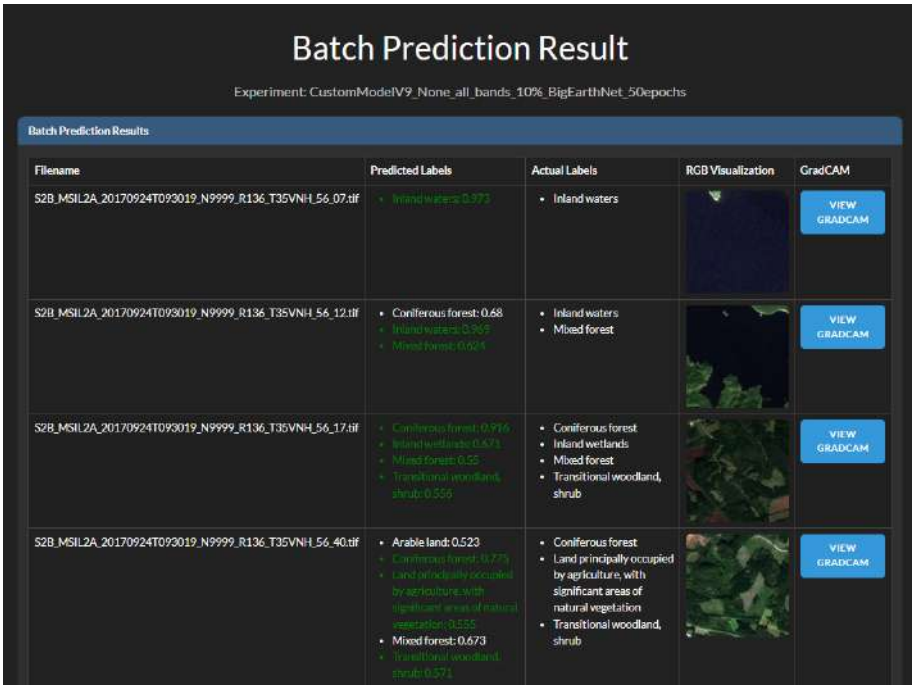


Figure D.3. Batch Result Page (batch_result.html)

Prediction functionality is a core functionality within the app and is supported by a multi-step interface. The upload page (*upload.html*) facilitates single or batch TIFF

uploads and experiment selection, while *select_bands.html* appears if band mismatches occur, allowing users to choose specific bands. Results are displayed on *result.html*, showing RGB images, predicted labels with probabilities, the actual labels, and Grad-CAM [70] visualisations of the predictions (see Figure D.4). Related pages like *batch_result.html* and *batch_gradcam_result.html* handle batch predictions and Grad-CAM [70] visualisations for batches (see Figure D.3).

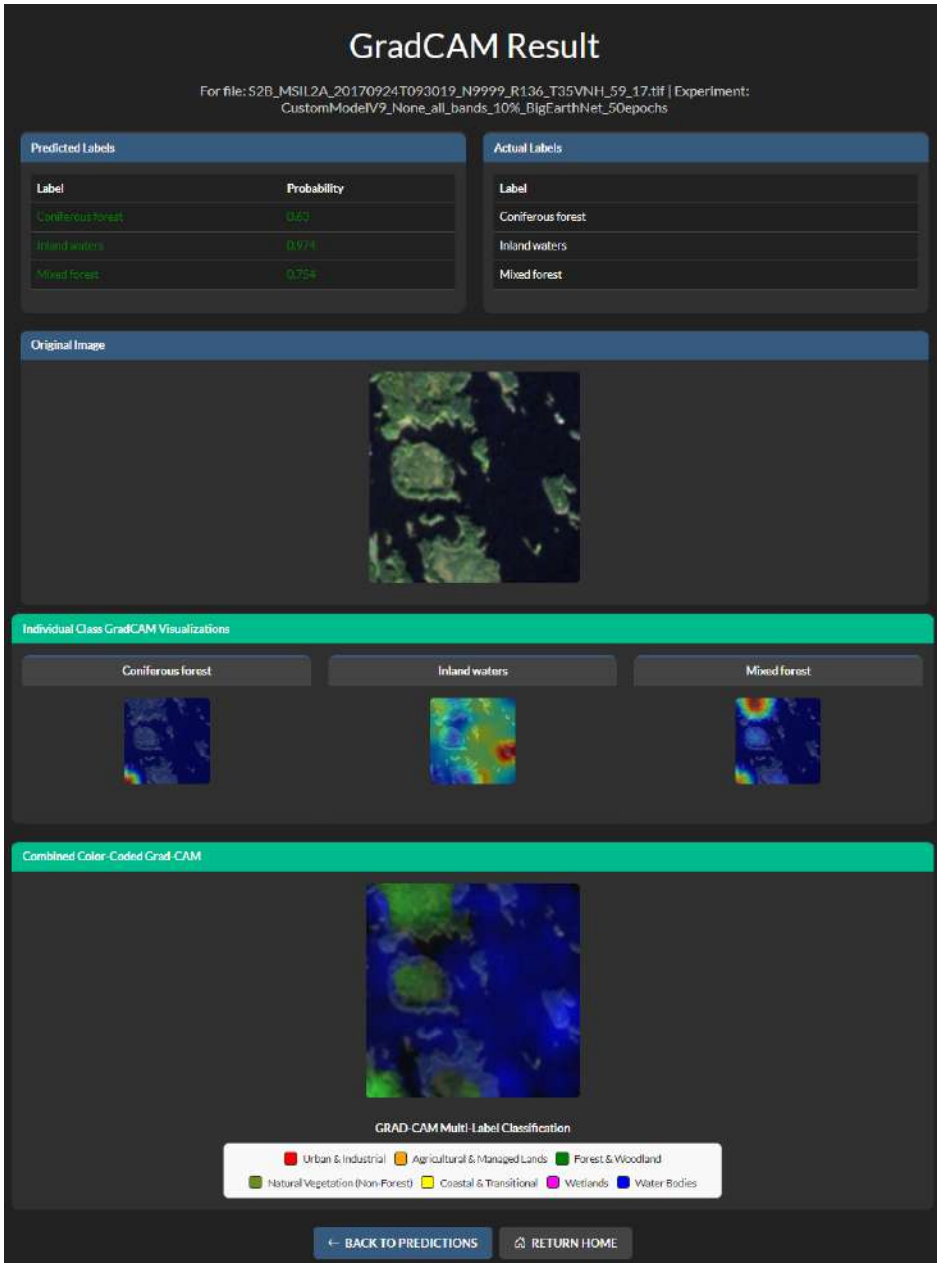


Figure D.4. Single Result Page (result.html)

The interactive map page (map_page.html) further enhances prediction capabilities by allowing users to select locations on a map to retrieve Sentinel-2 patches for inference. Users click on the map to fetch a 120x120 multi-spectral TIFF, visualised as an RGB overlay. After choosing a pre-trained experiment, the system predicts land cover labels and displays them with probabilities alongside Grad-CAM [70] heatmaps, providing a straightforward way of analysing real-world satellite data directly within inference.

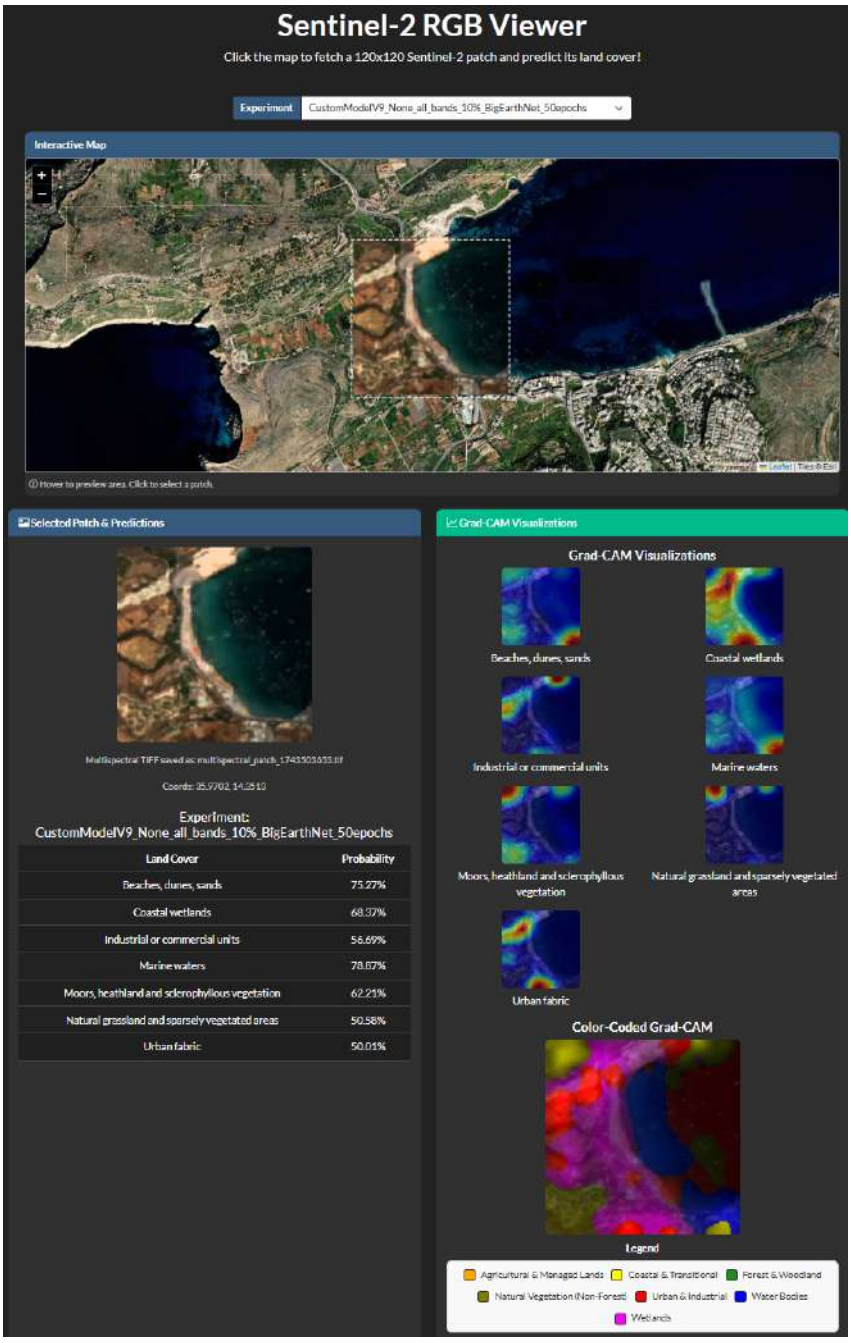


Figure D.5. Interactive Map Page (map_page.html)

Experiment management and visualisation features further enhance analysis capabilities. Pages like *experiments_overview.html* allow browsing and filtering of trained models, while *experiments_detail.html* provides in-depth analysis with metrics and visualisations (e.g., confusion matrices). Experiment comparison can be made efficiently through the *select_experiments.html* and *detailed_inference.html*, where users can select multiple experiments and compare their performance through detailed tables of aggregated and per-class metrics with metrics such as F-scores and inference time, facilitating model evaluation. To clarify the results, the best values for each metric are highlighted using the respective experiment's colour.

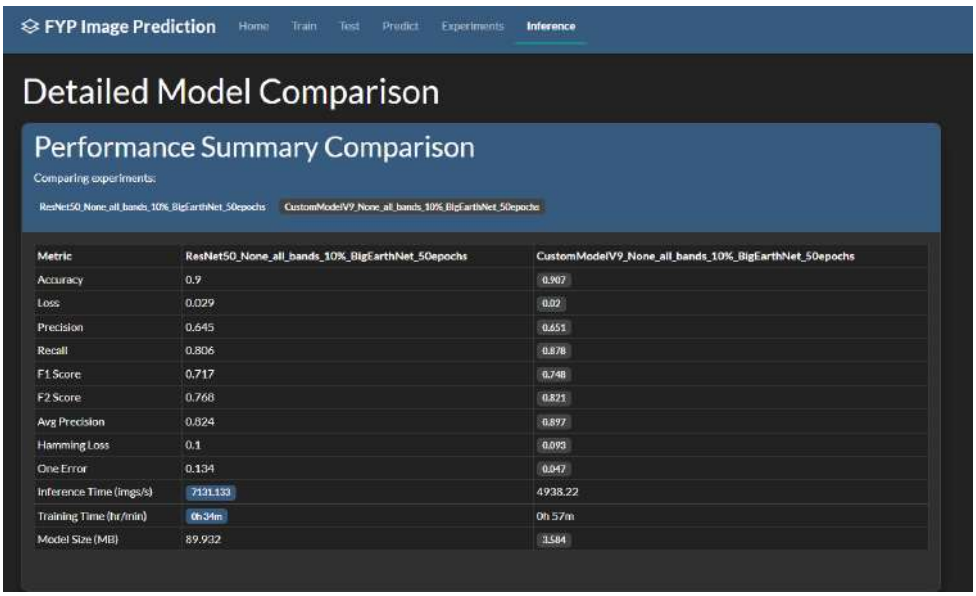


Figure D.6. Inference Comparison Page (detailed_inference.html)

Additional visualisation pages, such as *bubble_chart.html*, offer an interactive chart displaying various models based on the user's filter. The chart plots the models as bubbles, where their size represents the parameter count and their position the respective F2-score and training time. While *architectures.html* displays the model's architecture details, and the *notebook_view.html* page renders a data exploration notebook as HTML.



Figure D.7. Bubble Chart Page (bubble_chart.html)

D.3 Backend Architecture

The backend architecture, implemented primarily in *app.py* with support from *web_helper.py*, follows a modular Flask-based design. The *app.py* script defines routes such as /train, /test, and /predict to handle GET and POST requests, triggering training and testing processes by launching external scripts (trainer.py and tester.py) via subprocess *Popen* with parameters like model names and checkpoint paths.

The *web_helper.py* module provides essential functions, including load_model_from_experiment to retrieve trained models from checkpoint files (e.g., last.ckpt), *preprocess_tiff_image* to prepare uploaded TIFFs using transforms from *TransformConfig*, and *predict_image_for_model* for inference using PyTorch. Visualisation features, such as Grad-CAM [70] heatmaps, with outputs saved to a static folder and uploads managed in an uploads directory, ensuring a clear separation of logic and data handling.

Appendix E: Evaluation Metrics & Results

E.1 Metrics In Detail

This section defines the metrics used in Chapter 5 to evaluate the custom CNN and SOTA models for multi-label classification.

I. **Precision** measures the proportion of predicted positive labels that are correct. This is crucial for minimising false positives in overlapping classes. The formula is:

$$\text{Precision} = \frac{TP}{TP + FP}$$

where TP is true positives and FP is false negatives.

II. **Recall** assesses the fraction of actual positive labels correctly identified, this is important for capturing all relevant classes within a patch, and its formula is:

$$\text{Recall} = \frac{TP}{TP + FN}$$

where FN is false negatives.

III. **F1-Score** balances both precision and recall as their harmonic mean, making it a useful general performance indicator, and has the following formula:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

IV. **F2-Score** prioritises recall over precision, allowing it to address class imbalance by emphasising rare class detection and has the following formula:

$$F2 = \frac{5 \times \text{Precision} \times \text{Recall}}{4 \times \text{Precision} + \text{Recall}}$$

V. **Mean Average Precision (mAP)** evaluates ranking across multiple labels by averaging precision at each recall level, making it suitable for multi-label tasks.

$$mAP = \frac{1}{N} \sum_{i=1}^{N} \text{Average Precision}_i$$

VI. **Hamming Loss** calculates the fraction of incorrect prediction labels which is averaged over samples and labels.

$$\text{Hamming Loss} = \frac{1}{N \times L} \sum_{i=1}^{N} \sum_{j=1}^{L} I(y_{ij} \neq \hat{y}_{ij})$$

where N is the number of samples, L is the number of labels and I is the indicator function.

VII. **One Error** measures the fraction of samples where the top-ranked label is incorrect.

$$\text{One Error} = \frac{1}{N} \sum_{i=1}^{N} I(\hat{y}_i \neq y_i)$$

This study prioritised Recall, F2-Score, Hamming Loss, and mAP metrics. Recall ensures that all relevant labels, including rare classes, are identified. F2-Score reinforces this priority by weighting Recall more heavily than Precision, crucial for handling the class imbalance present in BigEarthNet. Hamming Loss measures label-wise errors, making it critical in this multi-label setting. Lastly, mAP assesses the quality of label ranking, ensuring the model assigns higher confidence to the most relevant classes. Together, these metrics capture performance, error distribution, and ranking challenges in multi-label RS data classification.

E.2 V9 Per-Class Performance Highlights

Table E.1. CustomModelV9 Per-Class Classification performance

Class	Precision	Recall	F1	F2	Accuracy
0. Agro-forestry areas	0.590	0.956	0.729	0.850	0.941
1. Arable land	0.859	0.865	0.862	0.864	0.882
2. Beaches, dunes, sands	0.368	0.700	0.483	0.593	0.999
3. Broad-leaved forest	0.687	0.890	0.775	0.840	0.844
4. Coastal wetlands	0.114	0.667	0.195	0.339	0.995
5. Complex cultivation patterns	0.505	0.842	0.631	0.743	0.825
6. Coniferous forest	0.793	0.942	0.861	0.908	0.900
7. Industrial or commercial units	0.252	0.520	0.339	0.429	0.964

Class	Precision	Recall	F1	F2	Accuracy
8. Inland waters	0.781	0.869	0.823	0.850	0.948
9. Inland wetlands	0.332	0.729	0.456	0.589	0.938
10. Land principally occupied by agriculture, with significant areas of natural vegetation	0.502	0.835	0.627	0.737	0.759
11. Marine waters	0.965	0.991	0.978	0.996	0.996
12. Mixed forest	0.740	0.926	0.823	0.882	0.850
13. Moors, heathland and sclerophyllous vegetation	0.427	0.863	0.571	0.716	0.961
14. Natural grassland and sparsely vegetated areas	0.381	0.595	0.464	0.535	0.978
15. Pastures	0.657	0.861	0.745	0.810	0.868
16. Permanent crops	0.352	0.709	0.471	0.590	0.927
17. Transitional woodland, shrub	0.561	0.883	0.686	0.792	0.734
18. Urban fabric	0.632	0.802	0.707	0.761	0.9295

E.3 Further Sample Visualisations

E.3.1 Additional Grad-CAM visualisation

The Grad-CAM [70] visualisations within this section serve as additional evidence to the heatmaps provided in Chapter 5. These images provide additional result context, to demonstrate CustomModelV9's consistency across varied scenes.

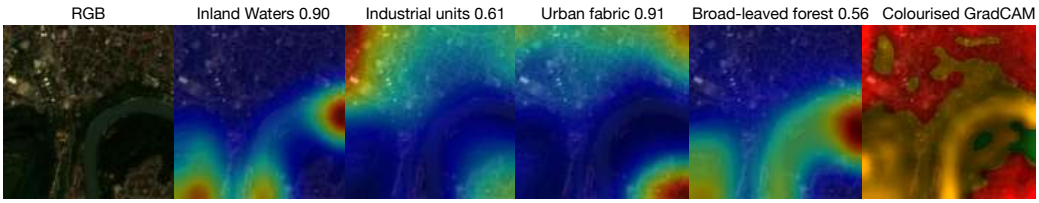


Figure E.1. Patch T33UUP_32_52 CustomModelV9 Grad-CAM

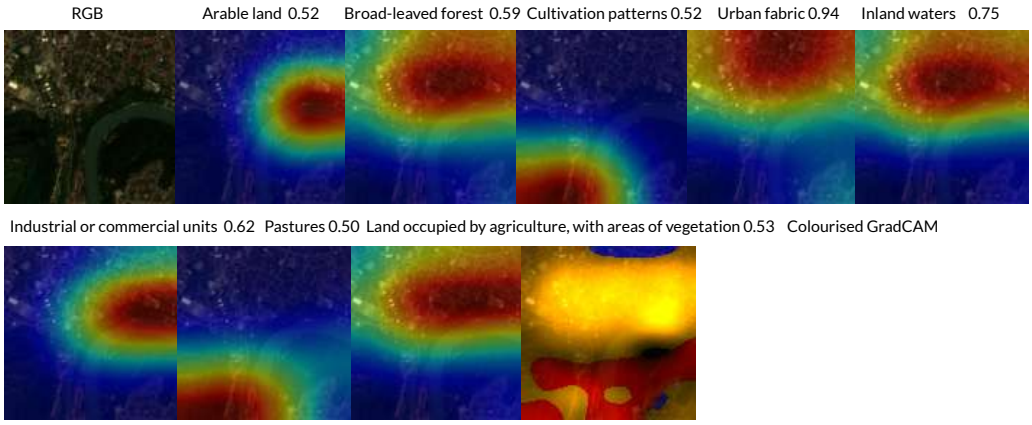


Figure E.2. Patch T33UUP_32_52 ResNet50 Grad-CAM

Figures E.1 and E.2 represent the heat-maps on the same test patch, comparing CustomModelV9 to ResNet50. In ResNet50's case, attention is much more diffused once again and often misaligned. In contrast, CustomModelV9 exhibits sharper, more concentrated focus on meaningful spatial regions. Additionally, ResNet50 over-predicts by assigning extra labels not present in ground truth, while CustomModelV9 correctly predicts only true labels, further highlighting V9's superior spatial precision.

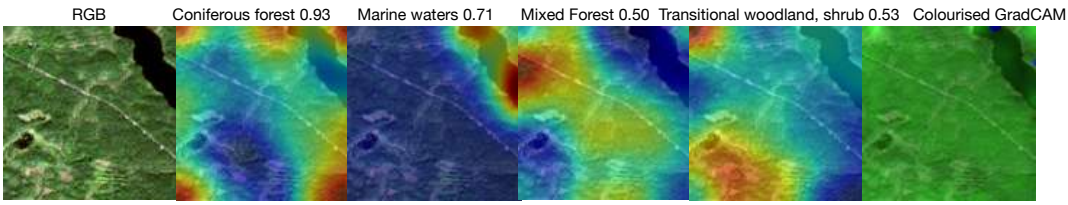


Figure E.3. Satellite Finland patch CustomModelV9 Grad-CAM

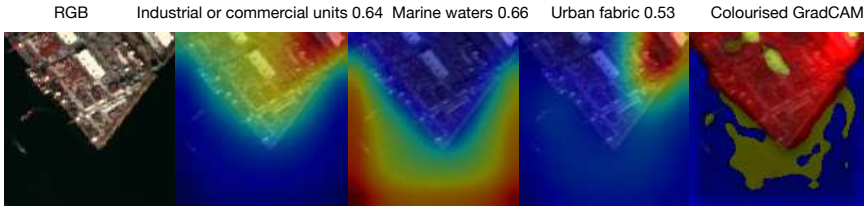


Figure E.4. Satellite Tokyo, Japan Patch CustomModelV9 Grad-CAM

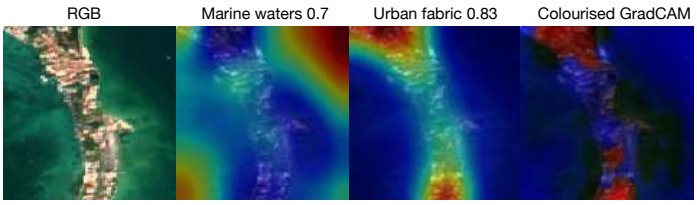


Figure E.5. Satellite Lago Di Como, Italy Patch CustomModelV9 Grad-CAM

Figures E.3 to E.5 display Grad-CAM heatmaps for three test samples retrieved from three real-world locations, Finland, Japan, and Italy via the Flask interactive prediction map (Appendix D). These patches are not part of the BigEarthNet dataset, yet CustomModelV9 correctly predicted the land cover classes in each case. The heatmaps show well-localised and confident activation regions which align with the actual spatial structure of the patches, further indicating the model's ability to generalise on unseen data. However, a noticeable drop in prediction confidence is observed, with several probabilities lying closer to the decision threshold, indicating higher uncertainty when applying predictions outside the original data distribution.

E.3.2 Batch Prediction Images

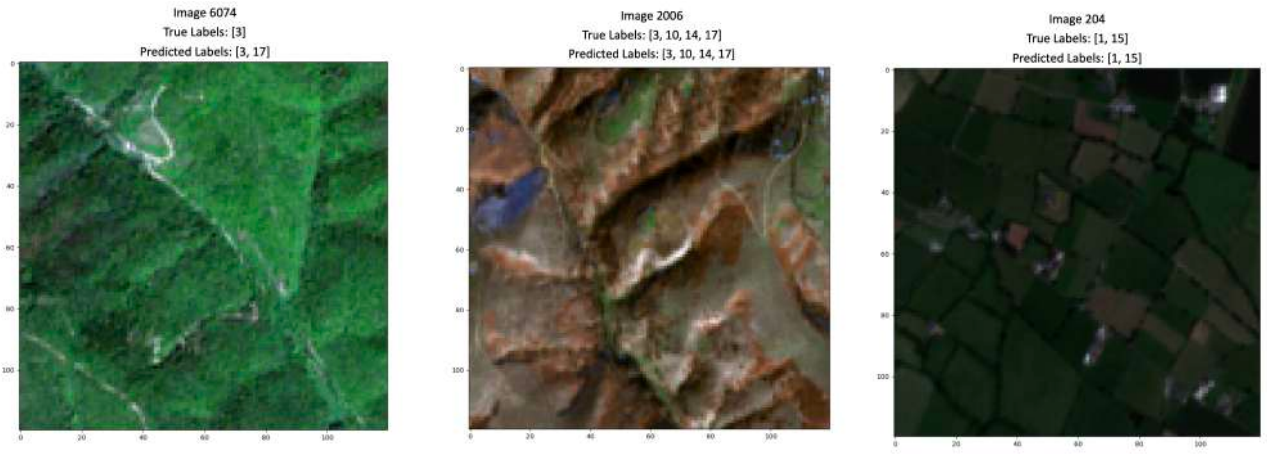


Figure E.6. Sample Batch Predictions

Figure E.6 presents three randomly selected BigEarthNet image patches, comparing predicted and ground truth labels. RGB composites were reconstructed using *Rasterio* by using Sentinel bands B04, B03, and B02.

- Left Image (ID: 6074): The model correctly predicts "Broad-leaved forest" (class 3) but also includes "Transitional woodland, shrub" (class 17), which is a closely related vegetation type. This form of over-prediction suggests that the model was slightly uncertain about dense and transitioning vegetation, which is acceptable given the spectral and visual similarity between the two.
- Middle Image (ID: 2006): The image depicts a more complex scene, containing a true combination of "Broad-leaved forest", "Land principally occupied by agriculture, with significant areas of vegetation", "Natural grassland and sparsely vegetated areas", and "Transitional woodland and shrub" (classes 3, 10, 14, 17). The model correctly predicted all four classes without any false positives, indicating the capacity to capture more detailed multi-label relationships between various spectral variations.
- Right Image (ID: 204): The image depicts another successful prediction, where both "Arable land" and "Pastures" (classes 1 and 15) were identified correctly. This indicates that the model can distinguish between two agriculturally intensive land

types, which have overlapping spatial textures and seasonal patterns, indicating strong prediction capabilities for agricultural domains.

E.3.3 Per-Label Confusion Matrices Grid

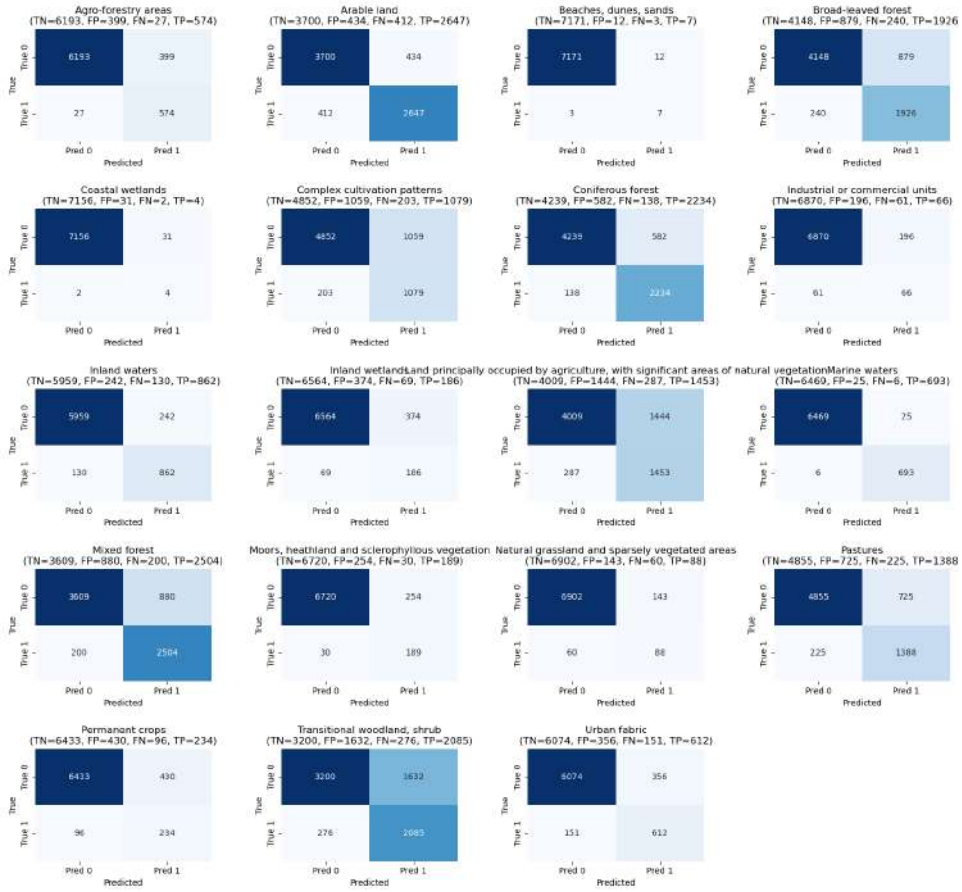


Figure E.7. Per-Label Confusion Matrices

Each 2x2 matrix visualises the model's classification performance on individual land cover classes, showing True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN). High TP and low FP are observed in classes like "Marine Waters", "Arable Land" and "Coniferous Forest". In contrast, rare classes such as "Coastal Wetlands" and "Beaches, dunes, sands" exhibit high FN which is consistent with per-class performance (Appendix E.2) and per-class multi-label analysis in Section 5.3, highlighting challenges in distinguishing underrepresented labels.

Appendix F: System Architecture & Project Timeline

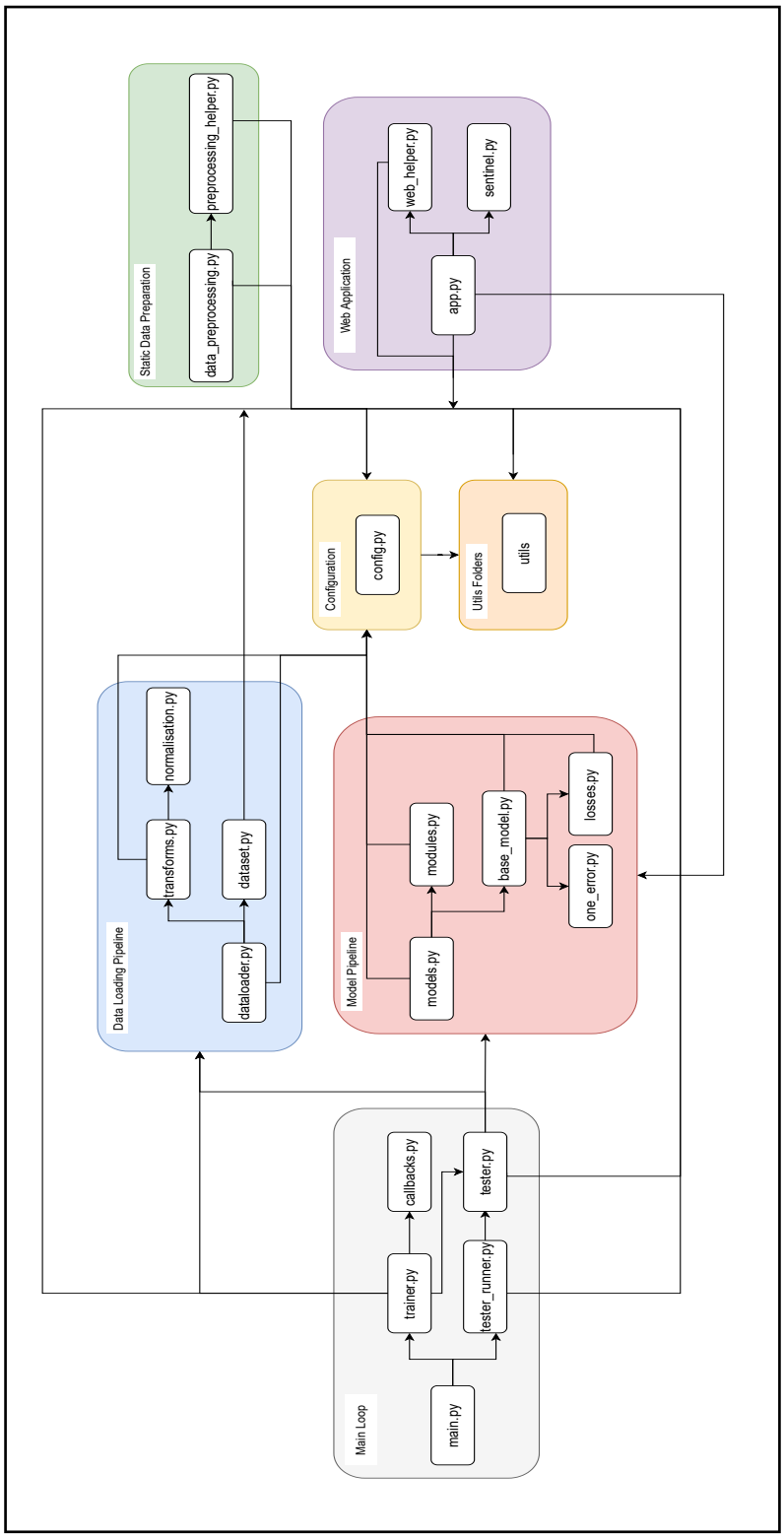


Figure F.1. System-Wide Architectural Flow Diagram

This flowchart provides a high-level overview of the system architecture, illustrating the interconnections between key Python files across the system's main components. The components are as follows:

I. **Main Loop:** Includes *main.py*, *trainer.py*, *tester.py*, *tester_runner.py*, and *callbacks.py*. This is the system's initiation, giving a start to the training and testing of the models. Upon execution of *main.py*, Figure F.2 is displayed, providing options to select a model, band combination, dataset percentage, weights, and execution mode (train only, test and test, or both), along with iteration options for running multiple configurations.

II. **Data Loading Pipeline:** This pipeline comprises *dataloader.py*, *dataset.py*, *transforms.py*, and *normalisation.py*. This pipeline handles data loading and the on-the-fly preprocessing phase, as described in Chapter 4.2.

III. **Model Pipeline:** This pipeline includes *models.py*, *modules.py*, *base_model.py*, *one_error.py*, and *losses.py*. It is responsible for defining the model architectures, custom modules, loss functions, and metrics used in the system.

IV. **Configuration:** The configuration file *config.py* contains the central configuration settings required by all pipelines within the system, allowing for a centralised workflow.

V. **Utils:** The utils folder stores the various utility functions used by various files within the project.

VI. **Web Application:** The web application comprises *app.py*, *web_helper.py*, and *sentinel.py*. It is responsible for implementing the web-based interface for predictions and visualisations, as described in Appendix D.

VII. **Static Data Preprocessing:** Consists of *data_preprocessing.py* and *preprocessing_helper.py*, and is in charge of the static preprocessing phase as described in Chapter 4.2

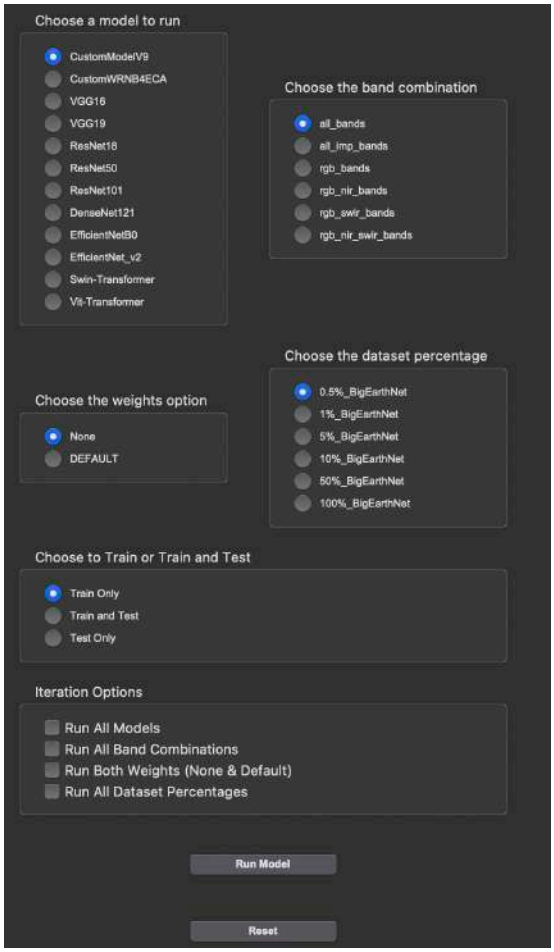


Figure F.2. System Execution Interface

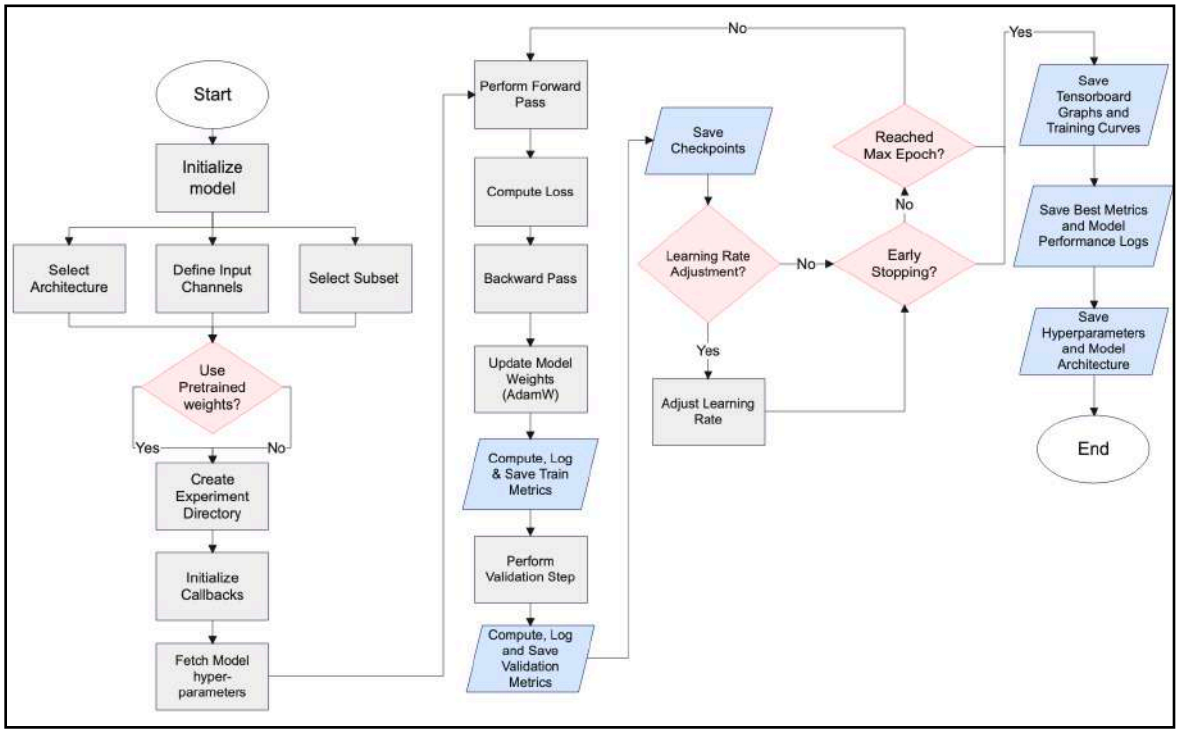


Figure F.3. End-to-End Training Procedure for RS Image Classification

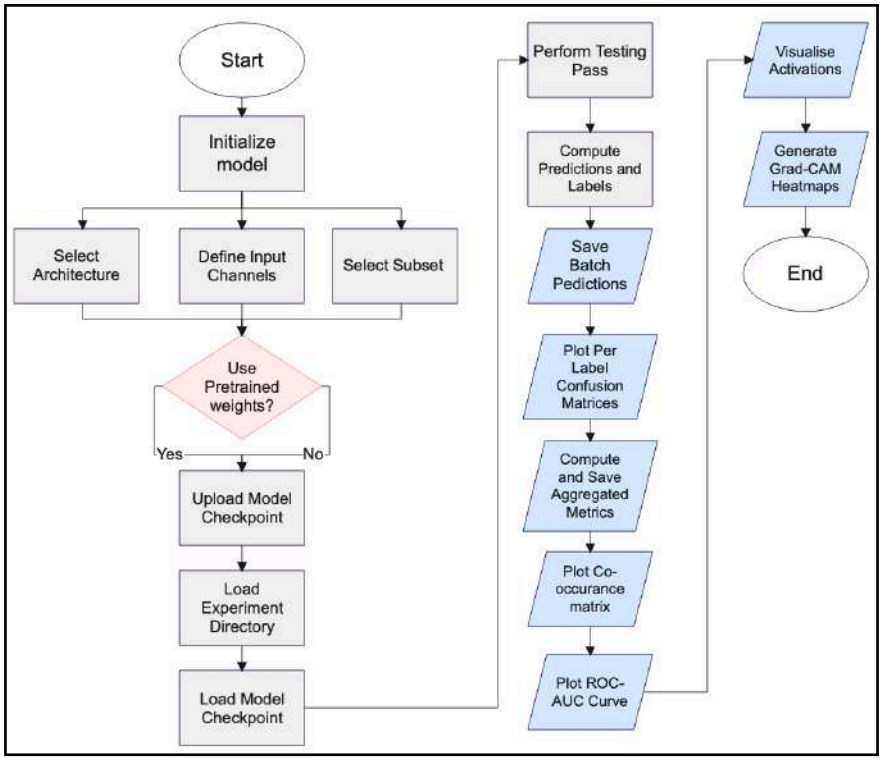


Figure F.4. End-to-End Evaluation Procedure for RS Image Classification

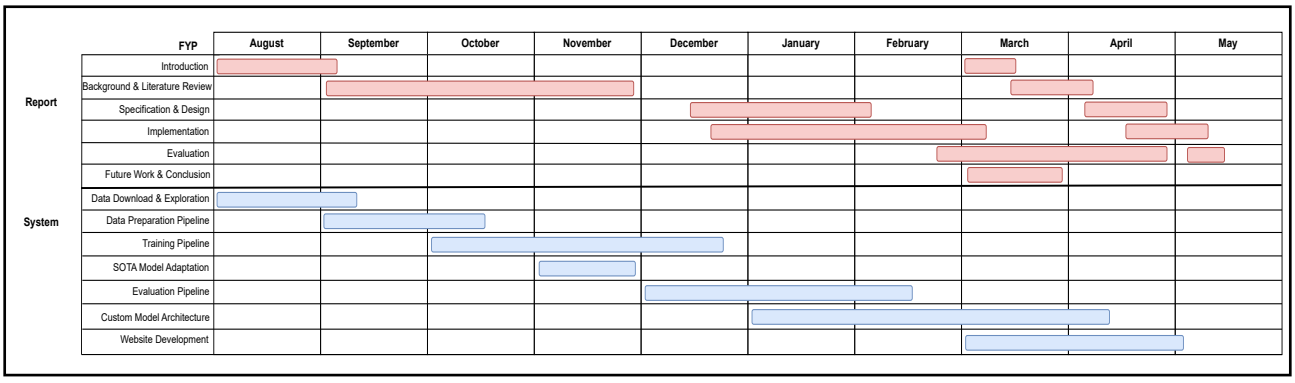


Figure F.5. Gantt Chart Project Timeline Overview